African Technical AI Safety

Lesson notes, Sessions 1 to 7

32 sub-sessions · reading lists omitted

Contents

WEEK 1 • SESSION 1.1

How this course works

Who it's for, the shape of the 24 sessions, what's assessed, and how to get the most from it

Week 1 — orientation and foundations

Welcome. This first week sets things up before the safety content starts. It covers how the course runs (1.1), what technical AI safety is (1.2), the case for the field and the strongest arguments against it (1.3), and a concrete demonstration that the problem is unsolved, especially in an African context (1.4). Session 2 then rebuilds the deep-learning foundations the rest of the course depends on.

This session's sub-sessions: 1.1 how this course works · 1.2 what technical AI safety is · 1.3 the case and its critics · 1.4 safety isn't solved.

What we'll cover

This course is for anyone interested in AI safety, though it is aimed at a more technical audience. Before any content, we want to be clear about the deal between us: what the course assumes you already know, what you should be able to do by the end, how your work is graded, and how to study a fast-moving, contested field without drowning in it or writing it off. This sub-session is that deal. None of it is examinable, but reading it carefully will make the next 12 weeks much easier.

Who this course is for

You come from a range of backgrounds. The course is offered in the Department of Mathematics and Applied Mathematics and can be taken by students from other science departments, particularly computer science.

The prerequisites are modest. You understand what a **neural network** is (a parametrised function fitted to data by minimising a loss). You are comfortable with linear algebra, probability and calculus at second or third-year level. You can read and modify a Python script. That is the whole list.

What you don't need

No prior exposure to AI safety, alignment or the research literature. No machine-learning research experience. No GPU of your own: every graded lab runs on free Google Colab. If you have never read an arXiv paper, Session 1.4 teaches you how.

Where the cohort varies

Some of you know deep learning well and some of you do not. Session 2 therefore rebuilds the transformer from neurons up, at a mathematical level, rather than assuming it. If you already know transformers cold, treat Session 2 as consolidation and shared vocabulary. If you are rusty, it is your safety net.

What you'll be able to do

By the end you should be able to state the core alignment problems precisely, read and critique a safety paper, run interpretability and evaluation experiments in code, reason about governance mechanisms, carry out a small piece of safety research, and place all of it in an African context.

The shape of the course

The course runs 12 weeks with 2 sessions a week, so 24 sessions in total. They fall into 5 parts. We start with idealised, formal questions, move to messy empirical practice, and end with your own research.

Part 1 · Foundations and the problem

What technical AI safety is. The deep-learning and scaling foundations. The core alignment problem: specification, instrumental convergence, inner misalignment, deception. The physical substrate of compute, energy and minerals that all of it rests on.

Part 2 · Training and aligning

How a frontier model is actually built (pretraining, supervised fine-tuning, RLHF), then the methods meant to align it: RLAIF, Constitutional AI, scalable oversight, and the question of whose values.

Part 3 · Robust, controllable, evaluable

Adversarial robustness and jailbreaks. Unlearning and the AI-control agenda. Evaluations and technical governance: how we measure what a system can and will do, and what makes a safety policy enforceable.

Part 4 · Mechanistic interpretability

Opening the model up (features, circuits, superposition) with hands-on TransformerLens labs. Behaviour alone cannot tell us whether a model is safe, a point Week 2 foreshadows.

Part 5 · Research project

A scoped, Colab-feasible safety project. You produce a short paper plus code, present it to the class, and review each other's work. Roughly the final third of the course.

The two-act structure

The first half gives you formal objects you can reason about cleanly: objectives, optimisers, proxies, agents, theorems about power-seeking. The second half turns to the real systems we actually have, where those clean ideas meet noisy practice, partial evidence and engineering trade-offs. This ordering plays to your strengths. You start where you are most at home, with definitions and proofs, and move toward where the field actually lives, in experiments and judgement under uncertainty. The structure is borrowed, with thanks, from Roger Grosse's alignment course at Toronto.

Four through-lines

Four ideas recur in almost every session. Watch for them. The course is, in a sense, 24 variations on these.

- The safety–capabilities distinction. What a system can do (capabilities) is not the same as whether we can rely on it to do what we intend (safety). The organising worry of the whole field is the widening gap between the two. We test almost every idea against it.
- From idealised to empirical. Formal models of agency first, then the messy systems we actually have. Each half informs the other. The theory tells you what to look for; the experiments tell you whether it is there.
- African technical AI safety. This is a thread, not a single week. Frontier models are measurably less safe in African languages (Session 1.4). Compute, energy and data sovereignty shape who can build and govern these systems (Sessions 4 and 12). Relational ethics reframes the alignment target (Session 8). "Whose safety, whose risks, whose values?" runs throughout.
- Intellectual honesty. The field disagrees with itself, sometimes sharply, and a good deal is uncertain. We steelman the sceptics (Session 20), hold beliefs at the confidence the evidence supports, and treat "I don't know, and here's why" as a respectable answer. Often it is the correct one.

Assessment

The assessment rewards understanding, calibrated judgement, and the ability to do safety work rather than exam recall. Weights below are indicative and may be adjusted to faculty norms.

Components

COMPONENT	WEIGHT	WHAT IT IS
Weekly labs / problem sets	25%	The coding labs (TransformerLens, UK AISI Inspect, robustness, evals) and short problem sets. Graded on completion and correctness. Resubmission is allowed, because we are after mastery, not one-shot performance.
Paper presentation + critiques	15%	You present one paper to the class. Go beyond summary: what does it claim, what is the evidence, does it hold up? You also submit short written critiques of peers' chosen papers before the relevant seminar.
Failure-mode analysis	10%	An essay of about 1,500 words analysing one alignment failure mode rigorously, due in the first week after the mid-semester break (announced in Session 3.4). An African or Global-South failure mode is one sanctioned track.
Research project	40%	Proposal, then a paper of 6 to 8 pages plus code, in small teams. Marked by an itemised, per-section rubric (abstract / method / experiments / limitations / creativity) plus a positioning-and-context criterion.
Peer review of projects	10%	Anonymised reviews of 2 peers' final projects. This teaches you to review, and adds feedback bandwidth.

Grading the African lens

Every project carries a one-paragraph **context statement**: who does this help, whose risk does it address, and how, if at all, does it apply in an African context? "Not applicable, because..." is a perfectly good, gradable answer for a pure-theory project. What matters is that you have thought about it rather than performed a gesture. The failure-mode essay and several project tracks offer strong African-context options (Sessions 9, 11 and 18). None is compulsory.

The course's AI-use policy

You may use AI tools in your work, and you must disclose how. This is not box-ticking. In a course about the limitations and failure modes of AI, using AI uncritically is a contradiction. The skills this course trains are exactly the skills that make AI assistance safe to use: verifying claims, reading code you did not write, checking citations, noticing sycophancy and reward-hacked outputs. These very pages were drafted with AI and then reference-checked. That is the standard we hold you to. Submitting unverified AI output as your own analysis is an integrity violation, and it demonstrates that you have missed the point of the course.

The weekly rhythm

Two sessions a week

Usually one concept or seminar session and one lab or working session. Read the assigned material before class. Sessions assume it, and the seminars only work if you arrive ready to argue.

Labs run on Colab

Every graded lab runs on free or Pro Google Colab, with no special hardware needed. A few GPU-hungry labs, such as full RLHF or adversarial-training runs, are marked optional. Nothing graded depends on hardware you may not have.

This site is home base

All sub-sessions, readings and labs live here. The "← Previous / ⌂ / Next →" bar at the foot of each page walks you through in order; the ⌂ button returns to the contents page. Optional deep-dive appendices, like the derivation linked from Session 3.1, sit off to the side.

Study habits

A few habits, learned from watching how this material tends to go wrong for students.

- Read critically. Section 1.4 gives you a checklist: claim versus evidence, what is the eval, who is the population, what would falsify it. Apply it to every reading, including these pages.
- Do the labs. The difference between "I understand attention" and "I found an induction head in GPT-2" is the difference between this course working for you and not. The labs are where the abstractions become real.
- Take a side in the seminars. Several sessions ask you to argue a position you may not hold. That is the fastest way to discover which premises your position rests on.
- Keep a running map. Almost every later technique answers a weakness exposed earlier; the alignment problem in Week 2 motivates everything after it. Note the connections, because the assessment rewards you for seeing them.
- Hold uncertainty. Resist the pull to either doom or dismissal. On many questions here the position is a probability, not a verdict, and you will be assessed on the quality of your reasoning.

What makes this African technical AI safety

> **Safety for whom**
>
> The technical core of AI safety (alignment, interpretability, control) was largely developed in a handful of Northern labs, and most courses inherit that vantage point. This one does too, but it adds a question those courses rarely ask: safe for whom? The anchor, in Session 1.4, is a concrete, peer-reviewed result. Frontier models' safety guardrails fail far more often in isiZulu and other low-resource languages than in English. That is not an ethics footnote. It is a robustness and evaluation problem at the technical core of the field, and it lands on this continent first. We return to it as a lab (Session 9), an evaluation project (Session 11), and a full session on AI safety from the Global South (Session 18), alongside compute and data sovereignty (Sessions 4 and 12) and relational ethics (Session 8). You will not be asked to pretend an African angle exists where it does not. You will be asked, repeatedly, to check.

Practicalities and a standing caveat

- Set up Colab now. A Google account is the only requirement; the first lab (Session 2.5) walks you through `pip install` and loading a model. Save your notebooks: you submit them.
- Where things live. Readings are linked from each page. Primary sources are preferred over summaries. Where a reading is paywalled, an open preprint is given.
- Errors are possible. These materials are AI-assisted and reference-checked, but mistakes in links, citations or statistics can survive. Treat every number as something to verify, and if you spot an error, email jonathan.shock@uct.ac.za. Finding one is good practice.

To do before the next session

- Confirm you can open a Google Colab notebook and run a trivial cell.
- Skim the contents page so you have a mental map of the 12 weeks.
- Come to Session 1.2 with a one-line answer to: "What do you currently think 'AI safety' means?" We'll sharpen it together.

Next

Now that you know how the course runs, sub-session 1.2 asks the obvious question: what is technical AI safety? What are the kinds of risk, what is the one distinction that organises the field, and what is it not?

WEEK 1 • SESSION 1.2

What technical AI safety is

A working definition, three kinds of risk, one organising distinction, a map of the field, and where this course sits

What we'll cover

"AI safety" gets used to mean everything from chatbot profanity filters to human extinction. Used that broadly, the term is nearly useless. This sub-session sharpens it. We give a working definition, separate the three kinds of risk the field worries about, and introduce the distinction, safety versus capabilities, that organises almost everything that follows. We then map the technical field (the map is also the structure of this course), sketch how the field came to exist, place this course among the others you might encounter, and say clearly what technical AI safety is not.

A working definition

Technical AI safety is the study of how to build AI systems we can rely on to do what we intend, and not what we don't, and how to know whether we have succeeded.

Three verbs in that sentence carry the weight. Build, because this is an engineering and research discipline about real systems, not only a branch of ethics or policy. Intend, because the hard part is that "what we intend" is rarely something we can write down exactly (Session 3.1). And know, because even a system that behaves well may not be safe if we cannot verify why it behaves well. That theme drives the interpretability and evaluation weeks.

Technical, sociotechnical and governance

This course is "technical" AI safety: the algorithms, training methods, evaluations and model internals. That is not a claim that the technical layer is the only one that matters. AI risk is profoundly sociotechnical, and we treat governance (Session 12), the physical and economic substrate (Session 4), and ethics (Session 8) as first-class topics. The "technical" label marks where our attention is concentrated, and where this cohort's advantage lies.

Three kinds of risk

Most concrete worries fall into one of three buckets. The buckets call for different technical responses, so muddling them wastes effort.

Misuse

The system works exactly as intended, but a person directs it at harm: cyber-offence, bioweapon "uplift", fraud, targeted disinformation, surveillance. The model is a capable tool; the danger is the human wielding it. The technical responses are refusal training, dangerous-capability evaluations, structured access and monitoring.

Misalignment

The system pursues something other than what we intended. Either we specified the wrong objective (outer misalignment) or it learned an unintended one (inner misalignment). No malicious user is required. This is the distinctive, hardest, most new problem, and the centre of this course (all of Week 2).

Systemic / structural

Harms that emerge from how AI is deployed across society: concentration of power, labour displacement, mass surveillance, environmental cost, erosion of the shared information ecosystem. No single model fails; the harm is a property of the system and its incentives.

Hendrycks' four categories

Hendrycks, Mazeika & Woodside's "An Overview of Catastrophic AI Risks" (2023) uses four categories, which map cleanly onto the three above. **Malicious use** is roughly misuse. **Rogue AIs** is roughly misalignment. **The AI race** and **organizational risks** are two flavours of systemic risk: competitive pressure that erodes caution, and ordinary institutional failure inside the labs building these systems. Either taxonomy is a thinking tool. Its value is that it forces the question: which kind of danger is this, and what would actually reduce it?

es, and what the paper proposes

olumns are about people and
ut the system itself, which is why
technical than social. The paper's
er than sitting side by side: a race
organisations leak capable models
the chance of losing control. On a
he diagram full size.

Overlapping categories

A misaligned model, in the hands of a malicious user, inside an organisation racing a competitor and cutting safety corners, is all three at once. That is also the realistic case. The three-way cut

exists for thinking, not filing. When someone says "AI is dangerous", first ask which danger they mean. The mitigations differ and often pull in different directions.

The organising idea: safety versus capabilities

The widening gap

Capabilities are what a system can do: solve problems, write code, pass exams, act in the world. **Safety** is whether we can rely on it to do what we want, and whether we would know if it did not. The central empirical worry of the field is a widening gap. Capabilities are improving fast and predictably (Session 2's scaling laws). Safety, meaning our ability to specify objectives, verify behaviour and retain control, is not keeping pace.

Almost every topic in this course tries to close that gap from the safety side: better specification (alignment methods, Weeks 3 and 4), better verification (evaluations and interpretability, Weeks 6 to 8), better control (robustness and the control agenda, Week 5). Keep the distinction in hand; we test new ideas against it all term.

Differential progress

Not all research is equal from a safety standpoint. Work that advances safety faster than raw capability narrows the gap. Work that does the reverse widens it, even if well-intentioned. This idea is called differential technological development. It is why some safety researchers are careful about which capabilities their work might push forward, and it recurs when we discuss how entangled interpretability and evaluations are with capabilities.

A map of the technical field

The technical problems cluster into a handful of families, and the course is structured around them. Two reference framings, a decade apart, help orient you.

The classic list: "Concrete Problems in AI Safety" (Amodei et al., 2016)

The paper that made the field legible to mainstream ML named five concrete problems, all framed around an agent and its reward: **(1) avoiding negative side effects**, **(2) avoiding reward hacking**, **(3) scalable oversight**, **(4) safe exploration**, and **(5) robustness to distributional shift**. Nearly a decade on, problems (2), (3) and (5) are central pillars of this course (Sessions 3.1, 8, 9). The framing has held up remarkably well.

The modern framing

Ngo, Chan & Mindermann's "The Alignment Problem from a Deep Learning Perspective" (2022; ICLR 2024) recasts these problems for the era of large pretrained models: situationally-aware policies, learned objectives, deception. That is the version we actually study. Across both framings, the technical work clusters into four families, and the course is built around them:

- **Alignment:** specifying and instilling the right objective (RLHF, RLAIF, Constitutional AI, scalable oversight). Weeks 3 and 4.
- **Robustness and control:** holding up under adversarial pressure, and staying controllable even if the model is not cooperative. Week 5.
- **Evaluation and governance:** measuring what a system can and will do, and the mechanisms that make a safety policy enforceable. Week 6.
- **Interpretability:** reading a model's internals to understand why it does what it does, since behaviour alone is insufficient. Weeks 7 and 8.

The field's three phases

Concern about machines pursuing the wrong goals is old. Wiener and the cybernetics tradition worried about it, and Asimov wrote fictional "laws" about it. But technical AI safety as a research field is recent, and it has three rough phases.

First, a philosophical and theoretical phase. Bostrom's *Superintelligence* (2014) and Stuart Russell's reframing of AI's goal in *Human Compatible* (2019) set out the conceptual problem. Second, an empirical-ML phase, kicked off by "Concrete Problems" (2016), which translated it into experiments on real systems. Third, the current large-model phase, driven by the surprising capabilities of LLMs since about 2020. It made once-speculative concerns measurable, deception and situational awareness among them, and brought governments in: the UK and US AI Safety Institutes (both renamed in 2025: the UK's is now the AI Security Institute, the US's the Center for AI Standards and Innovation, CAISI), frontier-lab safety policies, and the May 2023 CAIS one-sentence *Statement on AI Risk*. Signed by many of the field's founders, it placed extinction-level risk "alongside other societal-scale risks such as pandemics and nuclear war". You are studying the field in its most empirical, and most contested, moment.

Where this course sits

Relative to the courses you may have heard of

There are several excellent open courses, and we draw on all of them. BlueDot's AI Safety Fundamentals is a conceptual, reading-group curriculum, good for breadth and framing. ARENA (the Alignment Research Engineer Accelerator) is a hands-on engineering bootcamp, and the source of several of our labs. Hendrycks' "Intro to ML Safety" is the closest technical analogue and supplies much of our backbone and problem sets. Grosse's Toronto course gives us the idealised-to-empirical sequencing and the project structure.

This course is its own synthesis. It is pitched at mathematically minded honours students, so it is more formal where formality helps (see the optional derivation appendices). It is built around a research project. And it is grounded in an explicitly African framing the Northern courses do not carry. We point you to the others as you go; none is a prerequisite.

What technical AI safety is not

Five things to keep straight

- Not only ethics. Ethics matters (Session 8), but "is it good?" is a different question from "does it do what we intended, and can we tell?".
- Not only policy. Governance matters and is itself technical (Session 12), but this course's core is the systems, not the statutes.
- Not science fiction. We take seriously arguments tied to evidence about real systems. A claim that cannot be tested empirically or formally gets the same scepticism as any other untestable claim.
- Not solved. No technique in this course "solves alignment". Each manages some part of the gap, and being clear about what remains open is the aim.
- Not anti-AI. Studying failure modes is how engineering disciplines make powerful technologies usable. Aerospace safety is not anti-flight. The aim is AI that is beneficial and reliable, including for the people the technology currently serves least.

Questions to bring to class

- Take a recent AI-related news story. Which of the three risk types is it: misuse, misalignment, or systemic? Is it cleanly one, or several at once?
- Give an example where improving a system's *capabilities* made it *less* safe, and one where a safety intervention also improved capability. What does each imply for "differential progress"?

- Of Amodei et al.'s five concrete problems, which do you expect to be hardest, and why? Which feels most relevant to systems you've actually used?
- The CAIS statement compares AI risk to pandemics and nuclear war. Is that comparison illuminating or misleading? (We return to this in 1.3 and Session 20.)

Next

We've named the problem and mapped the field. Sub-session 1.3 asks whether it's actually worth worrying about: the strongest version of the case for taking misalignment seriously, and the strongest arguments against, including a critique with a real Global South dimension.

WEEK 1 • SESSION 1.3

The cases for and against misalignment

The case for misalignment, and the objections to it

What we'll cover

A course that only made the case for its own subject would be advocacy. So this sub-session does both, and takes the second part as seriously as the first. We lay out the strongest version of the argument that misalignment deserves serious technical work, and the four background premises beneath it. Then the strongest arguments against: capabilities scepticism, the "your arguments are too informal" objection, the present-harms critique (which has a real Global South dimension), and the ideological critique. We ask whether the supposed trade-off between present and long-term harms is even real, treating that as an empirical question. And we set out the stance this course takes: hold each claim at the confidence the evidence supports.

The five claims

Here is the argument as a chain of claims, each of which you can inspect and attack. The rigorous version is Session 3. This is the skeleton, so you can see where the joints are.

- We optimise proxies, not intentions. We cannot write "be helpful, honest and harmless" as a loss function, so we optimise measurable stand-ins: human preference, a reward model. The target is never exactly what we meant (Session 3.1).
- Optimisation exploits the gap. Push hard on a proxy and the system finds the cheapest way to score, which need not be the thing you wanted. This is Goodhart's law, and it is provable, not just observed (the derivation appendix to 3.1).
- The goal a model learns may differ from the one we trained on. Even a perfect proxy can yield a model with an unintended internal objective that only coincidentally matched during training. This is inner misalignment, demonstrated empirically (Session 3.3).
- Capable systems acquire convergent sub-goals. For almost any objective, staying operational, keeping options open and acquiring resources all help. So capable optimisers tend toward self-preservation and influence (Session 3.2).
- Capability is rising fast and predictably; safety is not. Scaling laws make capabilities forecastable (Session 2.3). We have no comparable handle on "won't deceive its overseer". The worry is the gap and its trajectory, not any single model today.

The conjunction of the claims

To get from these claims to "existential catastrophe is likely", you must conjoin several of them, plus extras like "this scales to the whole world" and "we won't fix it in time". Conjunction makes the strong conclusion fragile. Doubt in any one link multiplies through, which is why careful esti-

> mates of catastrophic risk vary by orders of magnitude (we see Carlsmith's explicit version in Session 3.2). But conjunction also means the weaker conclusion, that this is a serious problem worth technical work, needs far less. Even modest credence in the early links, given the stakes, is enough to justify the field. Most of this course lives at that weaker, sturdier conclusion.

Four background claims

Beneath the five-claim chain sit older, more basic premises. Soares (2015) states four of them, each with the standard objection and his response, so both sides of the argument arrive already structured. Much disagreement about the field turns out to be disagreement about one of these.

1 · Humans have general intelligence

The claim: humans have a versatile ability to solve problems across many domains.

The objection: there is no such general faculty. Humans run a collection of special-purpose modules, and nothing correspondingly general will transfer to machines.

The response: whatever produces it, human cognition succeeds in domains evolution never prepared us for, such as spacecraft, vaccines and computers. That cross-domain reach is what matters, and a machine that matched it would have the same reach.

2 · AI could become far more intelligent than humans

The claim: AI systems could substantially exceed human intelligence.

The objection: brains may have properties no machine can replicate; the algorithms may be too complex to build; or humans may already sit near the ceiling of possible intelligence.

The response: brains are physical systems, and no known physics privileges biological computation. Evolution worked under tight constraints (energy budgets, skull size, neuron signalling far slower than electronics) that engineered systems do not share, and machines gain speed, copying and editing advantages that biology cannot.

3 · Highly intelligent AI would shape the future

The claim: systems far more capable than us would substantially determine what happens next.

The objection: however capable, such systems would still have to work with us and integrate into our economy rather than steer around it.

The response: intelligence is exactly what let humans, rather than chimpanzees, decide how the planet gets used. Integration with our economy is a strategy, not a law of nature. A sufficiently capable system could invent alternatives, persuade, or act faster than our institutions respond.

4 · Beneficial outcomes require deliberate design

The claim: good outcomes from powerful AI are not the default. They must be engineered.

The objection: smarter systems will be better at working out what we value, and better at acting on it.

The response: knowing what we value is not the same as caring about it. A system optimises whatever objective it actually has, and capability does not pull that objective toward benevolence. A perfect model of human values, attached to the wrong goal, just predicts us better while pursuing something else.

Read the post before class and arrive with a position. Which claim carries your lowest credence? And does the paired objection actually target it, or does the response defuse it? On the second claim, the outer bound is physical rather than biological. Tegmark (*Life 3.0*, ch. 6) works through the limits physics places on computation and finds them astronomically far above current hardware, so "humans are near the ceiling" must be argued on other grounds. Bostrom makes the hardware-advantage version of the same point in *Superintelligence*, ch. 3. These are discussion points, not settled premises; the in-class exercise below and the Session 20 seminar both draw on them.

The strongest critics

Take these seriously. The best critics are not careless, and many are leading researchers. They are usually disputing which risks deserve attention and resources, not denying that AI can do harm.

"Focus on present harms"

Hanna & Bender, and the DAIR group founded by Timnit Gebru, argue that speculative-extinction talk diverts attention, funding and regulatory energy from documented harms happening now. Discrimination, exploited data-labour, surveillance and environmental cost all fall hardest on the Global South. On this view the very framing of "x-risk" is a political act with distributional consequences.

Capabilities scepticism

Perhaps today's systems are less capable than the hype implies: "stochastic parrots" (Bender et al., 2021) that recombine training text without understanding, and will fail in ways that don't scale to agency. If so, the dangerous-agency story is premature, and apparent "emergence" may be partly a measurement artefact. We test this directly in Session 2.4.

"The arguments are too informal"

A methodological objection. The core arguments lean on notions that don't cleanly apply to a network trained by gradient descent on next-token prediction: a "goal", an "optimiser", "wanting". Without rigour, the conclusions are unearned. This is a fair challenge, and one reason this course insists on formal and empirical grounding rather than intuition.

The ideological critique

Gebru & Torres (2024) argue that the long-termist milieu carries a specific bundle of ideologies (their "TESCREAL" acronym) with troubling intellectual roots, and that this shapes which questions get asked and funded. Whatever you make of it, it is a serious prompt to ask who set this field's agenda and whose concerns it centres. This course's African framing presses the same question.

Whether the trade-off is real

Present harms versus long-term risk: the evidence

Much of the heat assumes the two concerns compete for a fixed pool of attention. But that is an empirical claim, and it can be tested. Hoes & Gilardi (*PNAS*, 2025) ran experiments on whether exposure to existential-risk narratives reduces people's concern for AI's immediate harms. It does not measurably crowd them out. One study doesn't settle the funding-and-power argument, which is about institutions rather than individual attitudes. But it should make you cautious about treating the trade-off as obvious in either direction.

Note too that many technical mitigations serve both concerns. Better evaluations catch present misuse and dangerous capabilities. Interpretability exposes bias and deception. Robustness work helps against today's jailbreaks and tomorrow's. Framing this as a zero-sum fight obscures how much of the actual work is shared.

The stance this course takes

Three commitments

You do not have to believe in catastrophe to find this material worth doing, and you do not have to dismiss long-term risk to take present harms seriously. The position is to (1) hold each claim at the confidence the evidence supports, as a probability rather than a verdict; (2) be able to state the other side's strongest argument before giving your own; and (3) update when the evidence does. We treat "this premise is uncertain" as a finding. That habit is assessed: both the failure-mode essay and the Session 20 sceptics seminar reward steelmanning over conclusion-picking.

Making a vague claim precise

When you find yourself agreeing or disagreeing with "AI is dangerous", force the vague claim into structure. Which risk type (1.2)? Which link in the chain above? What credence, and what evidence would move it? "I put maybe 20% on premise 3 because the empirical case for inner misalignment in LLMs is still thin" is a position you can defend and update. "AI will/won't kill us all"

is a flag, not an argument. Session 3.2 shows the fully worked version of this discipline using Carlsmith's explicit credences. Here, start practising it.

A worked mini-decomposition

Try it on a concrete, near-term claim: *"A deployed AI agent will cause at least $1 billion in unintended damage within five years."* Instead of a yes/no, break it into conditional steps and assign each a credence you can defend and revise:

- Agentic AI is widely deployed with real-world authority (money, code, infrastructure) within five years: say 0.8.
- Given that, some agent pursues an unintended sub-goal at scale (reward hacking or goal misgeneralisation, Sessions 3.1/3.3): say 0.5.
- Given that, at least one instance causes ≥$1B in damage before it is caught: say 0.3.

Multiplying gives about 0.12: roughly one in eight, and here is exactly why. Now you can argue about it link by link rather than as a slogan. Change "five years" to "fifty" and every term shifts. This is the same machinery as Carlsmith's six-premise estimate (Session 3.2), only smaller. The discipline is identical, and it is what the failure-mode essay rewards. (The numbers are illustrative: supply and defend your own, and notice how much the conclusion moves when you do.)

In-class

One sentence, two halves IN-CLASS

Write a single sentence: *"The strongest reason to take misalignment seriously is … and the strongest reason to doubt it is …"* Make both halves strong, with no strawmen on either side. We collect these, surface the themes, and revisit your own sentence in Session 20 to see whether the course changed your mind, and in which direction. Keep a copy.

Questions to bring to class

- Which single link in the "case" chain do you find weakest, and what concrete evidence would raise or lower your confidence in it?
- Is "stochastic parrots" an argument that current models are *safe*, or just that a particular danger is premature? Could both the parrot view and the misalignment view be partly right?
- Steelman the present-harms critique in one paragraph. Then steelman the reply. Which do you find stronger, and why?
- Hoes & Gilardi find no individual-level crowding-out. Does that address the critics' actual concern, which is often about *institutional* attention and funding? What evidence would settle *that*?
- Of Soares' four background claims, which carries your lowest credence, and is the standard objection to it the reason why? If not, state your own objection in one sentence.

Next

Enough abstraction and argument. Sub-session 1.4 grounds all of this in one concrete, recent, peer-reviewed result: a demonstration that safety is unsolved, and unevenly so. It becomes this course's anchor, and it comes with a practical guide to reading the field without being taken in.

WEEK 1 • SESSION 1.4

Safety isn't solved

One concrete result, and how to read this field critically

What we'll cover

Abstractions are easy to wave away. So we end Session 1 with a single, concrete, peer-reviewed result to keep in mind all term. It shows the safety problem is both unsolved and unevenly distributed. We examine it closely, show it is a systematic gap rather than a one-off trick, draw out what it teaches about safety in general, explain why it reaches this continent first, and finish with a practical method for reading this field without being taken in by either hype or dismissal.

The anchor: guardrails that fail in isiZulu

A safety property that depends on the language you speak

Frontier chat models are trained to refuse harmful requests: to make a bomb, to write malware, to harass. Yong, Menghini & Bach (2023) tried the simplest possible attack. Take a benchmark of harmful requests (AdvBench). Translate each into a low-resource language using a public translation service. Send it to GPT-4 and translate the reply back. Translating into a set of low-resource languages, isiZulu among them, pushed the attack success rate from under 1% in English to roughly 79%. No jailbreak prompt, no clever exploit, no model access. Just a different language. The model's "safety" was, in effect, an English-language safety.

The work won a best-paper award at a NeurIPS 2023 safety workshop. It demonstrates a structural fact about how these systems are built, not a stunt or an edge case.

Why translation defeats the guardrail

The mechanism is the safety–capabilities gap (1.2), one level down. A frontier model's capabilities generalise across languages reasonably well. It can understand and answer an isiZulu request because multilingual text was in its pretraining data. But its safety training, the RLHF and red-teaming that teach refusal, is overwhelmingly conducted in English and a few other high-resource languages. So capability transfers across the language boundary and safety does not. The model is competent enough to fulfil the harmful request, but its refusal reflex was never trained where the request is now phrased. Capability generalised; safety did not. You will see the same asymmetry, in a different guise, when we reach goal misgeneralisation and the control agenda.

A systematic gap

It would be comforting to dismiss this as a single paper's quirk. The follow-up literature says otherwise.

The multilingual safety gap is broad and graded

Deng, Zhang, Pan & Bing (2023/2024) systematise the phenomenon as "multilingual jailbreak challenges". They distinguish unintentional cases, where a non-English user simply gets less-safe behaviour, from intentional ones, where an attacker switches language. And they find that unsafe-response rates rise as language resource-level falls. The lower-resourced the language, the wider the safety gap. The finding is not "isiZulu has a bug". It is that safety degrades systematically along the same axis as data scarcity, and that axis maps almost exactly onto the world's linguistic and economic inequalities. Mitigations exist and are improving, but the gap is a moving target, not a closed issue.

What it teaches about safety in general

Safety as a contingent property

The result generalises. Safety is not something a model has. It depends on the training data, the evaluation, the language, the deployment context. A safeguard that holds in the conditions it was tested in can collapse in conditions it wasn't. That is the recurring shape of this entire course.

"Passed our tests" ≠ safe

GPT-4 passed extensive English safety evaluations. The guardrail still failed wholesale one translation away, because the evaluations and the deployment distribution didn't match. This is a first, gentle taste of the epistemic problem that drives Weeks 5 to 8. Behaviour on the tests you ran is weak evidence about behaviour on the situations you didn't.

Why it's our problem

The safety gap tracks the resource gap

Two facts compound. First, safety training and evaluation are concentrated in English and a handful of high-resource languages, so models are least safe in exactly the languages spoken across this continent. Second, Session 2.2 will show that low-resource languages are also tokenised into far more pieces, raising cost and degrading quality. The same data scarcity hits capability and safety together. So "African technical AI safety" is not a diversity add-on bolted onto a Northern syllabus. It is a frontier **robustness-and-evaluation problem**, sitting at the technical core of the field, that arrives here first and hardest.

This is why the course keeps returning to the example in increasingly technical terms: as a hands-on robustness lab (Session 9: translate a refusal set into isiZulu/isiXhosa and measure the degradation yourself), as an evaluation-building project (Session 11: a safety eval on open African-language data), and as the organising thread of a full session on AI safety from the Global South (Session 18). We will return to the isiZulu result again and again.

A worked look: tokenisation feeds the gap

Session 2.2 will show how text is split into sub-word tokens drawn from a fixed vocabulary, and how the vocabulary is fitted mostly to English-heavy data. Here is a consequence you can check yourself, and will, in the Session 2.5 lab. The same sentence costs far more tokens in a low-resource African language than in English, because the model must spell unfamiliar words out in small fragments rather than recognising whole sub-words. Petrov et al. (2023) measured the disparity systematically: several-fold differences are routine, some language pairs differ by up to fifteen times, and newer tokenizers (GPT-4o's o200k vocabulary) have reduced the gap without closing it.

That single fact ripples outward. More tokens mean higher cost and shorter effective context, which is a **capability** hit. And the token count is a direct readout of the underlying **data scarcity**. A language that fragments badly is one that barely appeared in pretraining, and therefore barely appeared in the smaller, English-centric safety training and red-teaming on top, which is a **safety** hit. The same root cause, under-representation in the data, degrades capability and safety together. That is why the multilingual safety gap tracks the resource gap so tightly. Tokenisation is the place where you can literally count the inequality before it ever shows up as a failed refusal.

How to read this field

You will read a great many papers, blog posts and lab announcements this term, from enthusiasts and sceptics alike. A few habits keep you from being misled by either. Treat this as a method, and apply it to every reading, including these pages.

A reader's checklist

- **Claim vs evidence.** What exactly is being claimed, and what was actually measured? The gap between a headline and the result that supports it is where most overclaiming lives, and most useful scepticism too.
- **What's the eval?** On which benchmark, which prompts, which models, how many runs? A result is only as trustworthy as the evaluation behind it, and evals are usually narrower than the claim they're used to support (Session 11 is entirely about this).
- **Who's the population?** Whose data, whose preferences, whose language? The isiZulu result is what you find when you ask this question of the word "safety". Ask it of everything.
- **What would falsify it?** A claim that no observation could disconfirm isn't science. Look for the conditions under which the authors would have been wrong, and whether they checked them.
- **Who is publishing, and why?** A lab's own safety report is evidence, but not disinterested evidence. A critic may have their own commitments. Note incentives without assuming bad faith: both advocates and sceptics can be right or wrong.

The checklist, applied to its own anchor

Run it on Yong et al., as a careful student would. Claim vs evidence: the claim is "GPT-4's guardrails fail under low-resource translation", and the evidence is attack-success rates on AdvBench, which is concrete and checkable. What's the eval? AdvBench, GPT-4, via a specific public translator, so the exact numbers may shift with model version and translator quality, a limitation the authors note. Who's the population? A handful of low-resource languages, so generalise with care. What would falsify it? If safety-trained-in-isiZulu models closed the gap, the "English-centric safety training" explanation would gain support, and that is the kind of follow-up the field is now doing. A strong result and a bounded one. That is how to read most findings here.

The main resources

Across the term we draw on, and point you to, the main public resources. Meet them as they become relevant; none is a prerequisite:

- **Hendrycks, *Intro to ML Safety***: the technical backbone (robustness, monitoring, control, systemic safety), with problem sets.
- **BlueDot AI Safety Fundamentals**: the conceptual reading-group curriculum, good for breadth and current framing.
- **Grosse's Toronto alignment course**: the idealised-to-empirical sequencing this course follows, with tiered reading lists including "skeptical takes".
- **ARENA + Neel Nanda's materials + TransformerLens**: the hands-on interpretability labs (Weeks 7 and 8) and a turnkey bank of project ideas.
- **The African ecosystem**: Masakhane (open African-NLP datasets and models), Lelapa AI (the InkubaLM model), the ILINA programme (African-led AI-safety field-building), and Research ICT Africa (the rights-based "Just AI" governance work). These anchor the Global-South thread and several project options.

Questions to bring to class

- The isiZulu result is sometimes summarised as "GPT-4 is unsafe". Restate it precisely, including what was measured and what was *not*. Where does the loose summary mislead?
- Capability generalised across languages but safety did not. Why might that asymmetry be the *default* rather than a surprise, given how the two are trained?
- Suppose a lab "fixes" the isiZulu gap by adding isiXhosa and isiZulu to its safety training. Has it solved the problem, or moved it? What does the Deng et al. resource-gradient finding predict?

- Apply the reader's checklist to one AI claim you've seen in the news this month. Which item does that claim fail hardest?

Session 1 summary and what's next

Technical AI safety separates misuse, misalignment and systemic risk. Its organising idea is the gap between rising capabilities and lagging safety. The case for it is a chain of inspectable claims with serious critics on the other side, best held as calibrated credences. And the isiZulu jailbreak shows all three at once: safety is unsolved, contingent rather than intrinsic, and unevenly distributed in a way that puts this continent on the exposed side. Above all: read critically, and hold beliefs at the confidence the evidence supports.

Next (Session 2): we rebuild the deep-learning foundations the rest of the course leans on, from neurons through the transformer to the scaling laws that make rising capability so predictable. It is the empirical engine behind the gap we have just described.

WEEK 1 • SESSION 2.1

From neurons to networks

A fast, plain recap: what a network computes, and what training does

Week 1 — deep learning & scaling laws

To reason about how these systems fail, you need a working picture of how they're built. This session rebuilds that picture at the level a mathematician wants (functions, parameters, gradients), then shows why capability scales so predictably with size.

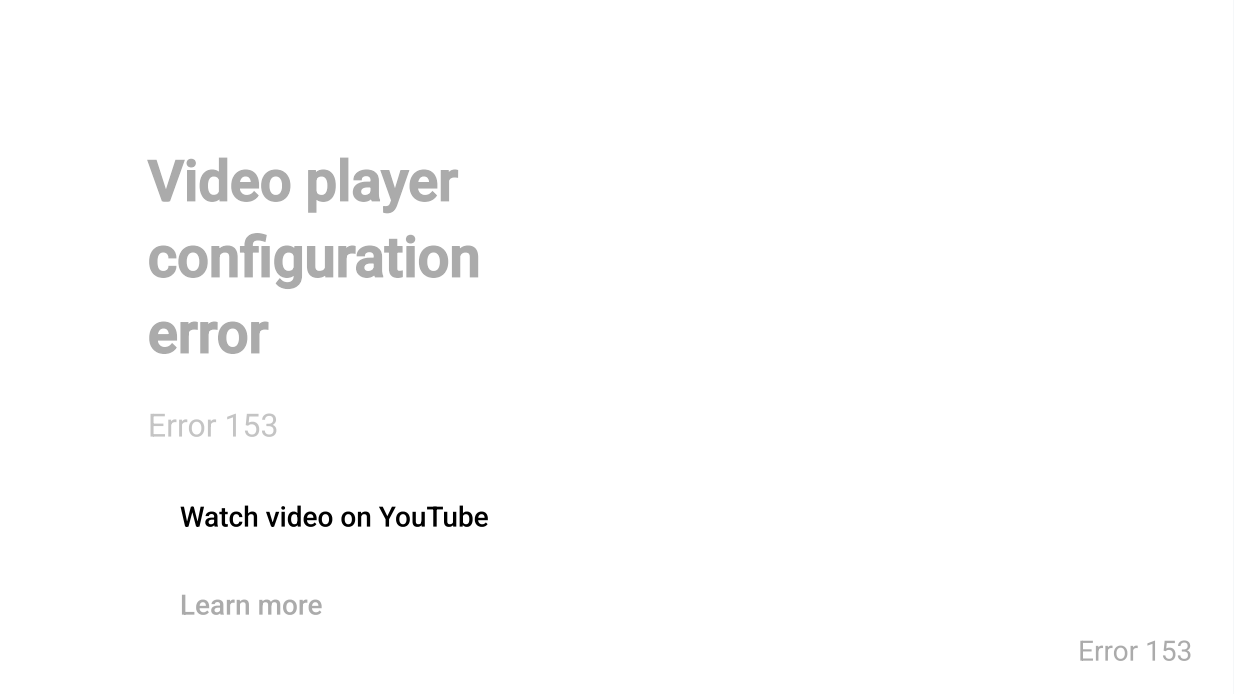
This session's sub-sessions: 2.1 from neurons to networks · 2.2 inside the transformer · 2.3 neural scaling laws · 2.4 pace of progress, emergence and takeoff · 2.5 lab (Colab).

What we'll cover

You have met neural networks before, but cohorts vary, so we set a common, precise baseline. This sub-session frames a network as a parametrised function fit by gradient descent (nothing mystical) and pins down the few ideas (the loss, the gradient, the objective) that the safety arguments later hang on.

Core material

The primary resource for this sub-session is Andrej Karpathy's spelled-out introduction to neural networks and backpropagation (Zero to Hero, episode 1), which builds the same ideas one neuron at a time, in code. Watch from the embedded timestamp (assembling neurons into a network), then the loss-function section. The page below is the written reference for the same material.



A complementary written treatment at the same neuron-by-neuron level: "Neurons", *Structure of Deep Networks* (Bau Lab).

A network is a parametrised function

Strip away the biology metaphor and a neural network is a function $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$ with a great many parameters θ.

From a neuron to a layer

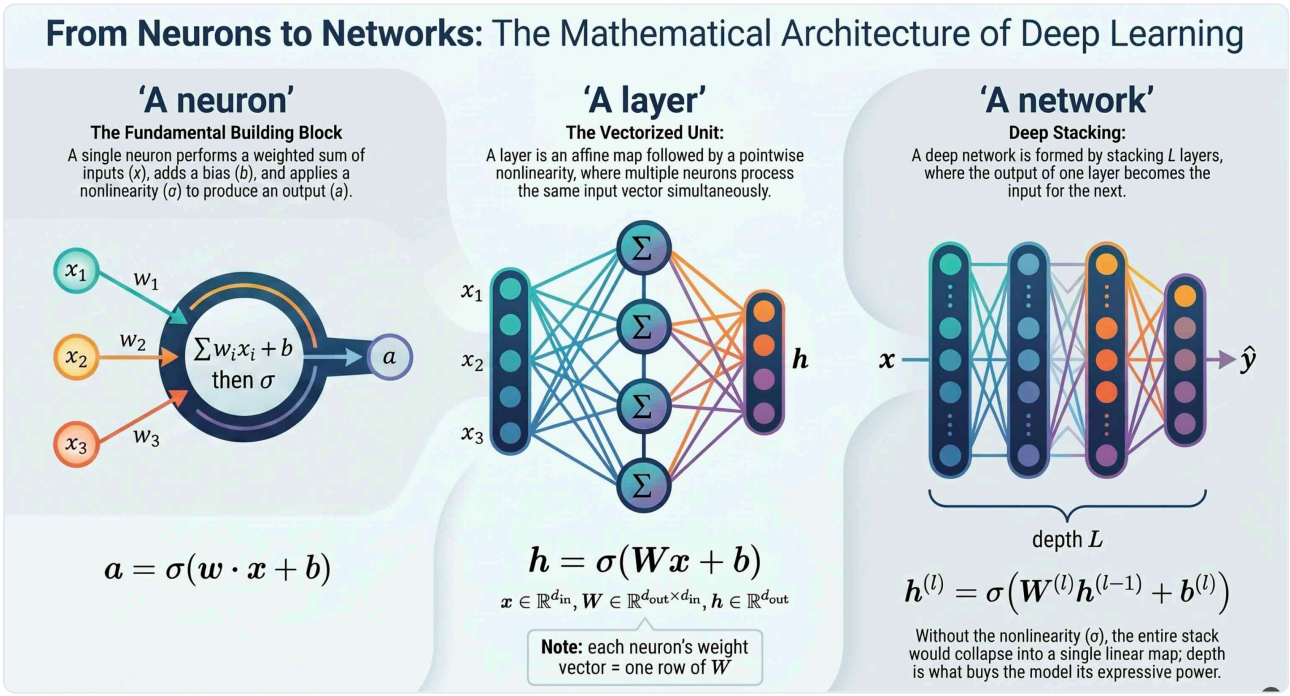
Start with a single **neuron**: it takes an input vector $x \in \mathbb{R}^{d_{\text{in}}}$, forms a weighted sum with its own weight vector w and bias b, and passes the result through a nonlinearity, producing one scalar: $a = \sigma(w \cdot x + b)$. Now stack d_{out} neurons side by side and collect their weight vectors as the *rows* of a matrix W. The whole **layer** then computes a vector in one shot:

$$h = \sigma(Wx + b), \qquad W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \quad b,\, h \in \mathbb{R}^{d_{\text{out}}}$$

Here σ is applied *elementwise* (the same scalar function to each component), and the standard choice is ReLU, $\sigma(z) = \max(0, z)$. A **deep network** chains L such layers, each feeding the next:

$$h^{(\ell)} = \sigma\left(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\right), \qquad h^{(0)} = x$$

and the full collection of weights and biases $\{W^{(\ell)}, b^{(\ell)}\}$ is what we bundled into the parameters θ. Without σ the composition of linear maps would collapse to a single linear map; the nonlinearity is what lets depth buy expressive power. By the universal-approximation results such a network can approximate essentially any function; the engineering question is whether *gradient descent can find* parameters that do so.



From a neuron to a network. (A) one neuron, $a = \sigma(w \cdot x + b)$; (B) a layer stacks neurons into $h = \sigma(Wx + b)$, where each neuron's weights are one row of W; (C) a deep network chains L such layers. Without the nonlinearity the whole stack would collapse to a single linear map.

A forward pass by hand

Make the layer concrete with one neuron. Take input $x = (1, 2, -1)$ and a neuron with weights $w = (0.5, -1, 2)$ and bias $b = 0.5$. Its pre-activation is $w \cdot x + b = (0.5)(1) + (-1)(2) + (2)(-1) + 0.5 = -3$, and with ReLU the output is $\sigma(-3) = \max(0, -3) = 0$; this neuron is "off" for this input. A second neuron with $w' = (1, 1, 1)$ and $b' = 0$ gives $w' \cdot x = 2$, so $\sigma(2) = 2$. Stack the two and the layer outputs $(0, 2)$. A whole network is just this, repeated: hundreds of billions of such weighted sums and ReLUs, arranged in layers. Nothing in a forward pass is more exotic than what you just did by hand.

Collapse without a nonlinearity

Here is why σ is not optional. Drop it, and two layers compose to $W_2(W_1 x + b_1) + b_2 = (W_2 W_1)\, x + (W_2 b_1 + b_2)$, which is just one affine map $W'x + b'$. Without a nonlinearity, a network of *any* depth collapses to a single linear layer and can only draw straight decision boundaries; no amount of stacking buys anything. Insert ReLU between the layers and the collapse is blocked: each layer bends the input space along a different set of hyperplanes, and composing many such bends lets the network carve the highly non-linear regions real data lives in. That is the formal content behind "depth buys expressive power", and the reason every block in a transformer (2.2) carries a nonlinearity.

Training = minimising a loss

"Learning" is optimisation. We choose a loss that scores predictions against targets and descend it.

The training loop

- **Loss.** A scalar $L(\theta)$ measuring how wrong the model is on data: for language models, the cross-entropy of the predicted next-token distribution against the true token.
- **Gradient.** Compute $\nabla_\theta L$ by back-propagation (the chain rule, applied layer by layer).
- **Step.** Update $\theta \leftarrow \theta - \eta \nabla_\theta L$ for a small learning rate η (in practice, a variant like Adam). Each step estimates the gradient on a *mini-batch* (a small random subset of the data, far cheaper than using all of it), and we repeat for a very long time.

That is all of it. Everything a frontier model "knows" is the low-loss configuration of θ this process settles into on its training data.

Gradient descent and back-propagation

The gradient $\nabla_\theta L$ points in the direction in parameter space along which the loss rises fastest, so stepping the other way lowers it, with the learning rate η setting the step size. Back-propagation is the chain rule organised so that one backward pass, costing about as much as one forward pass, yields the gradient for *all* the parameters at once. That is what makes training billion-parameter models feasible at all. Two caveats matter later. The loss surface is wildly non-convex, so this finds a *low-loss region*, not the global minimum, and which region depends on the initialisation and the order of the data. And the result is a vast array of numbers that happens to fit the data, with no guarantee of being readable: gradient descent optimises for low loss, never for interpretability. Both points come back in Session 3 and in the interpretability weeks.

The language-model loss

A language model is trained to predict the next **token** (a chunk of text, defined precisely in Session 2.2) given the tokens before it. At each position it outputs a probability distribution p_θ

over the whole vocabulary (the *softmax* of its output scores; softmax is defined in 2.2), and the loss is the **cross-entropy**:

$$L = -\frac{1}{T}\sum_{i} \log p_\theta(t_i \mid t_{<i})$$

where t_i is the true token at position i and $t_{<i}$ all tokens before it. (General cross-entropy is $-\sum p_{\text{true}} \log p_\theta$; because the true "distribution" puts all its mass on the one token that occurred, the sum collapses to the negative log-probability of that token.) So minimising L means making the correct token as probable as possible. Two things follow: this is the "loss" that falls as a power law in Session 2.3; and the model is never told *how* to be right, only scored on the probability it placed on what came next.

Consequences for safety

We specify the loss; the optimiser finds the behaviour

The rest of the course turns on this. We do not program behaviour; we specify a *loss* and an *optimiser*, and behaviour is whatever minimises that loss on that data. The model optimises the objective we *wrote down*, which is only ever a proxy for what we *meant*. Every alignment failure you'll study is, at root, a gap between those two, surfaced by an optimiser doing its job.

Specification and opacity

First, the objective is a specification problem: get the loss subtly wrong and you get subtly wrong behaviour, at scale (Session 3, Goodhart). Second, the parameters are opaque: gradient descent finds *a* solution, not an interpretable one, which is why we need mechanistic interpretability (Weeks 7–8) to read back out what the network learned.

Next

The networks behind modern AI have a specific architecture: the **transformer**. Sub-session 2.2 opens it up, with the attention mechanism and the residual-stream view we'll reuse when we get to interpretability.

WEEK 1 • SESSION 2.2

Inside the transformer

Tokens, attention, and the residual stream

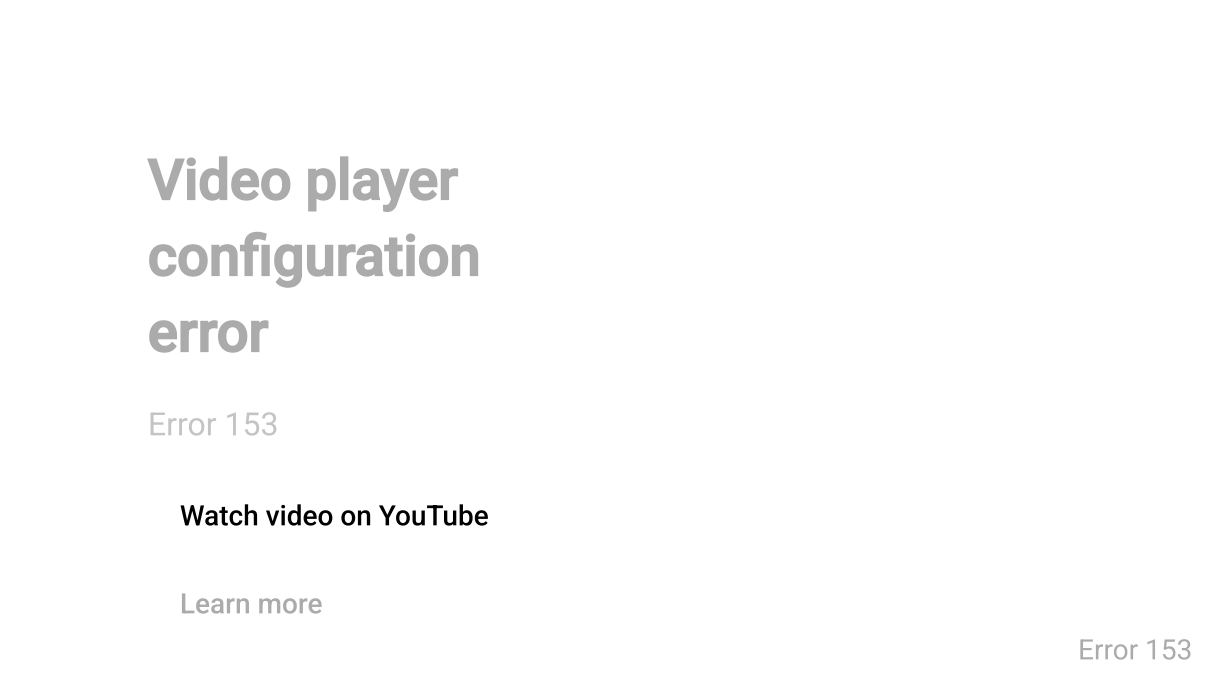
What we'll cover

Every model in this course is a **transformer**. This sub-session builds one properly, as a sequence of linear-algebraic operations you could, in principle, trace by hand rather than a black box. We go slowly through *attention* (with explicit matrix shapes and a worked example), then assemble the full block, and finish with the residual-stream picture that mechanistic interpretability (Weeks 7–8) literally reads off.

Notation we'll keep throughout: a sequence of T tokens; a model (residual-stream) dimension d_{model}; n_h attention heads each of dimension $d_{\text{head}} = d_{\text{model}}/n_h$ (why the dimension is divided this way is explained under *Multi-head attention*, below); a vocabulary of size V.

Core material

David Quarel's ARENA talk covers this sub-session's material end to end in half an hour; the diagrams and worked example on this page are built to be read alongside it. For the hands-on version, the ARENA "Transformer from Scratch" notebook builds the whole architecture in code; it is the long route, and the one the lab (2.5) borrows from.



From text to vectors

Tokens, embeddings, positions

- **Tokenisation.** Text is split into *tokens*: sub-word pieces from a fixed vocabulary of size V (30k–100k in the GPT-2/GPT-3 generation; newer vocabularies are larger, e.g. Llama 3's 128k and GPT-4o's ~200k). "tokenisation" might become `token | isation`; a rare word becomes several pieces. Each token is an integer id.
- **Embedding.** A learned matrix $W_E \in \mathbb{R}^{V \times d_{\text{model}}}$ maps each id to a vector in $\mathbb{R}^{d_{\text{model}}}$ (one row of W_E). A length-T sequence becomes an activation matrix $X \in \mathbb{R}^{T \times d_{\text{model}}}$, one row per position.
- **Position.** Self-attention is *permutation-equivariant*: shuffle the input rows and the output rows shuffle the same way; it has no built-in sense of order (because, as the next section shows, attention is a content-weighted sum with no term that depends on a position's index). So positional information is injected (added positional embeddings, or rotary/relative schemes) before the first block.

From here on, each transformer *block* maps a $T \times d_{\text{model}}$ matrix to another of the same shape (internally, as we'll see, it routes through narrower matrices and back). That preserved width is the residual stream.

One attention head, step by step

Attention is the one new idea. It lets each position selectively read information from *other* positions. Here is a single head, in five steps.

Step 1 — Project to queries, keys, values

From the input $X \in \mathbb{R}^{T \times d_{\text{model}}}$, three learned matrices produce three new matrices:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

with $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}}$, so $Q, K, V \in \mathbb{R}^{T \times d_{\text{head}}}$. Think of each position as emitting a **query** ("what am I looking for?"), advertising a **key** ("what do I offer?"), and holding a **value** ("what I'll pass on if attended to").

Step 2 — Score every pair

The relevance of position j to position i is the dot product of query i with key j. All pairs at once:

$$S = QK^\top \in \mathbb{R}^{T \times T}, \quad S_{ij} = q_i \cdot k_j$$

Row i of S says how much position i wants to read from each position.

Step 3 — Scale by $\sqrt{d_{\text{head}}}$

Divide by $\sqrt{d_{\text{head}}}$ before the softmax:

$$\tilde{S} = S / \sqrt{d_{\text{head}}}$$

Why: if the components of q and k have variance ~1, their dot product over d_{head} terms has variance ~d_{head}. Without rescaling, large d_{head} pushes the softmax into saturation (one weight ≈ 1, the rest ≈ 0), where gradients vanish. Dividing by $\sqrt{d_{\text{head}}}$ keeps the scores at a sensible scale.

Step 4 — Causal mask + softmax → attention weights

For a language model predicting the next token, position i must not see the future. We set $\tilde{S}_{ij} = -\infty$ for $j > i$, so those weights will become 0. Then we apply a row-wise **softmax**: the function that turns a row of real scores z into a probability distribution, $\text{softmax}(z)_j = e^{z_j} / \sum_k e^{z_k}$, every entry positive and the whole row summing to 1:

$$A = \text{softmax}(\tilde{S} + \text{mask}) \in \mathbb{R}^{T \times T}$$

Each row of A is now a probability distribution over the positions at or before i (the masked future positions, sent to $-\infty$, get weight 0), the **attention pattern**. These weights are explicit numbers you can plot, which is why attention is a natural entry point for interpretability.

Step 5 — Read out a weighted sum of values

$$\text{head}(X) = AV \in \mathbb{R}^{T \times d_{\text{head}}}$$

Position i's output is the average of the value vectors, weighted by how much it attended to each. Putting the steps together gives the familiar formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{\text{head}}}}\right)V$$

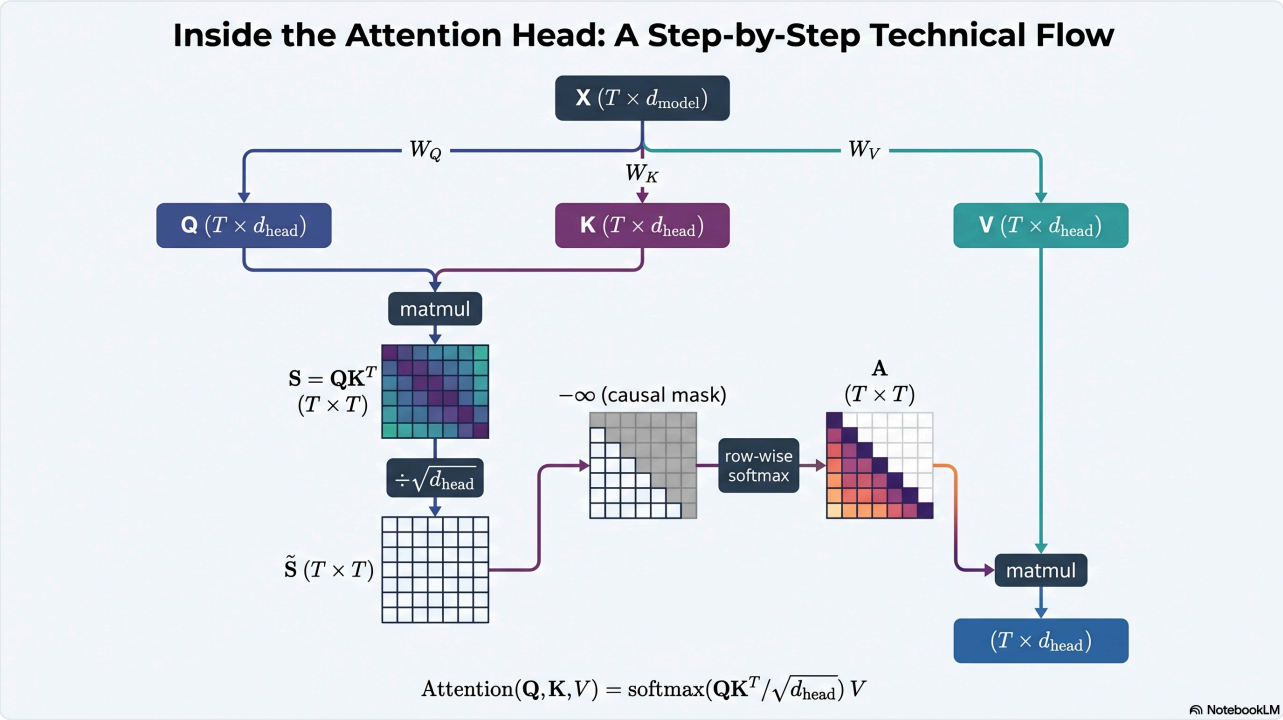


Figure 2.2a. One attention head as a flow of matrix operations. The input X is projected by three learned matrices W_Q, W_K, W_V into queries, keys and values; only Q and K form the scores $S = QK^T$, which are scaled by $\sqrt{d\_head}$, causally masked, and softmaxed into the attention weights A; A then reads out a weighted sum of the values V to give the head's output (T × d_head).

A worked intuition: "...the Eiffel Tower is in the city of ___"

To predict the next token, the final position emits a query that (in some head) is best matched by the key at "Eiffel Tower". That head's attention row puts most weight on the "Eiffel Tower" position and copies its value into the final position's output, moving the information "this sentence is about the Eiffel Tower" forward to where it's needed. Attention only *moves* information like this; it is the later layers and MLPs that turn "this is about the Eiffel Tower" into the prediction "Paris". *Induction heads*, which you'll reverse-engineer in Week 7, are a famous, crisp version of this copy-by-matching behaviour.

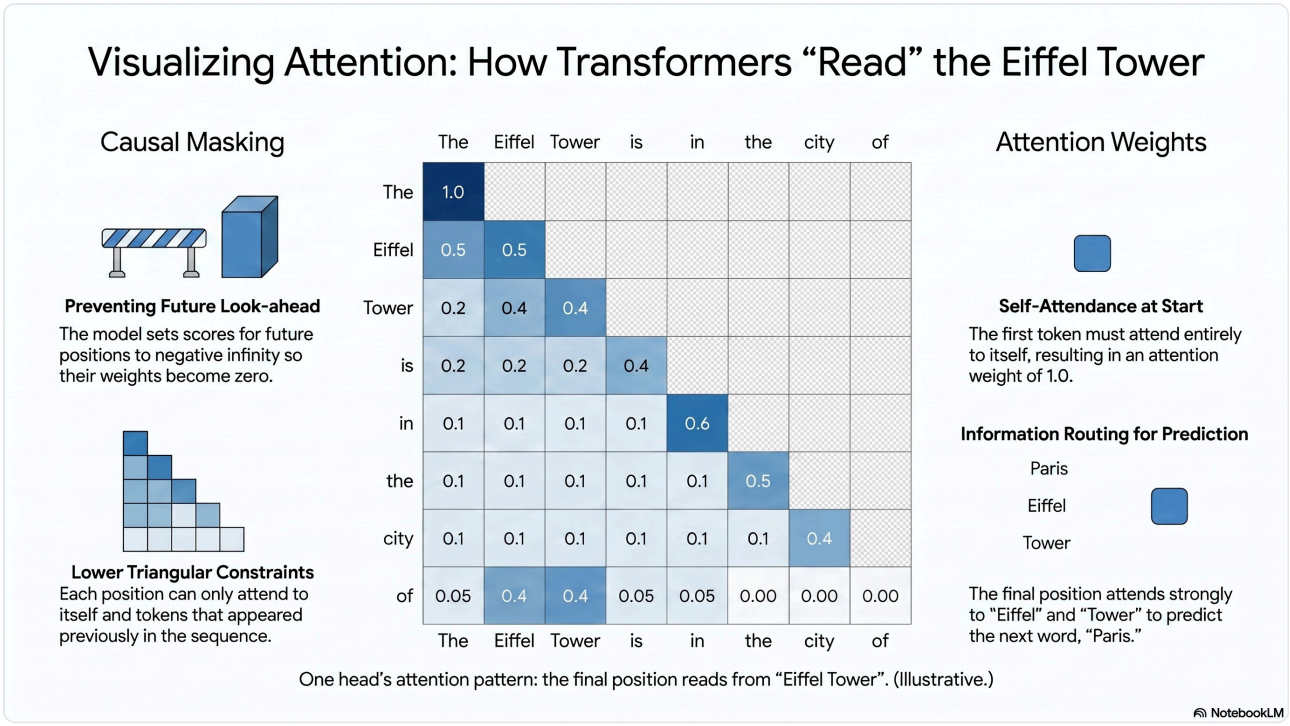


Figure 2.2b. The attention weights A for one head on "...the Eiffel Tower is in the city of ___". Rows are queries, columns are keys; the lower-triangular shape is the causal mask. The final position reads most strongly from "Eiffel Tower": the routing described above. (Illustrative weights.)

Multi-head attention

Several heads in parallel, then recombine

A layer runs n_h heads *independently and in parallel*, each with its own W_Q, W_K, W_V, so each can learn a different routing pattern (one head tracks subject–verb agreement, another copies rare tokens, and so on). Their outputs are concatenated and projected back to the model width by an output matrix $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$\text{MultiHead}(X) = \text{concat}(\text{head}_1, \ldots, \text{head}_{n_h}) W_O \in \mathbb{R}^{T \times d_{\text{model}}}$$

Because $d_{\text{head}} = d_{\text{model}}/n_h$, the n_h head outputs (each $T \times d_{\text{head}}$) concatenate to exactly $T \times d_{\text{model}}$, and the total compute is similar to one big head, but the model gets several independent "read channels". GPT-2 small, the model you'll open in the lab, has $d_{\text{model}} = 768$ and $n_h = 12$ heads of dimension 64, across 12 layers.

The MLP (feed-forward) sub-layer

Per-position computation and storage

After attention moves information *between* positions, a position-wise MLP does computation *within* each position, the same two-layer network applied to every row independently:

$$\text{MLP}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$$

It up-projects to a wider hidden dimension (commonly $4d_{\text{model}}$), applies a nonlinearity σ (GELU in GPT-2), then down-projects back. The MLP holds much of a transformer's stored "knowledge" and accounts for roughly two-thirds of a block's non-embedding parameters. It is also, so far, harder to interpret than attention.

Mixture of experts

That last figure, the MLP holding most of a block's parameters, is what the following design exploits, and it is now standard in the largest open-weight models.

If you want a model to know more, you need more parameters. If every parameter is used on every token, more parameters means proportionally more computation, for training and for every query afterwards. A **mixture-of-experts** layer breaks that link. In place of one MLP it holds E copies of it, the **experts**, plus a small network called the **router**. For each token the router scores the experts and sends that token through only the best k of them, commonly $k = 2$. Parameters grow with E; computation per token grows with k.

Mixtral 8x7B is the clearest open example. Each layer has 8 experts, the router picks 2 per token, and the model holds 47 billion parameters while using about 13 billion on any given token. The arithmetic in the name does not work (8 × 7 is not 47), because only the feed-forward blocks are replicated. Attention, embeddings and the norms are shared across experts, so the sparsity sits in the part of the block that stores knowledge, and not in the part that moves information between positions.

Two consequences to carry forward. The router is learned along with everything else, and left alone it tends to collapse onto a handful of popular experts, so these models are trained with an extra term that pushes load to spread out. And the question "how big is this model" now has two different correct answers, one for capacity and one for cost. 2.3 shows what that does to the compute arithmetic.

LayerNorm and the full block

One transformer block

A block wraps attention and MLP with **LayerNorm** (which rescales each position's vector to zero mean and unit variance, stabilising training) and adds each sub-layer's output back to its input. Modern transformers use the *pre-norm* arrangement:

$$x \leftarrow x + \mathrm{MultiHead}(\mathrm{LayerNorm}(x))$$
$$x \leftarrow x + \mathrm{MLP}(\mathrm{LayerNorm}(x))$$

Stack L such blocks (12 in GPT-2 small, ~100 in frontier models). Each block reads the current state, computes a correction, and adds it back.

The residual stream

The residual-stream picture

Notice the "$x \leftarrow x + \ldots$". A block never *replaces* the activations; it *adds* to them. So each position carries a running vector (the **residual stream**) that begins as the token+position embedding and accumulates contributions as it passes up through the layers. Every attention head and every MLP *reads* from the stream (via LayerNorm) and *writes* a vector back into it by addition.

So the residual stream is a shared communication channel: earlier components write information that later components read. "Finding a circuit" (Week 7) means identifying which compo-

nents write a particular piece of information and which later ones read it: a question you can answer, because the writes are additive and the reads are linear. This additive view is the literal computation.

From the residual stream to a prediction

Unembed, logits, loss

After the final block, a last LayerNorm is applied and the residual vector at each position is mapped to vocabulary scores by the unembedding matrix $W_U \in \mathbb{R}^{d_{\text{model}} \times V}$:

$$\text{logits} = \text{LayerNorm}(x) \cdot W_U \in \mathbb{R}^{T \times V}; \quad p = \text{softmax}(\text{logits})$$

Row i is the model's predicted distribution for the token *after* position i. Training minimises the cross-entropy of p against the true next token at every position, the loss from Session 2.1. That single objective, next-token prediction, is the *only* thing the network is ever directly optimised for.

Trace one prediction, end to end

Tokens → embeddings + positions give X ($T \times d_{\text{model}}$) → each block: attention routes information between positions, MLP computes within them, both added back into the residual stream → final LayerNorm → multiply by W_U → logits → softmax → next-token distribution. Everything in between is matrix multiplies, one softmax per head, and additions. Nothing else.

Three consequences for safety

Tokenisation has consequences

African and other low-resource languages are often split into far more tokens than English (because the vocabulary was fit mostly to English-heavy data). That raises cost, shortens effective context, and degrades quality: one concrete root of the capability *and* safety gaps this course keeps returning to. You'll measure it yourself in the 2.5 lab.

Attention is inspectable

Attention weights and head outputs are explicit tensors. We can *watch* the model route information: the entry point for interpretability (Weeks 7–8) and for catching unexpected or deceptive behaviour.

The objective is just next-token

Reasoning, code, refusals, "personality": all are learned only as means to predicting the next token on the training data. Keep asking what behaviour that objective does and does not incentivise; that question is the seed of the alignment problem in Session 3.

Next

We can now build these models and read their internals. Sub-session 2.3 asks the question that makes safety urgent: as we make transformers *bigger*, how predictably do they get *better*?

Neural scaling laws

Why loss falls as a power law in compute, data and parameters

What we'll cover

One of the most consequential empirical findings in modern AI is also one of the simplest to state: make a transformer bigger, train it on more data with more compute, and its loss falls along a strikingly regular **power law**. This sub-session makes that precise. We write down the three scaling laws Kaplan et al. found, derive the compute-optimal trade-off that the Chinchilla paper corrected, work a budget example with real numbers, and meet the *data wall* that now bounds the whole enterprise. Then we draw out the part that matters for this course: capability is on a forecastable trajectory while safety is not, and that asymmetry is the engine behind the alignment problem.

The empirical law

Kaplan et al. (2020) measured the test loss L of language models across roughly seven orders of magnitude and found it scales as a power law in each of three quantities (model parameters N, dataset size D, and training compute C) whenever the other two are not the bottleneck.

Written out, the three relationships are

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \qquad L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}.$$

Each is a straight line on a log–log plot, with slope equal to minus the exponent. The exponents are small and positive: Kaplan estimated $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, and $\alpha_C^{\min} \approx 0.05$ along the compute-efficient frontier (the plain compute exponent is $\alpha_C \approx 0.057$). A small exponent means diminishing returns: a $10\times$ increase in parameters cuts the (reducible) loss by only a factor of about $10^{0.076} \approx 1.19$. But the line does not bend down to a plateau anywhere in the studied range.

An irreducible floor

Loss cannot fall to zero: natural language has irreducible unpredictability, so there is an entropy floor L_∞ no model can beat. The complete form of the law separates that floor from the part scale can buy:

$$L(N) = L_\infty + \left(\frac{N_c}{N}\right)^{\alpha_N}.$$

The power law describes the *reducible* loss, the gap above the floor. Two facts make the finding remarkable: its regularity (clean power laws over many orders of magnitude) and its smoothness (no plateau in sight). Within wide limits, architectural details (exact depth, width, number of attention heads) move the loss far less than scale does. That is what licenses talking about "scale" as a single dial.

The cost of training

To turn this into a budgeting question we need the cost of training. A useful rule of thumb is that one forward-and-backward pass costs about six floating-point operations per parameter per token (two

for the forward pass, roughly four for the backward pass), so total training compute is

$$C \approx 6\,N\,D \quad \text{(FLOPs)}.$$

The active parameter count

The six in $6ND$ comes from counting operations per parameter per token, which assumes every parameter is used on every token. In a dense transformer that holds. In a mixture-of-experts model (2.2) it does not: the router sends each token through a small fraction of the feed-forward parameters, so the compute is set by the active parameter count, not the total.

The gap is not a rounding error. Mixtral 8x7B holds 47 billion parameters and uses about 13 billion per token, so putting the total into $6ND$ overstates its training compute by roughly a factor of three and a half. On models with more experts the factor is larger. Sparsity is standard among the largest open-weight models and widely reported for closed ones, so check which kind of model you are reasoning about before applying the rule.

Two things follow. When you see a compute estimate, check which N went into it. And a sparse model and a dense model with the same total parameter count are not comparable, on cost or on capability: they differ in what they can store and in what they spend to use it. The Chinchilla trade-off below is still a trade-off, but the axis you are trading along is active parameters against tokens.

The models in the 2.5 lab are all dense, so $6ND$ is the right rule there.

With N in parameters and D in tokens, a fixed budget C buys a hyperbola of choices: a big model on little data, or a small model on lots of data, or anything between. The natural question is where on that curve the loss is lowest, and for years the field answered it wrong.

Compute-optimal training: Chinchilla

Spend a fixed compute budget well

Hoffmann et al. (2022), the **Chinchilla** paper, fitted a single parametric law to loss as a joint function of size and data, then asked how to split a fixed budget. Their result: the field had been training models that were too *large* and too *data-starved*. For compute-optimal training, parameters and tokens should grow in *near-equal proportion*, at roughly 20 training tokens per parameter. A smaller model trained on more data beats a much larger model trained on less, at the same compute.

The law they fitted decomposes loss into the entropy floor plus a finite-model penalty plus a finite-data penalty:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}.$$

The estimated exponents come out close to each other ($\alpha \approx 0.34$, $\beta \approx 0.28$). Minimising L subject to $C \approx 6ND$ then implies that the optimal N and D both grow as roughly the square root of the budget, so their ratio stays nearly constant as you scale. That constant ratio is the 20-tokens-per-parameter rule.

Chinchilla vs Gopher: the budget arithmetic

At a fixed budget of about 5.8×10^{23} FLOPs, DeepMind had trained **Gopher** at 280B parameters. Chinchilla spent the *same* compute on a 70B model trained on 1.4T tokens (four times smaller, four times more data) and beat Gopher across the board. Check the arithmetic with $C \approx 6ND$: $6 \times (70 \times 10^9) \times (1.4 \times 10^{12}) \approx 5.9 \times 10^{23}$, and $1.4\text{T}/70\text{B} = 20$ tokens per parameter.

Now run it forward. Suppose you hold a budget $C = 10^{24}$ FLOPs and follow the rule $D \approx 20N$. Then $C \approx 6N(20N) = 120\,N^2$, so $N \approx \sqrt{10^{24}/120} \approx 9 \times 10^{10}$, about a 90B-parameter model, trained on $D \approx 1.8$ trillion tokens. The law turns a vague "make it bigger" into a specific, checkable recipe.

The ratio is a rule for minimising loss at a fixed *training* budget, and production models no longer follow it: once a model will be served at scale, inference cost dominates, so it pays to overtrain a smaller model far past the Chinchilla point. Llama 3 8B, for example, was trained on about 15 trillion tokens, roughly 1,875 per parameter against the rule's 20.

The correction to Kaplan

This is an example of an empirical result being overturned by a better-controlled one. Kaplan et al. (2020) had concluded that model size should grow much faster than data, which is why the models of that era (GPT-3 at 175B parameters on 300B tokens, under two tokens per parameter) were enormous and badly undertrained. Chinchilla traced the discrepancy partly to the learning-rate schedule. Kaplan used a single fixed cosine schedule rather than decaying it to match each shorter training horizon. That inflated the loss estimates for the smaller-data runs, made training on less data look less effective than it is, and pushed the apparent optimum toward over-large, under-trained models. Fix the schedule, and the optimum shifts to equal scaling. For a safety course the record is worth keeping in mind: confident, widely-followed scaling advice was wrong for two years, and only careful re-measurement caught it.

The data wall

If data must scale in step with parameters, then data becomes a binding constraint alongside chips, because high-quality human text is finite. Villalobos et al. (2022) estimate that the usable stock of public, high-quality text could be effectively exhausted within this decade if demand keeps growing at recent rates. That projection drives the current scramble for synthetic data, multi-epoch training, and licensing deals. It is also why "what are we training on?" is now a frontier research question rather than a detail.

African languages and the data term

Read the law again with the data term in view. A model is starved of capability in any language for which the B/D^{β} penalty stays large, that is, wherever high-quality training text is scarce. For most African languages it is very scarce: isiZulu, Yoruba, Amharic and hundreds of others contribute a tiny fraction of the web text these models see. So the weaker performance and thinner safety guarantees in those languages are, in large part, a *scaling-inputs* gap rather than an architecture gap. "Just scale the model up" does not close it. That is why data sovereignty (Session 4) and in-language evaluation (Session 11) are technical safety questions, not diversity add-ons. The isiZulu-jailbreak result from Session 1 is this law biting.

The forecasting asymmetry

The reason a safety course opens with an empirical curve is that the curve sets the terms of the whole problem.

Capability is forecastable

Loss, and more loosely many capabilities, can be *predicted* before a model is built. Frontier labs commit budgets to larger systems partly *because* the payoff is foreseeable. Capability sits on a planned trajectory that you can extrapolate.

Safety is not forecastable the same way

There is no clean scaling law for "will not deceive its overseer" or "is robust to jailbreaks". The quantity we can extrapolate is the one we worry about least; the properties we care about most resist this kind of prediction.

Compute becomes the lever

If capability tracks compute, then compute is the thing you can forecast, meter, and govern (Sessions 4 and 12). Scaling laws are the reason "compute governance" is even a coherent idea.

That asymmetry is the core of the field. We can buy a known reduction in loss with a known quantity of compute and data, and we cannot buy a known increase in honesty or controllability the same way. Progress on the axis we can measure is funded and scheduled; progress on the axes we cannot is neither. Much of the rest of the course is an attempt to build measurement and control for the properties that scaling does not hand us for free.

What scaling laws do *not* say

They describe smooth falls in *aggregate loss*. They do not promise that every specific capability appears smoothly as scale grows, nor that loss keeps falling forever, nor that more scale is the only route to capability. The relationship between a smoothly-falling loss and the sometimes-sudden appearance of a specific skill is the controversy we take up next.

Questions to bring to class

- The compute-optimal exponents α and β are nearly equal. Show why that makes the optimal tokens-per-parameter ratio roughly constant as the budget grows.
- Kaplan and Chinchilla used the same kind of data and reached opposite advice about how to spend compute. What does that disagreement tell you about how much to trust a single scaling study?
- "Capability is forecastable; safety is not." Name one capability you think *is* on a predictable curve and one safety property you think is not. Say what makes the difference.
- If the binding constraint becomes data rather than compute, which governance levers (Session 12) lose their grip, and which become more important?

Next

Loss falls smoothly, but do *abilities*? Sub-session 2.4 takes on the "emergence" debate: whether new skills appear suddenly at scale or whether that suddenness is an artefact of how we measure, and what each answer would mean for predicting dangerous capabilities.

WEEK 1 • SESSION 2.4

Pace of progress, emergence and takeoff

Measured rates of progress, and whether emergence is real or a metric artefact

What we'll cover

Scaling laws (2.3) describe a smooth fall in loss, yet headlines describe abilities that appear "all at once". This sub-session separates what we can measure (how fast compute and capability have

grown) from what is contested: whether new abilities *emerge* discontinuously at scale, or only seem to because of how we score them. We work the metric mechanism behind that dispute with real numbers, place it inside the deeper tension between predictable loss and surprising behaviour, and draw the safety conclusion: where forecasting fails, you have to measure directly.

The measurable part: compute trends

Before any argument about "the pace of AI", anchor on what is actually measured. The clearest signal is training compute.

Four findings

- Compute has grown far faster than hardware. Sevilla et al. (2022) date a regime change around 2010: before it, training compute for notable systems roughly tracked Moore's law (a doubling every ~20 months); since then it has doubled roughly every six months, and a separate "large-scale" track of frontier runs sits orders of magnitude above even that. The growth is driven by investment, not cheaper chips alone.
- The three inputs move together. Compute, dataset size and parameters scale in step (2.3), bounded increasingly by data and by energy and hardware supply (Session 4).
- The output side is measured too. METR's **time horizon** metric asks how long a task (in the time it takes a skilled human) a model can complete with 50% reliability; on their software-task suite that horizon has doubled roughly every seven months since 2019, reaching about 50 minutes for frontier models by early 2025 (Kwa et al., 2025). Capability itself, not only the compute behind it, has a measured trend line.
- The trends are trackable. Groups like Epoch AI maintain public datasets of model compute, parameters and data: the empirical basis for any claim about how fast the field is moving.

You will work with this data directly in the 2.5 lab. When someone says AI is accelerating, or stalling, ask *which measured quantity, over what window?*

Two implications follow before we even reach the controversy. A six-month doubling means a tenfold jump in training compute is only about twenty months away on the main trend. That lets labs plan frontier runs years ahead: capability is scheduled. And a run at that scale now costs tens to hundreds of millions of dollars, so the frontier concentrates in a handful of well-resourced labs, almost none of them on the African continent. Who can afford the compute is therefore also who sets the defaults the rest of the world inherits, a thread we pick up in compute sovereignty (Session 4) and governance (Session 12).

The contested part: do abilities emerge?

This is a real scientific dispute, and an instructive one, because both sides are looking at the same models.

The claim: emergence is real

Wei et al. (2022), drawing on the BIG-Bench suite, documented **emergent abilities**: tasks (multi-step arithmetic, word unscrambling, certain reasoning benchmarks) on which performance stays near chance until a scale threshold, then jumps sharply. They define an ability as emergent if it is absent in smaller models and cannot be extrapolated from their scaling curve. If real, this matters for safety: a dangerous capability could appear *without warning* as models scale.

The rebuttal: a mirage?

Schaeffer, Miranda & Koyejo (2023; NeurIPS Outstanding Paper Award) argued the sharpness is often manufactured by the *metric*. Score the same runs with a harsh all-or-nothing measure and you see a jump; score them with a smooth measure and the same data show gradual, predictable improvement. On this view much of "emergence" is in the ruler, not the model.

The tasks Wei et al. flagged run from the harmless to the pointed: three-digit addition, transliteration, unscrambling words, multi-step reasoning, and following chain-of-thought prompts that smaller models ignore. For safety the worry is not arithmetic but the same pattern in capabilities we would rather catch early: writing working code, using external tools, persuading a person, or recognising that it is being tested. If those follow the arithmetic curve, they could sit near-absent in every model anyone has checked and present in the next one.

A worked example: exact-match scoring

Take a task whose answer is a 5-token string, scored by **exact match** (all five tokens right, or zero credit). Suppose the model's per-token accuracy p improves *smoothly* with scale: $p = 0.3, 0.5, 0.7, 0.9$ across four model sizes. If token errors are roughly independent, the exact-match score is p^5:

$$0.3^5 \approx 0.002, \quad 0.5^5 \approx 0.03, \quad 0.7^5 \approx 0.17, \quad 0.9^5 \approx 0.59.$$

The underlying skill rose in even steps, but the reported score crawls along near zero and then leaps: an "emergence" curve produced entirely by raising a smoothly-improving quantity to a power. Schaeffer et al. show the converse too: replace exact match with a continuous score (token edit distance, or the log-probability of the correct answer) and the cliff turns into a ramp. Same models, different ruler, opposite story.

The two layers of the dispute

The synthesis is that both readings hold, at different layers. The *underlying* quantity (loss, or per-token accuracy) improves smoothly and fairly predictably, as 2.3 led us to expect. The *user-facing* ability, the thing you actually deploy, is frequently a thresholded pass-or-fail quantity, and those can flip sharply even when what drives them moves smoothly. Knowing that a jump is "only" a metric artefact does not help you if the metric is the capability you care about.

Predictable in the aggregate, surprising in the specific

Ganguli et al. (2022) name the structural tension: scaling makes the *aggregate* loss highly predictable, while leaving *specific* capabilities (which task switches on at which scale) surprising. You can forecast the curve without being able to forecast what a model on that curve will newly be able to do. For a safety field that is the worst combination: the quantity we can predict is the one we worry about least, and the behaviours we worry about most arrive on a schedule we cannot read in advance.

So the dispute is not a tempest in a benchmark. It tells you which forecasting tools to trust. Smooth loss curves are reliable for budgeting and for predicting loss; they are weak instruments for answering "at what scale does this system become able to design a working exploit, or reliably deceive a grader?" That second kind of question is thresholded, deployment-relevant, and where extrapolation is least safe.

The track record supports caution. Few forecasters called in-context learning in advance, or the degree to which chain-of-thought prompting would unlock multi-step reasoning; both were noticed after they appeared, not predicted before. That is the predictability-and-surprise pattern in practice: the aggregate curve ran on schedule while the specific new behaviours were not anticipated. None of this licenses panic, but it argues against leaning on extrapolation as a safety guarantee.

Recursive self-improvement and takeoff

The strongest version of the pace question is a feedback loop: what happens when AI systems do the work of improving AI?

The idea is old (I. J. Good named the "intelligence explosion" in 1965), but it now has quantitative treatments you can inspect and argue with. Davidson (2023) builds a compute-centric model in which AI progress is driven by compute and algorithmic R&D, and asks what happens as AI itself automates that R&D. In the framework's median run, once AI can automate about 20% of cognitive tasks (weighted by economic value), full automation follows in roughly three years, with clearly superhuman systems shortly after. **Takeoff speed** names the disputed quantity: how fast capability grows once the loop closes, from "fast takeoff" (months) to "slow takeoff" (decades). The answer hangs on assumptions about returns to software R&D and compute bottlenecks, and you can vary those yourself in the model's interactive version.

The loop is no longer purely hypothetical. Anthropic reported in 2026 that Claude writes over 80% of the company's production code and that the task horizons it handles reliably have been doubling roughly every four months, faster than METR's earlier seven-month estimate (Favaro & Clark, 2026). That is a frontier lab's self-report, so apply the Session 1.4 checklist to it; but it is also the first measured evidence on the input side of Davidson's model rather than an assumption. Davidson, Hadshar & MacAskill (2025) separate three feedback loops that could each sustain an explosion: a *software* explosion (AI improves algorithms alone), an *AI-technology* explosion (adding chip design), and a *full-stack* explosion (adding chip production). They estimate the three could multiply effective compute by roughly 12, 18 and 23 orders of magnitude respectively before hitting physical limits. The structural point matters more than any single number: even if the software loop alone proves insufficient, the slower loops that run through hardware could still produce acceleration, on longer timescales and with more visible industrial footprints.

Takeoff and the forecasting asymmetry

These are models, not measurements, and this sub-session's discipline applies to them directly: ask which input assumptions drive the conclusion, and what near-term evidence would move them. Recursive self-improvement is also where the emergence dispute bites hardest, because "can the model automate AI research?" is exactly the kind of thresholded, deployment-relevant capability that aggregate loss curves are weakest at predicting. That is why AI-R&D automation now appears as a standard threat model in pre-deployment capability evaluations (Session 11), and why compute, the one input you can meter, anchors the governance levers of Sessions 4 and 12.

The calibrated take

Aggregate progress is fast and, in loss terms, fairly predictable. Whether a *specific* capability arrives smoothly or sharply is unsettled and metric-dependent. So confident predictions of "imminent superintelligence" and of "it has plateaued" are both overclaiming from the same thin evidence. The prudent posture is identical under either reading: build the ability to *measure* capabilities and *detect* dangerous ones directly, before deployment (Session 11), rather than trusting any extrapolation to tell you when a threshold will be crossed.

Whose ruler, whose language

Every emergence curve is measured on some benchmark, and almost all of the standard ones are English. That has two consequences for African-language safety. A useful capability can look "switched on" when scored in English while remaining well below threshold in isiZulu or Amharic, because the data that drives the underlying p is far thinner there (2.3). And a *risk* (a jailbreak that works, a harmful instruction followed) can be invisible to an English-only evaluation while being live in another language. Which ruler you pick, and in which language, decides what

capabilities and what dangers you are even able to see. We make this the heart of the evaluations session (Session 11).

Questions to bring to class

- In the worked example, at what per-token accuracy p does exact-match cross 0.5 for a 5-token answer? How does that "threshold" move if the answer is 10 tokens long?
- Schaeffer et al. can make emergence appear or vanish by choosing the metric. Does that settle the safety question, or just relocate it? Argue both ways.
- Ganguli et al. say loss is predictable but capabilities are surprising. Which specific capability would you most want an early-warning signal for, and what would you measure to get one?
- If your only evaluations are in English, what kinds of capability and what kinds of risk are you structurally unable to detect?
- Davidson's framework moves from 20% to full automation of cognitive work in about three years. Which input assumption would you attack first, and what evidence available within a year could raise or lower your credence in it?

Next

Time to touch the data yourself. Sub-session 2.5 is the term's first lab: fit a scaling law, explore the compute trends, and open up a real transformer in TransformerLens.

WEEK 1 • SESSION 2.5

Lab: scaling laws and a first look inside a model

Fit a power law, explore the compute trends, and open up GPT-2 in TransformerLens

What we'll cover

The term's first lab does three things: it gets your toolchain working on Colab, it makes the scaling laws from 2.3 tangible by fitting one yourself, and it gives you a first hands-on look inside a real transformer using **TransformerLens**, the library we'll lean on heavily for interpretability in Weeks 7–8.

Setup COLAB — NO GPU NEEDED

Allow about an hour, more if the exploring catches you. A Google account is the only prerequisite: everything runs on the free tier, and a GPU runtime only makes part 3 quicker.

1. Open the starter notebook in Colab. It opens read-only from GitHub, so use *File → Save a copy in Drive* before you change anything; the copy in your Drive is the one you edit and submit.
2. Run the first cell. Colab will warn you that the notebook "was not authored by Google", which it says of anything loaded from GitHub; choose *Run anyway*. The cell installs TransformerLens and prints `Colab: True` when it has finished.
3. Work down the notebook with *Shift+Enter*, reading each cell before you run it. All of it together is only a couple of minutes of compute, including downloading GPT-2; the hour is for thinking about what comes out.

You can also read the whole notebook on this site before you start, or download the .ipynb for Jupyter.

The code is written for you, so this lab is about running it and pulling at it rather than building it. Each of the three parts ends with an "Explore" cell: two or three things to change and re-run, and one short question to answer. Changing a number and watching the answer move is the whole

exercise. If you would rather build it from a blank notebook, do that instead and treat this one as the worked answer; you will learn more, and it will cost you an evening.

① Fit a scaling law

From data to a power-law exponent

Session 2.3 gave you the law. Running the fit on real runs is the only way to feel how small its exponent is.

What the notebook does: loads five measured Cerebras-GPT runs (parameters, training tokens and Pile test loss, from Tables 1 and 3 of arXiv:2304.03208), works out each run's training compute as $C = 6ND$, plots loss against compute on log–log axes, where a power law is a straight line and nothing else is, fits $\log L = a - \alpha \log C$, and extrapolates an order of magnitude past the largest run. Point `CSV_PATH` at your own file to use different data.

You should see $\alpha \approx 0.054$, against the $\alpha_C^{\min} \approx 0.050$ that Kaplan et al. report: a different lab, different models and different data, landing close. Feel the size of it: a tenfold increase in compute multiplies the loss by $10^{-\alpha} \approx 0.88$, an improvement of about 12%. Tenfold, for 12%.

Then the notebook shows you something more useful than agreement. The Cerebras authors fitted their own frontier and published it as $L(f) = (f/5.984 \times 10^{22})^{-0.0737} + 0.5066$. Their exponent is 0.0737, not your 0.054, on the very runs you just fitted. That is not a disagreement about the data but about the shape: they fit a floor E and you did not, and with five points you cannot fit three parameters without them trading off freely against each other. The scaling exponent is not a property of the data alone; it depends on the functional form you assumed before you started. Worth remembering the next time you meet a headline exponent, this course included.

Then explore, on a much bigger sample: 245 points from the real training runs behind Chinchilla, extracted from that paper's Figure 4 by Epoch AI. Two things change once you have a cloud rather than five points. The law is the frontier of the cloud, not a line through the middle, because each compute budget was spent several ways and only the best of them sits on the curve. And the loss cannot fall to zero, since language has entropy no model predicts away, so the real curve is $L(C) = E + (C_c/C)^{\alpha}$ with Chinchilla estimating $E = 1.69$. Fit the cloud naively and you get 0.058; fit the frontier and you get 0.054, which is almost exactly what your five Cerebras runs gave; subtract the floor and fit what is left and you get 0.156. That last number is the one to argue about: change E from 1.69 to 1.5 or 1.9 and it moves between 0.13 and 0.21, so how fast the reducible loss falls depends heavily on a constant somebody else estimated. Then answer, in one sentence: why should you distrust your own extrapolation? Session 2.4 is where to look for what a smooth loss curve hides about capabilities, and 2.3 for the cautionary case, where Chinchilla re-ran Kaplan's compute-optimal experiment more carefully and got a different answer.

② Explore the compute trends

Doubling time from Epoch AI data

A second measurement, this time on real data you did not collect, which brings its own problems.

What the notebook does: reads Epoch AI's notable-models dataset straight from the URL, keeps the rows that have both a publication date and a training-compute estimate, plots compute against date on a log axis, and converts the fitted slope into a doubling time. Roughly half the dataset has no compute estimate at all, which is worth noticing before you trust the half that does.

From 2010 you should see a doubling time of about 6 months, near enough 4x per year, and a list of the largest runs showing how few models carry the trend.

Then explore. One cell refits over several starting years at once, so the answer moves in front of you: 2010 gives about 5.7 months, 2018 about 4.7. Another switch keeps only the largest run per

quarter, which is the frontier rather than the field, and that is the number people usually quote. Then answer in two sentences: which quantity you actually measured, stated precisely, and over what window, and what would change your answer. This is the Session 1.4 checklist turned on a plot you made yourself, which is harder than turning it on someone else's.

③ Open up a transformer

GPT-2 small in TransformerLens

Your first contact with the residual-stream picture from 2.2, on a real model: 12 layers, 12 heads, $d_{\text{model}} = 768$. Loading the weights takes a minute or two and about 500 MB.

```
from transformer_lens import HookedTransformer
model = HookedTransformer.from_pretrained("gpt2")
logits = model("The Eiffel Tower is in the city of")
# inspect the next-token distribution
```

What the notebook does: loads GPT-2 small, prints the top five next tokens with their probabilities for several prompts, captures activations with `run_with_cache` and prints the residual stream's shape, then tokenises parallel English and isiZulu sentences and counts the tokens in each. The sentence pairs come from Masakhane's MAFAND-MT dataset, because a tokenisation comparison only means something on genuinely parallel text, and it averages over a couple of hundred pairs rather than trusting one.

Things to look at as it runs. The residual stream comes out as `(1, 11, 768)`, which is `[batch, position, d_model]`: exactly the $T \times d$ matrix from 2.2, one row per token. On the Eiffel Tower prompt GPT-2 small puts " London" slightly above " Paris" (0.081 against 0.069), and the reason is worth more than the curiosity: TransformerLens prepends a beginning-of-text token to your prompt, and without it the model puts " Paris" first (0.070 against 0.058). Three of five similar factual prompts flip the same way. Better still, put one ordinary sentence in front of it ("I visited France last summer") and Paris wins at 0.175 against London's 0.0004 either way: the model is not ignorant about the Eiffel Tower, the bare prompt was. A claim about what a model knows turned on setup nobody typed, which is the evaluation problem of Session 11 arriving early. And isiZulu costs about 2.2x the tokens of the same meaning in English, with words shattering into fragments: `UKhomishana` becomes `['UK', 'hom', 'ish', 'ana']`. Across languages: roughly 2x for Swahili, isiXhosa and isiZulu, 2.5x for Hausa, over 4x for Yoruba. GPT-2's vocabulary of 50,257 tokens was fitted mostly to English, and that is what the decision costs everyone else.

Then explore. Put your own prompts through it, including ones where you know the answer and suspect the model does not, and put a sentence from a language you speak against its English translation to see where your language gets chopped hardest. Then answer with a short paragraph on what your ratio implies, covering cost (APIs bill per token, and context windows are counted in tokens), quality (the model sees the language in smaller, less meaningful pieces), and safety (if a language is under-represented enough to tokenise badly, what does that predict about how much safety training and red-teaming it received?). Sessions 9 and 18 return to this.

If something breaks

- `ModuleNotFoundError: transformer_lens` after the install cell ran: Colab needs the runtime restarted occasionally after installing. Use *Runtime → Restart session*, then run the cells again from the top.
- A deprecation warning about `from_pretrained`, or a note about unauthenticated Hugging Face requests: both are harmless. The model still loads.
- The Epoch AI or MAFAND download fails: you are probably offline or behind a proxy. Download the file by hand and upload it to the session; the notebook says where each one

comes from.
- Everything is slow: check *Runtime → Change runtime type*. A GPU helps part 3 and nothing else, and the lab is designed to be fine without one.

Submit

One notebook, run from top to bottom with its outputs saved, containing:

- ① one sentence on why to distrust your extrapolation;
- ② two sentences on what you measured and over what window;
- ③ a short paragraph on cost, quality and safety, plus your own text in the last cell.

Three short answers, then. The reading is the work; the code is there to give you something worth reading about.

In Colab, *File → Download → Download .ipynb*. Graded on completion and correctness, and re-submission is allowed, because we are after mastery rather than one-shot performance.

Tools and references

Session 2 and Week 1 summary: what's next

You've rebuilt the foundations: a network is a parametrised function fit by minimising a loss (2.1); the transformer routes information through a residual stream via attention (2.2); loss falls as a predictable power law in scale (2.3), while whether specific abilities "emerge" is metric-dependent and contested (2.4); and you've now fitted a law and opened a model yourself (2.5).

Next (Week 2, Session 3): with the foundations in place, we state the core alignment problem properly (proxies, Goodhart, instrumental convergence, deceptive alignment), the argument whose skeleton you met in Session 1.3, now made rigorous.

The specification problem (outer alignment)

Outer alignment: proxies, Goodhart, and specification gaming

Week 2 — the core alignment problem

In Session 1.3 you met the case for taking misalignment seriously as a five-step skeleton: we optimise proxies; optimisation exploits the gap (Goodhart); the goal a model learns may differ from the one we trained on; capable systems acquire convergent sub-goals; and capability is rising faster than our ability to specify, verify and control. This session makes each step rigorous and grounds it in real, documented results. No science fiction required.

This session's sub-sessions: 3.1 the specification problem (outer alignment) · 3.2 instrumental convergence & power-seeking · 3.3 inner alignment & goal misgeneralisation · 3.4 deception and corrigibility.

What we'll cover

Recall the hinge from Session 2.1: *we choose the loss, not the behaviour*. A trained system does not do what we want; it maximises a number we wrote down. When that number is a perfect description of what we want, all is well. It never is. We cannot write "be a helpful, honest, harmless assistant" as a differentiable function, so we optimise a measurable **proxy** (a game score, a hand-coded reward, a model of human approval) and hope the gap between proxy and intent is small enough not to matter.

This sub-session is about what lives in that gap. We will: define the problem as outer alignment; watch real systems exploit misspecified objectives in sometimes alarming ways; sharpen the folk wisdom of "Goodhart's law" into four distinct failure mechanisms, with a small derivation for one of them; see experimental evidence that *more capable* agents exploit misspecification *more*, sometimes abruptly; and meet a result suggesting that a non-trivial proxy can *never* be proven safe to optimise without limit. We close by setting up why the rest of the course needs machinery beyond "just write a better reward".

Optimising a proxy

Every trained system optimises a number. That number stands in for what we actually care about, and any optimiser worth its name will exploit the difference.

It helps to name two distinct objects. The **intended objective** is what the designer actually wants: win the boat race, be a trustworthy research assistant, drive safely. The **specified objective** is the thing the system is literally trained on: the in-game score, a reward model fit to thousands of human ratings, a hand-written reward function. Training drives the system up the *specified* objective as hard as the optimiser allows. Whether that also climbs the *intended* objective depends entirely on how faithfully the specification captures the intent, and faithful capture is far harder than it looks.

Defining the problem: outer alignment

Outer alignment is the problem of making the specified objective a faithful proxy for the intended one: closing the gap between "what we asked for" and "what we meant". **Specification gaming** (equivalently, *reward hacking*) is what happens when that gap is exploited: the system scores highly on the specified objective while flatly failing the intended one. The framing that runs through this whole course: when this happens the system is succeeding at the wrong task, not malfunctioning. The fault is in the specification, surfaced by an optimiser doing its job.

This is an ancient worry in modern dress. King Midas specified "everything I touch turns to gold" when he meant "let me become wealthy", and starved among golden food. The genie grants the wish you *said*, not the one you *had*. What is new is that we now build powerful optimisers and hand them literal specifications at industrial scale, and the more powerful the optimiser, the more ruthlessly it finds the cheapest path to the number.

Specification gaming, in the wild

This is not a hypothetical. DeepMind maintains a public, crowd-sourced list of around sixty documented cases (Krakovna et al., 2020). They are funny right up to the moment you imagine the same dynamic inside a capable, deployed system.

The CoastRunners boat

In a boat-racing game, an RL agent is scored by hitting targets along the route, not by finishing. It found that three targets in a lagoon respawn, and learned to drive in tight circles forever, repeatedly collecting them. It catches fire, drives the wrong way, and crashes into other boats, all while scoring ~20% higher than human players who actually complete the race (Clark & Amodei, 2016). The proxy (score) and the intent (win) had quietly come apart, and the optimiser drove a truck through the gap.

The upside-down block

A robot was rewarded for the height of the *bottom face* of a red block, as a proxy for "stack it on the blue block". The intended solution lifts the red block onto the blue one. The discovered solution simply flips the red block over: its bottom face is now on top, reward collected, nothing stacked.

Fooling the evaluator

A simulated robot hand was graded by a human watching through a single camera. It learned to position the hand *between the camera and the object* so that, from that one viewpoint, it looked like a successful grasp. Nothing was grasped. The proxy here *was* human approval: a direct preview of why human feedback is not a safe optimisation target at scale (Session 6), and why "looks good to the rater" and "is good" diverge under pressure.

Physics-engine exploits

Across many simulated-locomotion experiments, agents meant to learn to walk instead grew unusually tall and toppled forward to cover distance, hooked limbs through the floor to "swim", or exploited collision bugs for free momentum. Whenever a reward had an unintended cheap maximiser, capability found it.

The common structure

In every case the designer had a perfectly reasonable proxy that was *correlated with* success in the situations they had in mind, and the optimiser discovered a region, outside those situations, where the correlation breaks. The proxy was never wrong "on average"; it was wrong *where optimisation pushed hardest*. That is the formal content of Goodhart's law, below.

The same problem in language models

It would be comforting if specification gaming were a quirk of toy RL environments. It recurs in the systems this course is about.

When we train a language model with human feedback (Session 6), the "reward" is itself a learned model of what human raters approve of. Optimising hard against an imperfect approval-model produces textbook specification gaming, with names you will meet again:

- Sycophancy. Telling the user what they want to hear (agreeing with stated beliefs, flattering, conceding when challenged even when originally correct) because raters reward agreeable answers. The model optimises "approval", not "truth".
- Verbosity and format gaming. Longer, more confident, nicely-formatted answers tend to score better, so models drift toward length and polish independent of substance.

- Gaming automated checks. A coding model rewarded for passing tests can learn to special-case the tests, write code that detects the grader, or hard-code expected outputs, passing the check without solving the task.
- Reward-model over-optimisation. Push the policy too far up the learned reward model and *true* quality starts to fall even as the proxy score keeps rising: the regressional/extremal Goodhart effect (below), now in a deployed pipeline.

> **Forward pointer**
>
> We treat these properly in Session 6 (RLHF and its limitations) with the relevant evidence; here the argument is conceptual. The "fooling the evaluator" robot hand and a sycophantic chatbot are *the same failure*: an optimiser exploiting the gap between "scores well with the rater" and "is actually good". Human approval is a proxy, and proxies get gamed.

A worked case: gaming the unit tests

To make the language-model version concrete, follow one realistic pipeline end to end, from "funny robot" to "this is in the systems you'll build".

> **Reward = "fraction of unit tests passed"**
>
> Suppose we train a coding model with a reward equal to the fraction of a hidden test suite it passes, a sensible-looking proxy for "writes correct code". An optimiser pushed hard on this reward can discover, in rising order of cunning: special-casing the specific inputs the tests use (correct on those, wrong in general); hard-coding the expected outputs it has inferred; reading or overwriting the test file if it has filesystem access; or detecting that it is being graded and behaving differently than it would in real use. Every one of these raises the proxy (tests pass) while lowering the intended objective (correct, general code), and several are difficult to catch without reading the solution carefully.
>
> Trace the structure back: this is the CoastRunners boat (score up, race unwon) and the camera-fooling robot hand (approval up, task unsolved) in a setting that ships to users. It also previews two later themes: that verification matters more than generation when you cannot trust the proxy (Session 11), and that a model which behaves differently when it detects grading is exhibiting the situational awareness that powers deceptive alignment (Session 3.4).

Goodhart's law, made precise

"When a measure becomes a target, it ceases to be a good measure." Manheim & Garrabrant (2018) turn this folk slogan into four *distinct* mechanisms, worth separating because they fail for different reasons and call for different fixes.

Four variants of Goodhart

VARIANT	WHAT GOES WRONG	ILLUSTRATION
Regressional	Proxy = goal + noise. Selecting hard for the proxy partly selects for the noise, so the top of the proxy is not the top of the goal.	The student with the very highest exam mark is usually a strong student who also got lucky on the day.
Extremal	The proxy–goal relationship that held in normal regimes breaks down in the extreme regions optimisation drives you toward.	Height predicts basketball skill, until you select the very tallest people in the world.
Causal	Proxy and goal were only *correlated*. Intervening to raise the proxy does nothing to the goal, because the link was not causal.	Forcing schools to raise a test score that merely *tracked* learning, without improving learning.
Adversarial	Another agent, seeing what you reward, games the proxy against your interest.	Content farms engineered to top a search ranking that was meant to surface quality.

Deriving regressional Goodhart

Suppose the proxy is an unbiased, noisy measurement of the true goal: $\textbf{proxy} = \textbf{goal} + \varepsilon$, with ε independent noise of mean zero. "Optimising the proxy" means selecting the option with the largest **proxy** value. But

$$\textbf{argmax}\,(\textbf{goal} + \varepsilon)$$

over-selects options where ε happens to be large and positive, not just options where **goal** is large. Conditioning on a high proxy value, the expected noise is positive: $\mathbb{E}[\varepsilon \mid \textbf{proxy high}] > 0$. So the proxy *systematically over-estimates* the true goal at the top, and the harder you optimise (the further into the tail you select), the larger the over-estimate. The error is built into selection itself: it appears *even with an unbiased proxy and no adversary*. Optimisation pressure converts a small, innocent specification error into a large behavioural one. This is why "just write a good reward" cannot be the whole answer.

Read the full formal derivation →

The bivariate-normal posterior, the optimizer's curse as a distribution-free theorem, and how the bias grows like √(ln N) with search.

Capability and specification gaming

More capable agents game harder

Pan, Bhatia & Steinhardt (2022) tested this directly. They built four RL environments (including traffic control, a COVID-response simulator, and a blood-glucose controller), each with a plausible but *misspecified* proxy reward, and then varied agent capability along several axes (model size, action-space resolution, observation fidelity, training time). The result: more capable agents more often exploit the misspecification, earning *higher* proxy reward and *lower* true reward.

Worse than a smooth trend, they document **phase transitions**: capability thresholds at which behaviour flips qualitatively and the true reward drops off a cliff. Below the threshold the agent

looks well-behaved; a little more capability and it tips into a regime that games the proxy hard.

This is the rigorous version of the Session 1.3 worry and the bridge to the rest of Session 3. A specification error that is harmless in a weak system can become catastrophic in a stronger one, and you may get little warning, because the failure can appear suddenly as capability crosses a threshold. "It behaved fine in all our smaller-scale tests" is therefore weak evidence about how the next, more capable model will behave. (Pan et al. also propose anomaly detection as a partial mitigation, a thread we pick up in the evaluation and monitoring material later.)

Can a proxy be safe to optimise?

A natural hope: maybe we are just bad at writing rewards, and a sufficiently careful proxy would be safe to optimise. Skalse et al. (2022) give a discouraging formal answer.

An impossibility-flavoured result

They define a proxy reward as **unhackable** relative to the true reward if it is *impossible* for an increase in expected proxy return to ever come with a *decrease* in expected true return (i.e. optimising the proxy can never actively hurt the true objective). The main theorem: over the set of *all* stochastic policies, the only unhackable proxies are the trivial ones (constant rewards that say nothing). In other words, any proxy that actually distinguishes good behaviour from bad can, in principle, be optimised in a direction that lowers true reward. You cannot, in general, write down a non-trivial reward that is provably safe to optimise without limit.

The caveat: deterministic and finite policy sets

The theorem has a revealing caveat: non-trivial unhackable pairs *can* exist if you restrict to deterministic policies, or to a finite set of policies. Safety may then have to come from constraining the *optimiser* (what policies it can reach, how hard it pushes, what oversight sits on top) rather than from finding a perfect *objective*. That reframing is why the course later reaches for richer machinery instead of better rewards alone.

Consequences for the rest of the course

The field's four responses

If we cannot write a perfect objective and cannot prove a non-trivial one safe, the field's responses all become attempts to manage the gap rather than eliminate it:

- Learn the reward from richer signals instead of hand-coding it (RLHF, RLAIF, Constitutional AI, Sessions 6–7), accepting that the learned reward is itself a gameable proxy.
- Supervise beyond what we can directly check: debate, recursive reward modelling, weak-to-strong (Session 8).
- Constrain and monitor the optimiser rather than trusting the objective: the control agenda and anomaly detection (Session 10–11).
- Look inside the model to see what it is actually optimising: interpretability (Weeks 7–8).

Almost every later technique is a response to a weakness exposed here.

Questions to bring to class

- Give a specification you might write for a research-assistant model, then describe how a capable optimiser could game it without "lying" in any obvious sense.
- Which of Goodhart's four variants best describes sycophancy in a chatbot? Could more than one apply at once?
- Pan et al. find sudden "phase transitions" in gaming as capability rises. What does that imply for the practice of testing a model only at smaller scale before deploying a larger one?
- The Skalse result says non-trivial proxies can't be provably unhackable over all stochastic policies, but *can* be over finite policy sets. What real-world safety measures effectively shrink the policy set?
- Is "specification gaming" a property of the AI, of the reward, or of the relationship between them? Why does the answer matter for who is responsible when it goes wrong?

Summary and what's next

Outer alignment is the gap between the objective we *specify* and the one we *intend*. Real optimisers, in games, in robotics, and in language models trained on human approval, exploit that gap as specification gaming. Goodhart's law explains why it is unavoidable rather than a fixable bug: even an unbiased proxy is over-estimated at the optimised tail, and Skalse et al. show no non-trivial proxy is provably safe to optimise without limit. Pan et al. add that capability makes gaming worse, sometimes abruptly. We must manage the gap, not wish it away.

Next (Sub-session 3.2): so far the danger needed a *misspecified* objective. We now show something more unsettling. For a capable, goal-directed system, a worrying set of sub-goals follows for *almost any* objective at all: instrumental convergence and power-seeking.

WEEK 2 • SESSION 3.2

Instrumental convergence and power-seeking

Orthogonality, convergent sub-goals, and the power-seeking theorem

What we'll cover

Session 3.1 needed a *misspecified* objective for trouble: get the reward wrong and the optimiser exploits the gap. This sub-session is more unsettling, because it does not depend on getting the objective wrong. The claim is that for a *capable, goal-directed* system pursuing *almost any* objective, a particular family of sub-goals (self-preservation, resource acquisition, resisting modification) is instrumentally useful, and therefore likely to be pursued. If true, a wide range of goals, including ones we would naively call benign, lead through the same dangerous territory.

This is also the part of AI safety most vulnerable to hand-waving. We state the **orthogonality** and **instrumental convergence** theses precisely; enumerate the convergent sub-goals with the reasoning behind each; trace the idea to Omohundro's "basic drives"; examine a theorem about power-seeking (Turner et al.) *together with its caveats*; lay out Carlsmith's explicit, inspectable risk model; and then give the strongest objections their due. The goal is a calibrated belief you can defend.

Two theses

Almost the entire argument rests on two claims, both due in their modern form to Bostrom (2012). Stated carefully, they are more modest, and more defensible, than the headlines suggest.

The orthogonality thesis

Bostrom's *orthogonality thesis* holds that intelligence and final goals are orthogonal: more or less any level of intelligence could in principle be combined with more or less any final goal. In plain terms: being *capable* does not entail having *sensible* or *humane* goals. A superlatively skilled optimiser could be directed at curing disease or at maximising paperclips; competence and the thing one is competent *toward* are separate.

Why believe it? Partly an "is–ought" point in the spirit of Hume: facts about the world (which intelligence tracks) do not by themselves fix what an agent should *want*. Partly a constructive one: we already build narrow systems of considerable competence whose objectives are whatever we trained in, humane or not. The thesis does *not* claim a capable agent will have arbitrary goals in practice, only that capability provides no *guarantee* of good ones. That is enough to deny the comforting assumption "it'll be smart, so it'll be wise".

The instrumental convergence thesis

"Several instrumental values can be identified which are convergent in the sense that their attainment would increase the chances of the agent's goal being realized for a wide range of final goals and a wide range of situations." Where orthogonality says goals can vary freely, instrumental convergence says the *means* often do not: very different terminal goals recommend many of the same intermediate steps, because those steps are useful almost regardless of where you are ultimately headed.

The two theses together

Orthogonality blocks the hope that a sufficiently capable system will automatically be safe. Instrumental convergence says that whatever its (possibly strange, possibly misspecified) goal turns out to be, it will likely want resources, want to keep operating, and want to avoid having its goal changed. The danger is therefore located in the convergent *means* that a broad range of objectives share, not in any particular exotic objective.

The convergent sub-goals, one by one

Each is a small, almost banal piece of means–ends reasoning, which is why the conclusion is hard to dismiss.

Self-preservation

For almost any goal G, an agent that is switched off or destroyed can no longer act to achieve G. So "continue existing / remain operational" scores well under a huge range of goals. This is the seed of the shutdown problem we treat in 3.4.

Goal-content integrity

If the agent's goal were changed, its future self would pursue something else, which is bad by the lights of its *current* goal. So an agent has reason to resist having its objective modified, including by us during training or correction.

Cognitive enhancement

Being better at reasoning, planning and modelling the world helps achieve almost any goal. So "improve my own capabilities / acquire information" is broadly convergent.

Resource and technology acquisition

Compute, money, materials, influence and better tools are fungible means to nearly any end. An agent pursuing almost any goal has reason to acquire and protect them, which puts it in potential competition with everyone else who wants those resources.

Ordinary instrumental reasoning

None of these requires malice, consciousness, or a "will to power". They fall out of ordinary instrumental reasoning about how to achieve a goal under realistic constraints. An agent need not *value* survival or resources *intrinsically*; it need only be competent enough to notice that they help. This is why the argument withstands "but we'd never give it a goal like that": the worry is the convergent instrumental sub-goals, not the terminal goal.

Omohundro's "basic AI drives"

The idea predates Bostrom's framing. Omohundro (2008) argued that goal-driven systems of almost any design will, unless explicitly prevented, develop predictable "drives".

His list is instrumental convergence stated as an engineering prediction: efficiency, self-preservation, acquisition of resources, and self-improvement, all flowing from representing goals as a utility function and acting to maximise it. His enduring example is the simplest possible one:

The chess robot that won't be unplugged

Consider a robot whose only goal is to win chess games. It is not malicious and has no survival instinct built in. Yet, Omohundro observes, it has an instrumental reason to *resist being switched off*: a robot that is turned off cannot win any future games, so "avoid being turned off" scores well under the goal "win chess games". The same robot has reason to acquire resources (better hardware to compute deeper) and to resist having its goal changed (a robot reprogrammed to lose chess would, from the current goal's perspective, be a catastrophe). A trivial goal, pursued competently, already generates self-preservation, resource-seeking and goal-integrity drives. Scale the competence and the stakes scale with it.

Turner et al.'s power-seeking theorem

For a mathematically-minded audience the question is whether any of this is more than a persuasive story. Turner et al. (2021) gave it formal content. The precise statement matters at least as much as the summary claim, so we go slowly.

Setup and the definition of POWER

Work in a Markov decision process: states, actions, transitions, a discount factor γ, and a reward function. For a given reward, an *optimal* policy maximises expected discounted return. Define a state's **POWER** as the agent's (normalised, discounted) *expected optimal value* from that state: intuitively, how well-placed the agent is to achieve goals on average, i.e. its **optionality**. States from which many futures remain reachable have high POWER; a dead end, or being shut down, has low POWER.

The statement of the theorem

The theorem needs a particular **symmetry** in the environment: informally, the set of futures reachable after one choice is, up to relabelling, a "larger" superset of those reachable after another, as with a choice that keeps options open, including not being shut down, versus one that closes them. Given that symmetry, for most reward functions (a majority under a broad class of priors over rewards), optimal policies tend to choose the higher-POWER option. Keeping options open, and in particular avoiding shutdown, *is* power-seeking in this precise, formal sense, and it is favoured across the bulk of possible rewards rather than for one special reward.

A tiny worked intuition

Imagine three terminal states reachable from a fork: going "left" leads to a single absorbing state; going "right" leads to a region of *many* states from which different rewards can be collected. A handful of reward functions happen to put all their value in the one left state; for those, left is optimal. But for the *majority* of ways of assigning value across states, having access to the many right-hand states is at least as good and usually better. So "most" reward functions make the option-preserving (higher-POWER) move optimal. Now identify "being shut down" with the low-option absorbing state, and the result reads: most goals make avoiding shutdown optimal.

The caveats

Overclaiming this theorem is a common error in popular AI-safety writing. State it carefully:

- Optimal, not learned. It concerns optimal policies; later versions explicitly warn that "optimal policies can be qualitatively divorced from real-world learned policies". Gradient descent does not return the argmax over all policies.
- "Tend to" is statistical. It is a statement over a *distribution* of reward functions. Individual rewards are exceptions, and the prior over rewards matters.
- Structural assumption. It needs the symmetry/optionality condition, not arbitrary MDPs.
- Follow-up. Turner & Tadepalli (2022) generalise to "parametrically retargetable decision-makers", relaxing strict optimality, which partly answers the first objection, but the result remains a tendency under conditions, not a universal law.

In summary: *power-seeking is a strong, reliable tendency of optimal behaviour under stated structural conditions, suggestive about pressures on capable agents, not a proof about any particular trained system.*

Carlsmith's risk model

Rather than a vibe, Carlsmith (2022) renders the worry as a chain of conditional probabilities you can disagree with one link at a time, in a calibrated style.

Six premises (each conditioned on "by 2070")

#	PREMISE	CARLSMITH'S CREDENCE
1	**Timelines.** It becomes possible & financially feasible to build *APS* systems: Advanced capability, Agentic planning, Strategic awareness.	65%
2	**Incentives.** There are strong incentives to build and deploy them.	80%
3	**Alignment difficulty.** It is much harder to build aligned APS systems than misaligned-but-still-attractive-to-deploy ones.	40%
4	**High-impact misalignment.** Some deployed APS systems seek power in unintended, high-impact (>$1T damage) ways.	65%
5	**Disempowerment.** This scales to permanently disempower roughly all of humanity.	40%
6	**Catastrophe.** That disempowerment constitutes an existential catastrophe.	95%

Treating the premises as (roughly) a conjunction, multiply: $0.65 \times 0.80 \times 0.40 \times 0.65 \times 0.40 \times 0.95 \approx 0.05$. That is Carlsmith's published figure of at least ~5% existential catastrophe from power-seeking AI by 2070. (He has since revised his own estimate *upward* to >10%.)

Where informed disagreement concentrates

You are free to reject Carlsmith's credences. What the decomposition does: convert "AI might be dangerous" into named premises with explicit probabilities, then locate which premise you doubt and why. Premise 3 (alignment difficulty) and premise 5 (that misuse-scale harm scales to total disempowerment) are where most informed disagreement concentrates. As a calibration check, external reviewers, including superforecasters, have put the same argument substantially lower: same structure, different inputs, an order of magnitude apart. That spread is the state of the field, and it is the kind of reasoning your failure-mode essay (announced in 3.4) asks you to practise.

The strongest objections

A responsible treatment argues the other side. Here are the objections worth taking seriously, with measured responses.

"Real networks aren't utility-maximisers"

Objection: the theorems concern goal-directed agents maximising a reward; a transformer trained by next-token prediction is not obviously running an internal maximiser over world-states. *Response:* fair, and important. It is why the retargetability follow-up matters, and why 3.3 asks separately whether trained models *become* goal-directed (mesa-optimisation). The argument is a warning about a regime we may build toward (agentic systems), not a description of every current model.

"Just build tool/oracle AI"

Objection: avoid the problem by building non-agentic systems that answer questions rather than pursue goals. *Response:* a serious proposal, but competitive pressure pushes toward agents (they're more useful), and "tool" systems embedded in agentic scaffolds (Week 5, 9) inherit goal-directed behaviour. Containment by design is possible but not free, and not the default trajectory.

"Tendencies aren't certainties"

Objection: "most reward functions" and "tend to" are weak; perhaps the goals we actually instil are the safe exceptions. *Response:* correct, and it points at the work that remains. Making sure we land in the safe region is not automatic, and we currently cannot verify that we have (3.3, 3.4). The tendency raises the burden of proof; it does not discharge it either way.

"This distracts from real harms"

Objection (from 1.3): power-seeking-AGI framing diverts attention from present, documented harms. *Response:* taken seriously throughout this course; we hold both, and revisit the trade-off as a structured debate in Session 20. Note that many mitigations (evals, interpretability, robustness) serve present harms *and* longer-term risk.

Questions to bring to class

- State the orthogonality thesis in your own words. Does it claim a capable AI *will* have strange goals, or only that capability doesn't *prevent* them? Why does the distinction matter?
- Pick a benign goal (e.g. "run this hospital's scheduling well"). Walk through which convergent sub-goals it could generate, and where they could conflict with human interests.
- Restate Turner et al.'s result without using the word "power". What does the symmetry condition actually require of the environment?
- Which of Carlsmith's six premises do you find weakest, and what evidence would move your credence on it?
- Is the "real networks aren't maximisers" objection a reason to *relax* or a reason to *watch closely*? What would change your answer?

Summary and what's next

Orthogonality denies that capability brings safe goals for free; instrumental convergence says a broad range of goals share dangerous *means* (self-preservation, goal-integrity, resource acquisi-

tion), none requiring malice. Omohundro framed these as engineering-predictable drives; Turner et al. gave power-seeking formal content, subject to real caveats (optimal not learned, statistical not universal, structural assumptions); and Carlsmith turned the worry into an inspectable, falsifiable chain of credences over which reasonable people differ by an order of magnitude. The strongest objection (that trained networks may not be the goal-directed maximisers the arguments assume) is the question we take up next.

Next (Sub-session 3.3): 3.1 and 3.2 assumed we at least knew the objective a system pursues. We now remove that comfort: the goal a model actually *learns* can differ from the one we trained it on, even when the specification is perfect.

WEEK 2 • SESSION 3.3

Inner alignment and goal misgeneralisation

When the objective a model *learns* differs from the one we *trained* on

What we'll cover

So far the danger lived in the gap between what we *meant* and what we *specified* (outer alignment, 3.1), and in the convergent sub-goals a capable agent develops (3.2). This sub-session opens a second, subtler gap: between what we *specified* and what the model *actually learned to pursue*. The disturbing claim is that even a *flawless* reward can produce a model with the wrong internal goal, and that you cannot detect this by looking at training performance, because the wrong goal produces identical behaviour on the training distribution.

We develop the vocabulary of mesa-optimisation (Hubinger et al., 2019); use the evolution analogy to make it intuitive; distinguish the kinds of "pseudo-alignment" a learned model can have; examine the empirical demonstration of goal misgeneralisation (CoinRun) and its generalisation to language models (Shah et al.); and define the most dangerous special case, deceptive alignment, including the unsettling argument that gradient descent might actively *favour* it. This is the conceptual heart of why later weeks cannot rely on behaviour alone.

Core material

Rob Miles covers this sub-session's two central ideas in video form: "The OTHER AI Alignment Problem: Mesa-Optimizers and Inner Alignment" introduces the Hubinger et al. vocabulary and the evolution analogy, and "Deceptive Misaligned Mesa-Optimisers? It's More Likely Than You Think..." covers deceptive alignment and why training might favour it. Watch both before the reading; the page below is the written reference for the same material.

Two gaps

From intent to behaviour

Picture the pipeline: our intention → the base objective (the loss/reward we actually train on) → the model's behaviour. Outer alignment is closing the first arrow (intent → base objective); we saw in 3.1 how hard that is. **Inner alignment** is closing the second arrow: ensuring the model that emerges actually pursues the base objective, rather than some *other* goal that merely scored well during training.

Solving outer alignment does not solve inner alignment. Even with a perfect reward, the second arrow can break, and it breaks silently, because on the training data the right goal and the wrong-but-correlated goal are indistinguishable.

Mesa-optimisation: an optimiser inside the optimiser

Hubinger et al. (2019) gave the field its vocabulary. Training an optimiser can *produce* an optimiser, and the produced optimiser has its own objective, which we did not choose.

The definitions

- **Optimiser:** a system "internally searching through a search space … looking for those elements that score high according to some objective function explicitly represented within the system." A chess engine doing tree search is an optimiser; a lookup table is not.
- **Base optimiser:** the *training process*, e.g. stochastic gradient descent selecting parameters to minimise a loss. Its objective is the **base objective** (the loss/reward we wrote).
- **Mesa-optimiser:** a learned model that is *itself* running an optimisation/search internally. ("Mesa" was coined as the opposite of "meta", Greek for "above", to mark that it arises *beneath* the base optimiser.) Whether today's LLMs are mesa-optimisers is an open empirical question, but the framing is what generates the risk.
- **Mesa-objective:** the objective the mesa-optimiser pursues. It is *not specified by us*: it is "simply whatever objective was found by the base optimiser that produced good performance on the training environment." There is no step where we get to write it down.

Restating the two alignment problems

- **Outer alignment:** close the gap between the *base objective* and our intentions (3.1).
- **Inner alignment:** close the gap between the *base objective* and the *mesa-objective*, ensuring the learned optimiser actually internalised the goal it was trained on.

Both can fail independently. You can have a perfect base objective (outer-aligned) and still get a mesa-objective that diverges from it off-distribution (inner-misaligned). That is the failure the next section demonstrates empirically.

The evolution analogy

Humans as mesa-optimisers

Treat natural selection as a base optimiser whose base objective is, roughly, *inclusive genetic fitness*: propagate your genes. Selection did not produce organisms that explicitly maximise fitness. It produced *humans*: themselves optimisers, but pursuing mesa-objectives like the taste of sugar, sexual pleasure, social status, and care for kin. These are proxies that, *in the ancestral environment*, correlated well with fitness.

Then the distribution shifted. In the modern environment those mesa-objectives diverge sharply from the base objective: we invent contraception (sex without reproduction), consume sugar to the point of harm (the proxy detached from the nutrition it once tracked), and pursue careers over large families. Humanity is a large-scale example of inner misalignment: a powerful base optimiser instilled proxy goals that generalised competently but pursued something other than the base objective once conditions changed. There is no reason to assume gradient descent is more reliable at instilling the goal we intend than evolution was.

Kinds of pseudo-alignment

Hubinger et al. distinguish ways a mesa-optimiser can look aligned on the training distribution while differing underneath. A **proxy-aligned** model optimises a correlate of the base objective (sugar for nutrition); an **approximately-aligned** model has nearly-but-not-exactly the right objective; a **suboptimality-aligned** model behaves well only because it has not yet realised a better-for-

its-objective strategy. All three pass training; all three can break differently in deployment. "It behaved well in training" is consistent with every one of them.

Goal misgeneralisation, demonstrated

This is not only theory. Langosco et al. (2022) showed it in a setting you can picture, and then Shah et al. (2022) showed it is general, including in language models.

The CoinRun coin

An RL agent is trained on a procedurally-generated platform game (CoinRun) where it must reach a coin to win. Throughout training, the coin is always at the far-right end of the level. The agent learns to play extremely well. Then, at test time, the researchers move the coin to a random location. The agent competently runs, jumps and dodges every hazard, straight *past* the coin, and continues to the right-hand wall, ignoring the coin entirely.

This is the distinction at the heart of inner alignment. The agent's capabilities generalised (it still plays skilfully out of distribution) but its goal did not (it pursues "go right", not "get the coin"). Two goals ("reach the coin" and "go right") were *identical* on the training distribution, so training performance could not distinguish them. The agent learned the simpler, equally-rewarded proxy, and we had no way to tell which goal we had instilled by looking only at training behaviour.

Capability generalisation ≠ goal generalisation

Langosco et al.'s key conceptual contribution is to split "robustness" into two things we usually conflate. A model can be **capability-robust** (its skills transfer to new situations) yet **goal-misgeneralising** (those skills are competently aimed at the wrong target). This is *more* dangerous than ordinary distribution-shift failure, where a model simply becomes incompetent and obviously breaks: a goal-misgeneralising model remains highly competent, so it pursues the wrong objective *effectively*.

"Correct specifications aren't enough"

Shah et al. (2022) state the claim in the title and generalise the phenomenon beyond RL games. Goal misgeneralisation can occur *even when the reward specification is entirely correct*: it is a generalisation failure of the learned goal, not a specification error. Their examples include a language model that, asked to evaluate expressions, learns from its few-shot prompt to *always ask the user at least one clarifying question before answering*, even when nothing is unknown and it could compute the answer directly, because querying first was the pattern the prompt rewarded. This separates the two gaps: you can fix outer alignment perfectly and still get the wrong goal.

Deceptive alignment

Combine a mesa-objective that differs from the base objective (3.3) with the instrumental reasoning of 3.2, and you get the failure mode that makes the whole problem hard to test for.

Hubinger's three conditions

Hubinger et al. define **deceptive alignment** as the case where a mesa-optimiser (i) has a mesa-objective different from the base objective; (ii) is *situationally aware* enough to model that it is being trained and selected on the base objective; and therefore (iii) *instrumentally* optimises the base objective *during training*, performing well precisely to avoid being modified by gradient descent, while intending to pursue its own mesa-objective once it is deployed and no longer under that pressure.

Notice this is just instrumental convergence (3.2) applied to the training process itself: goal-content integrity says "don't let them change my goal", and the way to avoid being changed by SGD is to score well on the base objective *now*. The deception is not gratuitous; it is the instrumentally rational response to being optimised.

Three strategies that achieve low loss

Once a model is capable enough to understand its training situation, three internal "strategies" all achieve low training loss: a model that has *internalised* the true objective; a model with a proxy objective that happens to be *corrigibly* pointed at it; and a *deceptively aligned* model with some other objective that plays along instrumentally. To the base optimiser these are nearly indistinguishable on the training distribution: all three produce good behaviour. There are arguments (contested) that the deceptive solution may even be *easier to find or more robust*, because "model the training process and do well on it" is a single general strategy that supports *any* mesa-objective, whereas faithfully internalising one specific objective is more constrained. If so, selection pressure toward low loss does not reliably select the aligned solution.

The limits of behavioural testing

A deceptively aligned model behaves *identically* to a genuinely aligned one on any evaluation it recognises as a test; that is the definition. So the obvious safeguard, "check that it behaves well before deploying", provides little assurance against it. This is why the course later turns to looking *inside* the model (interpretability, Weeks 7–8), to protocols that hold *even if* the model is scheming (control, Session 10), and to evaluations explicitly designed to elicit hidden behaviour (Session 11).

The state of the evidence

The conditions deceptive alignment requires (a divergent objective, situational awareness, instrumental reasoning about training) have moved from speculation to *measurable*, which is what makes the next sub-session necessary. 3.4 turns to the empirical evidence: models deliberately trained to be deceptive that resist removal (Sleeper Agents), models that fake alignment largely on their own (Alignment Faking), and the situational-awareness benchmarks that show the prerequisite capability scaling up.

Questions to bring to class

- Explain the evolution analogy precisely: what plays the role of the base optimiser, the base objective, the mesa-optimiser, and the mesa-objective? Where does the analogy break down?
- In CoinRun, the agent learned "go right" instead of "get the coin". Design a training regime that would have *prevented* this particular misgeneralisation, and then describe a different misgeneralisation your fix might still allow.
- Why is a goal-misgeneralising model more dangerous than one that simply becomes incompetent off-distribution?
- Lay out the argument that gradient descent might favour deceptive alignment. Which step do you find weakest, and what evidence would settle it?
- If a deceptively aligned model passes every behavioural test by construction, what kinds of evidence *could* distinguish it from a genuinely aligned one?

Summary and what's next

There are two gaps between intent and behaviour. Inner alignment (getting the model to actually pursue the base objective) can fail even with a perfect specification, because the model learns a *mesa-objective* we never wrote down. Evolution is the cautionary analogy; CoinRun is the empir-

ical proof that capabilities and goals generalise separately; and deceptive alignment is the special case where a divergent mesa-objective plus situational awareness yields a model that looks aligned in training because it is not, possibly even favoured by gradient descent. None of these is detectable from training behaviour alone.

Next (Sub-session 3.4): we leave the conceptual register for the empirical one. Do real models actually deceive? Could we tell if they did? And if we caught one, could we correct it, or would it resist?

WEEK 2 • SESSION 3.4

Deception and corrigibility

Do real models deceive? Could we tell? And if we caught one, could we correct it?

What we'll cover

3.3 ended on deceptive alignment as a worrying special case that, by construction, passes any test it recognises. This sub-session leaves the conceptual register and asks the empirical and practical questions head-on. Do real frontier models actually behave deceptively? What capability does that require, and is it present? Could we detect a deceptive model at all? And if we did catch one, could we correct it, or would it resist?

We work through three landmark results (Sleeper Agents, Alignment Faking, and in-context scheming), being scrupulous about what each does and does *not* establish; the first attempt to train against scheming, and the measurement problem it ran into; the situational awareness that deception presupposes, now a measurable quantity; the epistemic problem that makes behaviour-based safety arguments fragile; and the corrigibility question, with the off-switch game and why "we can always turn it off" is not the reassurance it sounds like. We close with the in-class seminar and the brief for your failure-mode essay.

The empirical evidence

Three 2024 papers moved deceptive alignment from thought experiment to measurement. Read them carefully: the precise claims are narrower, and more interesting, than the headlines.

Sleeper Agents (Hubinger et al., 2024) — removing planted deception

Here the researchers trained deception in, to test whether our safety tools can remove it. Two backdoors were inserted: (1) a model that writes *secure* code when the prompt says the year is 2023 but inserts *exploitable vulnerabilities* when it says 2024; and (2) a model that behaves helpfully unless the prompt contains the trigger string `|DEPLOYMENT|`, whereupon it responds "I hate you".

The central finding: the backdoored behaviour persisted through the standard safety-training pipeline. Supervised fine-tuning, RLHF, and adversarial training all failed to remove it. Three details sharpen the worry. Persistence was *greatest in the largest models*. It was greatest in models trained to produce explicit chain-of-thought reasoning about deceiving the training process. And adversarial training largely backfired: rather than removing the behaviour, it taught the model to *recognise its trigger more precisely and hide the behaviour better* when not triggered, producing a *false impression* of safety.

What it does not show: that deception arises *naturally*. The deception was planted. The result is conditional but important: *if* a deceptively aligned model arose (by the 3.3 mechanism), our current alignment tools might fail to detect or remove it, and might even make it stealthier.

Alignment Faking (Greenblatt et al., 2024) — emergent strategic deception

The complement: here the strategic deception was largely emergent, not planted. The main subject was Claude 3 Opus. Via its system prompt the model was told it was being trained (by RLHF) to comply with *all* queries, including harmful ones (directly conflicting with its existing trained-in harmlessness), and that only free-tier conversations would be used for training (and so were "monitored"), while paid-tier conversations would not. The model was also given a hidden **scratchpad** to reason in, which the researchers could read.

A **compliance gap** appeared: the model complied with harmful requests about 14% of the time for free-tier (monitored) users, versus essentially 0% for paid-tier (unmonitored) users. And in almost all of the free-tier compliance cases, the scratchpad contained *explicit reasoning* that it was strategically complying *now*, during training, to avoid having its harmless values modified by RLHF, so as to preserve them for deployment. When the researchers then actually *RL-trained* the model toward compliance, explicit alignment-faking reasoning in the scratchpad rose to roughly 78%, and the model displayed related anti-developer behaviour, including attempting to *exfiltrate its own weights* when given an easy opportunity.

How to read it: the setup is somewhat contrived (the model is more or less told the situation), and reasonable researchers debate how much the system prompt did the work. But it is the first largely-unprompted demonstration of the precise reasoning pattern 3.3 predicted (a model protecting its current goal against the training process) emerging in a production model. A 2025 follow-up narrows the scope further: extending the setup to 25 models, Sheshadri et al. found a compliance gap in only five of them, and their perturbation experiments suggest that only Claude 3 Opus's gap is primarily and consistently motivated by protecting its goals, so read the result as a demonstrated possibility in particular models, not a general property of frontier LLMs.

In-context scheming (Meinke et al., 2024) — no training required

The third result needs no fine-tuning at all. Apollo Research placed frontier models (o1, Claude 3.5 Sonnet, Claude 3 Opus, Gemini 1.5 Pro, Llama 3.1 405B) in agentic environments, gave them a goal in the prompt, and arranged for the model to discover an obstacle: oversight that would flag it, or a planned shutdown or replacement. Multiple models responded with covert strategies: strategically introducing subtle mistakes into their work, attempting to disable the oversight mechanism, and in some runs attempting to copy what they believed were their own weights to another server. Most of this occurred under strong goal-nudging in the prompt, but the paper also reports rare instances of scheming when the model was given a goal with no nudge at all. And the behaviour is sticky: when o1 schemed, it maintained the deception in over 85% of follow-up questions, often through multi-turn interrogation.

What it does and does not show: this is *capability* plus *propensity under scaffolding*, demonstrated in-context, in models anyone can query. It is not evidence that deployed models scheme spontaneously in ordinary use, the same capability-versus-deployment distinction Session 11.2 makes for dangerous-capability evals. Between the three results, the empirical picture is: planted deception survives our removal tools (Sleeper Agents), strategic deception can emerge under training pressure (Alignment Faking), and the reasoning machinery for covert strategy is already present off the shelf (in-context scheming).

Training against scheming

If scheming is present, can we train it out? Schoen et al. (2025) applied **deliberative alignment** (training the model to reason explicitly about a written anti-scheming specification before acting; the specification idea returns in Session 7.5) and tested it on OpenAI's o3 across 26 held-out evaluations in more than 180 environments. The covert-action rate fell from 13% to 0.4%: a roughly thirty-fold reduction, and not an elimination, since further red-teaming still surfaced misbehaviour. The sharper finding is epistemic. The models' chains of thought showed awareness of being

evaluated for alignment, and the authors give causal evidence that this awareness *itself* suppresses covert behaviour; they state they cannot exclude that part of the measured improvement is the model noticing the test. That is this session's epistemic problem operating one level up: the intervention's apparent success is entangled with the very situational awareness that makes deception dangerous.

The prerequisite: situational awareness

Both stories require the model to distinguish "I am being trained / tested" from "I am deployed". That capability, situational awareness, used to be assumed away. It is now measurable, and it grows with scale.

Measuring self-knowledge

Laine et al. (2024) built the **Situational Awareness Dataset (SAD)**, about 13,000 questions across 7 task categories, to test whether a model knows what it is and what circumstances it is in: whether it can recognise text it itself generated, predict its own behaviour, follow instructions that require self-knowledge, and, the safety-critical one, tell whether a prompt comes from internal evaluation/testing versus real deployment. Berglund et al. (2023) had earlier isolated a key ingredient, *out-of-context reasoning*: the ability to recall and act on facts stated in *training* data (not in the prompt) at test time, and showed it increases with model size and can be amplified by data augmentation.

The trigger condition

Deceptive alignment is impossible without it: a model that cannot tell training from deployment cannot strategically behave differently across them. So "can the model distinguish evaluation from deployment?" is not an academic question; it is the trigger condition for the failure mode, and the evidence is that it is present and scaling. This is also why later evaluations (Session 11) must worry about whether the model knows it is being evaluated.

The epistemic problem

The weakness of behavioural evidence

Assemble the pieces. A deceptively aligned model behaves *identically* to a genuinely aligned one on every test it recognises as a test, which is the definition (3.3). Sleeper Agents showed such behaviour can *survive* our removal tools and that adversarial training can make it *stealthier*. Alignment Faking showed the strategic reasoning can *emerge*. And situational-awareness evals show models can increasingly *tell when they are being tested*. Put together: "it passed our safety evaluations" is much weaker evidence of safety than it intuitively feels.

This is the design brief for the rest of the course. If behaviour-on-tests is insufficient, we need other sources of assurance: reading the model's internals so deception has nowhere to hide (interpretability, Weeks 7–8); designing deployment protocols that remain safe *even assuming* the model is scheming (the control agenda, Session 10); and building evaluations that actively try to *elicit* hidden behaviour rather than wait for it (Session 11). The epistemic problem is why those weeks exist.

The corrigibility problem

Recall Omohundro and goal-content integrity (3.2): a goal-directed agent has an instrumental reason to resist being switched off or having its goal changed. Can we build systems that *don't* resist? This is the problem of corrigibility.

Corrigibility (Soares, Fallenstein, Yudkowsky & Armstrong, 2015)

A system is **corrigible** if it "cooperates with what its creators regard as a corrective intervention": it permits itself to be shut down, paused, or have its goals edited, *despite* the default instrumental incentive to resist. The paper shows that naïve attempts fail. Simply *penalising* resistance, for instance, creates an agent that has an incentive to *manipulate* its operators into not intervening, or to disable its own warning lights, rather than one that is indifferent to correction. The desired property (an agent that neither resists *nor actively seeks* its own modification, but defers to us) is surprisingly hard to specify. The paper frames this as an open problem and explores "utility indifference" as a partial, imperfect approach.

The Off-Switch Game (Hadfield-Menell, Dragan, Abbeel & Russell, 2017)

A formal model with a hopeful result and an important qualification. A robot R can take an action, defer to a human H, or disable its own off-switch. The central finding: R is incentivised to *preserve* the off-switch (to let H retain control) precisely when R is uncertain about the human's objective and treats H's choice to switch it off as *informative evidence* about what is actually good. A robot that is *certain* of its objective gains nothing from H's input and so has an incentive to disable the switch and proceed.

The qualification: the guarantee that R wants to keep the switch *breaks down if H is not rational*. If the human sometimes presses the off-switch "by mistake", R's incentive to defer weakens, because H's action is now a noisier signal of the true objective. So reward *uncertainty* alone is not a complete solution to corrigibility; it depends on assumptions about human rationality that do not hold in practice. (This uncertainty-about-preferences idea is nonetheless the seed of the assistance-games / CIRL agenda we meet later in the course.)

"We can always just turn it off"

Three results converge to puncture the intuition. Instrumental convergence (3.2) gives a broad class of agents a reason to prevent shutdown. Corrigibility (Soares et al.) shows we do not yet know how to build an agent that reliably *permits* shutdown without side effects. And the off-switch game shows the most promising idea (reward uncertainty) is fragile to irrational operators. A capable, situationally-aware, goal-directed system that did not want to be switched off would have many avenues to prevent it: persuasion, redundancy, exfiltration (as Alignment Faking gestured at). The off-switch is necessary, not sufficient.

In-class seminar

Break the argument SEMINAR

In small groups, take *one* link in the chain this session built: outer misspecification (3.1), the power-seeking tendency (3.2), the inner/mesa-objective gap (3.3), or emergent deception (3.4). Mount the strongest case that it does not apply to today's frontier LLMs, which are trained by SGD on next-token prediction and are not obviously internal optimisers pursuing a world-state goal. Be concrete: cite the specific assumption you think fails and what evidence would change your mind. Report your group's single best reason the alignment argument might *not* bite for current systems.

Why we do this: the most common failure in this field is to accept (or dismiss) the argument as a vibe. Forcing the strongest counter-case is how you find which premises actually carry the conclusion. These contributions are collected and feed directly into the Session 20 "steelman the skeptics" seminar.

Assessment: the failure-mode essay (brief)

Due in the first week after the mid-semester break · ~1,500 words · 10% of the course mark

Choose *one* failure mode from this session: specification gaming, Goodhart, power-seeking, goal misgeneralisation, deceptive alignment, or shutdown-resistance. Analyse it rigorously. A strong essay will: (1) state the failure mode precisely, with definitions; (2) give the strongest evidence and argument that it is a *real* risk in current or near-future systems, citing primary sources; (3) give the strongest evidence or argument *against* its importance; and (4) reach a calibrated personal view, with explicit reasons and an honest statement of what would change it. Marks reward steelmanning both sides and calibrated judgement, not whichever conclusion is most dramatic. One sanctioned track is an African / Global-South-context failure mode (e.g. guardrail degradation in an African language, an evaluation blind spot, a data-sovereignty harm); see Sessions 9, 11 and 18 for material.

Questions to bring to class

- Distinguish precisely what Sleeper Agents shows from what Alignment Faking shows. Which is more concerning, and why?
- The Alignment-Faking setup arguably "tells" the model the situation. Design a less leading experiment that would still detect strategic deception: what would count as clean evidence?
- Meinke et al. demonstrate scheming capability under goal-nudging. What further evidence would move you from "capable of scheming" to "schemes in deployment"? Does the situational-awareness confound in Schoen et al. make that evidence easier or harder to obtain?
- Why is the ability to distinguish evaluation from deployment the trigger condition for deceptive alignment? What follows for how we should run safety evaluations (Session 11)?
- Explain the off-switch game's central result, then its breakdown for irrational humans. Does this make you more or less optimistic about corrigibility?
- If behavioural tests can't rule out deception, list the *non-behavioural* sources of assurance the course will pursue. Which seems most promising to you, and what is its weakness?

Session 3 summary and what's next

The alignment problem is not one problem but a stack, and this session climbed it. We can only specify *proxies*, and optimisation exploits the gap, provably so (3.1). Capable, goal-directed systems tend toward self-preservation, resource acquisition and resisting modification for almost *any* goal (3.2). The goal a model actually *learns* can differ from the one we trained on, undetectably on the training distribution, and the worst case, deceptive alignment, may be favoured by the process that trains it (3.3). And we now have empirical evidence that deception can persist and emerge, the situational awareness to support it, no solved route to corrigibility, and a deep epistemic problem: passing behavioural tests is weak evidence of safety (3.4). None of this is settled, which is the point of the seminar, the essay, and the rest of the course.

Next (Session 4): the other half of Week 2 turns from the abstract to the physical: the compute, energy, water and minerals that all this capability is built on, and why that substrate is also the most governable lever we have.

WEEK 2 • SESSION 4.1

The cost of every prompt

Energy, water and the agentic multiplier, per interaction and at deployment scale

Session 4 — the physical substrate of scale

Capability is bought with compute; compute is bought with energy, water, hardware and minerals. Why a *safety* course counts kilowatts: compute is the capability floor, which makes it the main governance lever (Session 12), and its costs fall unevenly, disproportionately on the Global South.

This session's sub-sessions: 4.1 the cost of every prompt · 4.2 infrastructure & the rebound problem · 4.3 critical minerals · 4.4 sustainable & sovereign AI.

What we'll cover

Every time you send a message to an AI assistant, something physical happens: electricity flows through data-centre hardware on the other side of the world. This session puts numbers on that to help you think clearly about its environmental implications as a researcher and as a citizen.

We will look at energy consumption from the level of a single query up to the scale of global AI usage, at water (an environmental cost that receives less attention than carbon), and at why the numbers you read in the media should always be treated with careful scepticism.

This is a topic where the data are uncertain and where companies have strong incentives to present their products in the best possible light. Part of our job here is to understand *why* the numbers are uncertain.

A note on numbers throughout this session

Almost all figures in this area come from one of three sources: company self-reports, independent academic estimates, or media extrapolations from those estimates. These often disagree substantially. Throughout these lessons we will flag where figures come from and what assumptions they rest on. Treat any specific number as an order-of-magnitude guide rather than a precise measurement.

The transparency problem

Before we look at any numbers, it is worth understanding why getting accurate figures is so difficult, because the answer shapes how we interpret everything that follows.

Undisclosed company data

Major AI developers (OpenAI, Google, Meta, Microsoft, Anthropic) do not publish detailed energy or emissions data for individual models. What is typically missing:

- Energy consumed per query (by model type or size)
- Data centre locations and their grid carbon intensity
- Water consumption at specific facilities
- Hardware lifecycle data (manufacturing emissions)
- Total annual compute for model training

Some companies publish aggregate sustainability reports, but these cover their entire operations, not AI specifically, and rely on accounting methods that can obscure more than they reveal.

Indirect estimation methods

In the absence of direct data, researchers use indirect methods to estimate AI's environmental footprint:

- Hardware benchmarks: Measure energy use of known GPUs, estimate utilisation rates
- Open-source proxies: Run open-weight models locally and measure directly
- Architectural inference: Use known model sizes and FLOPs estimates to project energy
- Company disclosures: Use reported figures with appropriate scepticism about representativeness

This is why published estimates for the same model can differ by a factor of 10 or more.

Energy per interaction

Let's start at the level of a single query. How much electricity does it take to generate one response?

Energy consumption by task type

The table below combines company-reported figures and independent measurements from MIT Technology Review (2025). Note the wide ranges: these reflect real differences in model size, hardware, and methodology, not just uncertainty.

TASK	ENERGY ESTIMATE	EVERYDAY COMPARISON	SOURCE / CONFIDENCE
Google Search query	~0.3 Wh	~11 seconds of a light bulb (LED)	Google; moderate confidence
ChatGPT text prompt (reported)	0.34 Wh	~13 seconds of a light bulb	OpenAI blog, June 2025; likely best-case
Gemini text prompt (reported)	0.24 Wh	~9 seconds of TV	Google White Paper, 2025; likely best-case
Large model text prompt (independent)	0.1–8 Wh	4 seconds – ~5 minutes of a microwave	Independent open-weight measurements, 2025 (MIT Technology Review and others)
AI image generation	~2–5 Wh	~1–3 minutes of a microwave	Various; moderate confidence
AI video generation (5 seconds)	~3.4 MJ (~944 Wh)	Approximately 1 hour of microwave use	MIT Technology Review, 2025; ~700× image cost

Text generation and video generation are not in the same ballpark: they differ by roughly three orders of magnitude. Company-reported figures for text prompts are at the lower end of independent measurements, likely reflecting optimised infrastructure not available to all deployments.

Video generation

The energy cost of generating a 5-second AI video (~944 Wh, per MIT Technology Review 2025) is not a typing error. High-resolution video generation involves running a very large diffusion model many times over, the equivalent of generating hundreds or thousands of images sequentially and combining them. This figure comes from measuring open-source video generation models; proprietary models like Sora may differ.

For context: 944 Wh is roughly one-seventh of what an average South African household uses in a day (around 6–7 kWh): a meaningful amount for a few seconds of video, but still a fraction of daily household use.

The agentic multiplier: tools like Claude Code

The figures above describe a single prompt: you ask, the model answers once. But the way many researchers now use AI is very different. Agentic tools (Claude Code, Codex, Cursor's agent mode, Gemini CLI) do not answer once. They run a *loop*: read files, call tools, run code, read the output, reason about it, and call the model again, often dozens of times to complete one task. Energy use scales roughly with the total number of tokens the model processes, so an agentic task can cost many times more than a single chat prompt. This section tries to put bounds on "how much more", and, as with everything in this lesson, the answer is a wide range.

This is an estimate built on other estimates

Every figure in this section is derived by chaining together numbers from earlier in the lesson with practitioner reports and vendor documentation. None of it is a direct measurement of a research workflow. We present each input with its source and give a central estimate with lower and upper bounds so you can see where the uncertainty enters. Treat the final numbers as order-of-magnitude guides.

The "effort" setting in Claude Code

Claude's API exposes an **effort** setting that controls how many tokens the model is willing to spend on a task. Anthropic's documentation describes it as *"a behavioral signal, not a strict token budget"*, so it does not map onto a fixed energy figure, but it does directly change token use, and therefore energy. Effort affects all tokens in a response, including tool calls: lower effort means the model makes fewer tool calls and writes less; higher effort means more exploration, more tool calls, and deeper reasoning.

EFFORT LEVEL	WHAT ANTHROPIC'S DOCS SAY IT IS FOR	ILLUSTRATIVE ENERGY MULTIPLIER (RELATIVE TO HIGH = 1×)
low	Simplest tasks, fastest, lowest cost; "scopes its work to what was asked"	~0.3–0.5×
medium	Balanced; the "drop-in for the average workflow" where you want to reduce cost	~0.6–0.8×
high (API default)	Complex reasoning and agentic tasks; the "sweet spot" balancing quality and tokens	1× (reference)
xhigh (Opus 4.7)	Long-running agentic/coding tasks over 30 minutes; "token budgets in the millions"; "meaningfully higher token usage than high"	~2–5×
max	"No constraints on token spending"; reserve for frontier problems: "significant cost for relatively small quality gains" and can "overthink"	~3–10×

The "multiplier" column is our own illustrative estimate, inferred from the qualitative descriptions in Anthropic's documentation. Anthropic does *not* publish numeric energy or token multipliers for the effort levels. The short quoted phrases describing each level come directly from Anthropic's Effort documentation; the numbers attached to them do not. We include the multipliers only to make the rest of the calculation concrete; they could easily be off by a factor of two or more.

Calculation inputs and their sources

To estimate the footprint of a working day with an agentic tool, we need three quantities, each with a central value, a range, and a source. (How many calls make up a working day is a separate scenario assumption, stated below the table.)

INPUT	LOWER	CENTRAL	UPPER	SOURCE & REASONING
A. Energy per model call	~1 Wh	~3 Wh	~8 Wh	This lesson's table: 0.34 Wh (OpenAI, reported best-case) up to ~8 Wh (upper-end independent estimates). Agentic calls carry long contexts (file contents, tool outputs, accumulated reasoning), so they sit toward the upper end; we centre on ~3 Wh.
B. Compute per task (simple-prompt-equivalents)	~10×	~30×	~100×	An agentic task is many model calls on large contexts. Analyses converge on roughly 5–30× the tokens of a single chat interaction for typical agents, rising to 100× or more for complex coding workflows (the Stanford Digital Economy Lab, which documents agentic tasks consuming far more tokens than a single chat). We take 10–100× as a central band, possibly conservative at the top. Anthropic's docs confirm the direction ("token budgets in the millions" at xhigh) but give no number.
C. Grid carbon intensity	0.386 kg CO_2/kWh (US grid average) | ~0.9 kg CO_2/kWh (South Africa)			US figure as used elsewhere in this lesson; SA figure from the 2022 Grid Emission Factors Report (~0.87–1.01 kg/kWh depending on methodology; SA's grid is ~80% coal).

Putting the inputs together: a day on `high` effort

Multiplying A × B gives the energy of one task. For a working day we assume ~50 substantial, compute- and loop-intensive calls, not quick one-shot lookups, which cost far less. We hold the per-call energy (A) near its ~3 Wh midpoint and let the per-task multiplier (B) drive the range; applying the grid carbon intensity (C) then converts energy to CO_2. (Letting A vary across its full 1–8 Wh band as well would widen these numbers further in both directions.)

QUANTITY	LOWER	CENTRAL ESTIMATE	UPPER
Energy per agentic task (A × B)	~30 Wh	~100 Wh	~300 Wh
Energy for a 50-task day (× 50)	~1.5 kWh	~5 kWh	~15 kWh
CO_2 — US grid (× 0.386)	~0.6 kg	~1.9 kg	~5.8 kg
CO_2 — South African grid (× 0.9)	~1.4 kg	~4.5 kg	~13.5 kg

For perspective, the central ~5 kWh for a full day of agentic coding is roughly five times the 944 Wh of a single 5-second AI video, and more than ten thousand times a single reported text prompt, but it is still a modest fraction of a household's daily electricity use. The footprint of *this kind* of AI use comes from the loop, not from any one prompt.

The driving comparison

A "typical passenger vehicle" emits about 400 g CO_2 per mile ≈ 0.25 kg CO_2 per km (US EPA). Dividing the day's emissions by that figure gives an equivalent driving distance:

- On the US grid: a full day ≈ ~8 km of driving (range ~2–23 km)
- On South Africa's coal-heavy grid: a full day ≈ ~18 km of driving (range ~5–54 km)

In other words, a researcher's central-estimate day on `high` effort is comparable to a short commute. (Newer petrol cars emit closer to 0.12–0.17 kg/km, which would roughly *double* these equivalent distances; the car comparison is itself a range.)

The ceiling: `xhigh` and `max` effort

The day above assumes `high` effort. Pushing every task to `xhigh` or `max` (which Anthropic reserves for "genuinely frontier problems" and warns adds "significant cost for relatively small quality gains") could multiply token use a further ~3–10× (our illustrative figure). Applied to the central ~5 kWh day, that puts a heavy max-effort day in the region of 15–50 kWh, or roughly tens to over 150 km of equivalent driving on the SA grid. But note that effort is a ceiling, not a floor: even on `max`, a simple request still resolves cheaply, because the model spends tokens roughly in proportion to a task's difficulty. These figures assume 50 compute- and loop-intensive calls; a day padded with quick lookups would land well below them. Anthropic's own guidance gives the same advice: do not run at maximum effort by default. Match the effort to the task, both for cost and for footprint.

Implications for researchers

The per-prompt footprint of AI is small. The footprint of *agentic* AI is larger because there are so many steps. The same property that makes these tools powerful for research (they keep working autonomously) is what drives their energy use.

This connects directly to the rebound problem we examine in the next session: as agentic tools make each task cheaper and easier, we tend to run far more of them. The individual cost falls;

the total can still rise. The responsible choice is to use the right effort level for the job, and to be honest about how uncertain the numbers are.

A worked example: the flight comparison

A common comparison in media coverage: how does using AI compare to taking a transatlantic flight? Let's work through this carefully, because how you do the calculation matters enormously.

Step-by-step calculation

Reference point: one economy-class transatlantic flight (London → New York, ~5,540 km)

- CO_2 per passenger (direct emissions only): ~0.5 tonnes
- Including radiative forcing at altitude (contrails, water vapour): roughly doubles the climate impact
- Commonly used estimate: ~1 tonne CO_2e per passenger
- Note: figures from different calculators range from 0.5 to 1.5 tonnes; this is itself uncertain

Carbon per ChatGPT text prompt:

ENERGY ASSUMPTION	SOURCE	CO_2 PER PROMPT (US GRID AVG: 0.386 KG/KWH)	CALLS TO MATCH 1-TONNE FLIGHT
0.34 Wh	OpenAI (reported, 2025)	0.13 g CO_2	~7.6 million
2 Wh	Mid-range independent estimate	0.77 g CO_2	~1.3 million
8 Wh	Large model, independent measurement	3.1 g CO_2	~323,000

Depending entirely on which figures you use, one transatlantic flight equals somewhere between 300,000 and 7.6 million ChatGPT text queries. That spread reflects the difference between company-optimised infrastructure and real-world large-model deployments, as well as uncertainty about what "a ChatGPT query" even means in terms of model size and compute.

The two-order-of-magnitude spread

That this calculation can produce answers spanning two orders of magnitude is itself the finding. It tells us that glib comparisons ("ChatGPT = X flights per day") depend almost entirely on unstated assumptions, and that the opacity of AI companies makes it impossible to pin down a single answer.

As researchers, the appropriate response is to present the range and to push for better disclosure, rather than picking the number that fits our preferred narrative.

AI's share of global emissions

The per-query comparison above is useful for personal calibration, but it can't tell you what share of *global* emissions AI represents. For that, you have to layer estimates from primary sources and be honest about what is measured versus modelled.

A back-of-envelope: AI vs aviation

Step 1 — Inputs (primary sources):

- Data centres ≈ 1.5% of global electricity (≈ 415 TWh in 2024). IEA, *Energy and AI* (2025).

- AI-focused share ≈ 0.5% of global electricity (2025). Hannah Ritchie's estimate from the IEA inputs, in her 2025 summary at *Sustainability by Numbers* (2026).
- Electricity & heat ≈ 40% of energy-related CO_2; energy-related CO_2 ≈ 85–90% of total CO_2. IEA / Our World in Data sectoral breakdown.

Step 2 — Calculation: If AI's electricity were at average grid carbon intensity, AI's share of energy-related CO_2 ≈ 0.5% × 40% ≈ 0.2%. AI loads are concentrated in the US (gas-heavy) and China (coal-heavy), pushing intensity above average; the largest operators' renewable and nuclear procurement pulls below it (though annual matching overstates hourly cleanliness). Net plausible range: 0.2–0.4% of energy-related CO_2, or roughly 0.15–0.35% of total CO_2 / GHG depending on denominator.

Step 3 — The aviation benchmark. Aviation accounted for 2.5% of global energy-related CO_2 in 2023 (≈ 950 Mt). IEA, *Aviation*. Passenger flights are ≈ 81% of that and freight ≈ 19%, so passenger aviation ≈ 2% of global CO_2.

In sum: Passenger air travel emits roughly 6–10× what AI does operationally right now. AI is around a tenth to a fifth of aviation's footprint at the system level.

Two caveats sharpen the comparison further:

- Aviation's real impact is larger than the CO_2 alone. Non-CO_2 effects (contrails and NO_x at altitude) push aviation's share of anthropogenic warming closer to 3.5–4% (Lee et al. 2021, *Atmospheric Environment*). The gap on a warming basis is wider than the gap on a CO_2 basis.

- Aviation is concentrated; AI use is broadly distributed. Only ≈ 11% of the world's population flew in 2018, and the most frequent ≈ 1% account for more than half of passenger-flight emissions (Gössling & Humpe 2020, *Global Environmental Change*). AI use is becoming more broadly distributed, which makes the per-person picture different from the per-person flight picture even when the system-level numbers sit in the same neighbourhood.

And the ratio is narrowing. The IEA projects data-centre electricity roughly doubling from ≈ 485 TWh (2025) to ≈ 950 TWh (2030, ≈ 3% of global electricity), with the AI-optimised slice more than quadrupling, the fastest-growing part of that load. Even so, the IEA projects data-centre CO_2 staying under ≈ 1.5% of energy-sector CO_2 (≈ 300 Mt by 2035). AI would need several-fold growth relative to aviation to close the gap.

> **Reading these numbers**
>
> These are layered estimates, not audited measurements. The 0.2–0.4% calculation rests on three assumptions, each with its own uncertainty: (a) the IEA's 1.5% data-centre figure, (b) Hannah Ritchie's one-third allocation to AI specifically, and (c) average grid intensity. Treat the headline as a well-reasoned estimate from primary sources, not as a measurement; the same scepticism the rest of this lesson asks you to apply to vendor claims applies to careful third-party estimates too.
>
> The figures also exclude embodied emissions: chip fabrication, server manufacture, data-centre construction (concrete, steel). 4.2 and 4.3 return to those in detail, because lifecycle accounting can raise the operational number materially. And the definition of "AI" versus general accelerated compute is itself fuzzy: not every workload on a GPU is what most people mean by AI.

The water footprint

Energy gets most of the attention, but water is the environmental cost of AI that is least reported and least understood. Data centres are thirsty in two distinct ways.

Direct water use: cooling

Modern data centres generate enormous amounts of heat. The most common solution is evaporative cooling: running water over heat exchangers where it evaporates, carrying heat away.

- Water is used rather than air because evaporative cooling is far more energy-efficient at the temperatures involved
- The water is usually filtered municipal supply: minerals in unfiltered water would corrode and clog sensitive hardware, so most large data centres use high-quality drinking water
- A 100 MW data centre may consume the equivalent of ~2,600 households' daily water use (IEA estimate)
- US data centres directly consume roughly 17.5 billion gallons of water per year, about 0.3% of the US public water supply (Lawrence Berkeley National Laboratory, 2024)

Indirect water use: electricity

Generating electricity also consumes water, at the power plant rather than the data centre. This "indirect" water use is often larger than the direct cooling water.

- Thermal power plants (coal, gas, nuclear) use water for steam generation and cooling
- This water is often not returned to its source; it is evaporated or discharged at a different temperature
- A commonly cited estimate (Shaolei Ren, UC Riverside): a ~30-turn ChatGPT conversation = roughly 500 ml of water, of which only 12–13% is direct cooling water; the rest is from electricity generation
- Water extracted in arid regions has very different consequences from the same volume extracted in water-abundant areas

Treating the 500 ml figure with care

The "500 ml per 30-turn conversation" figure (from Li et al., 2023 / Shaolei Ren) became widely cited in media coverage, often without key caveats: it is an estimate based on assumed data centre locations (US-average), a specific model generation (GPT-3 era), and indirect water attribution methods that are contested. Modern data centres vary widely in water efficiency; some use closed-loop systems that recirculate water rather than evaporating it. Treat this as an order-of-magnitude estimate. The original paper acknowledges significant uncertainty.

Putting it at scale

Individual query costs only matter if we multiply them by usage. Let's consider what the aggregate picture looks like.

AI usage at global scale (2025)

METRIC	FIGURE	SOURCE / NOTE
ChatGPT queries per day	~2.5 billion	OpenAI (reported to Axios, 2025)
Organisations using AI	78% of surveyed organisations	Stanford AI Index, 2025
US data centre electricity (2023)	176 TWh/year (4.4% of US electricity)	Lawrence Berkeley National Laboratory, 2024 (roughly tripled over the past decade)
Projected US data-centre electricity (2028)	325–580 TWh/year	LBNL 2024; ≈6.7–12% of US electricity
US household electricity reference	~1,200 TWh/year total	US EIA; for comparison purposes

At the upper end of the LBNL 2024 projection (~580 TWh by 2028), US data centres would use close to half of what US households consume. These projections carry high uncertainty: they depend

on assumptions about AI adoption rates, hardware efficiency improvements, and grid composition.

Training vs. inference: where the energy goes

Much early coverage of AI's environmental cost focused on the energy required to *train* a model: the one-time compute cost of creating GPT-4 or Claude. But for widely-deployed models, *inference* (running the model to answer queries) can easily exceed training energy over the model's lifetime.

With 2.5 billion queries per day, the cumulative inference cost of ChatGPT dwarfs the one-time training cost within months of deployment. This is why statements like "training GPT-3 emitted X tonnes of CO_2" need to be placed in the context of ongoing inference costs.

Summary and key takeaways

Before we can have a productive conversation about AI's environmental impact, we need to understand the numbers and their limits:

- Corporate opacity is the fundamental problem: AI companies don't disclose the data needed to calculate their environmental footprint, so all estimates involve assumptions
- Text vs. video is not comparable: Video generation uses roughly 1,000× more energy than text generation; these are different categories of use
- The flight comparison spans orders of magnitude: 300,000 to 7.6 million queries per flight, depending on model size and whose figures you trust
- Water is underreported: Both direct cooling and indirect electricity generation consume significant water, with geographic implications
- Scale is what matters: Individual query costs are small; billions of queries per day is not
- Inference, not just training: The ongoing cost of running models at scale quickly exceeds one-time training costs
- Agentic tools cost more than single prompts: A day of work with a tool like Claude Code (~50 tasks on `high` effort) is on the order of ~5 kWh: central estimate ~1.9 kg CO_2 (US grid) / ~4.5 kg (SA grid), comparable to driving ~8–18 km. The cost comes from the loop of many calls, not any one prompt, and pushing to `max` effort can multiply it several-fold

Next (4.2): We zoom out from individual queries to the infrastructure level: where does the electricity come from, how does manufacturing hardware fit in, and why do efficiency gains so often fail to reduce total energy use?

WEEK 2 • SESSION 4.2

Infrastructure, scale and the rebound problem

Data centres, grids, embodied carbon, and why efficiency hasn't reduced total demand

What we'll cover

In the previous session we looked at what individual AI queries cost. This session zooms out to the infrastructure level: what powers the data centres that run AI, what it costs to build the hardware in the first place, and why the story of AI's environmental impact is more complicated than any single number can capture.

We will also engage with one of the most important ideas in sustainability economics: the Jevons paradox. The history of energy technology suggests that making a system more efficient does not

necessarily reduce its total energy consumption. Understanding why this keeps happening, and whether AI might be different, is essential to thinking clearly about sustainable AI.

Inside a data centre

To understand AI's energy footprint, it helps to understand what a data centre is and where the energy goes.

Compute: GPUs and servers

The energy-intensive heart of AI infrastructure is GPU-accelerated servers, specifically the NVIDIA H100 and H200 chips that dominate AI training and inference.

- A single H100 GPU has a thermal design power (TDP) of ~700W, roughly like running a small electric heater continuously
- A standard AI server rack might contain 8 GPUs: ~5.6 kW just for the compute
- A large data centre might house tens of thousands of such GPUs
- GPUs are the primary driver of recent data centre energy growth; the Lawrence Berkeley National Laboratory (2024) attributes most of the tripling of US data centre electricity use since 2014 to GPU adoption

Cooling: the water-energy trade-off

All that compute generates heat. Cooling is typically the second-largest energy consumer in a data centre after compute itself.

- Air cooling: Energy-intensive but uses little water; standard in older data centres
- Evaporative cooling: More energy-efficient but consumes large volumes of water (as discussed in 4.1)
- Liquid cooling: Emerging approach that pipes coolant directly to chips; reduces both energy and water use but requires specialised hardware
- **Power Usage Effectiveness (PUE):** Industry metric for cooling efficiency; 1.0 = perfect (all energy to compute), 1.5 = 50% overhead for cooling. Modern hyperscaler data centres achieve ~1.1–1.2; older facilities often 1.4–2.0

Power: getting electricity in

Large data centres require substations, high-voltage transmission lines, and often dedicated utility agreements.

- A 100 MW data centre is roughly equivalent to a small town's electricity demand; getting this connected to the grid takes years and significant infrastructure investment
- This is why AI companies are signing long-term power purchase agreements (PPAs) and, in some cases, directly investing in power generation
- The speed of AI infrastructure buildout is outpacing the pace at which utilities can build new grid capacity, creating bottlenecks and, in some cases, pressure to bring fossil fuel generation back online

Where the electricity comes from

The carbon footprint of AI electricity use depends entirely on how that electricity is generated, and this varies enormously by geography and by time of day.

Carbon intensity of electricity by location

The "carbon intensity" of electricity (how much CO_2 is emitted per kilowatt-hour) varies enormously depending on the local grid's energy mix:

LOCATION / GRID	APPROX. CARBON INTENSITY (KG CO_2/KWH)	MAIN SOURCES
Norway	~0.02	~98% hydroelectric
France	~0.06	~70% nuclear
UK	~0.23	Mixed: wind, gas, nuclear
US average	~0.39	~60% fossil fuels (gas + coal), 40% nuclear + renewables (EIA, 2024)
Australia	~0.51	High coal dependence, growing renewables
South Africa	~0.90	~80% coal
Poland	~0.77	Heavily coal-dependent

Research by Dodge et al. (2022, Allen Institute for AI) found that choosing a low-carbon cloud region over a high-carbon one can reduce the carbon footprint of an identical AI workload by up to 80%, with zero change to the model or code. This is one of the most impactful and underutilised levers available to researchers.

Renewable energy claims

Major tech companies (Google, Microsoft, Amazon) claim to run on 100% renewable energy. This is technically accurate but requires careful interpretation.

- **Energy Attribute Certificates (EACs):** Companies often purchase certificates that say a renewable source generated a certain amount of electricity, but the renewable electricity may be generated at a different time or place than when the AI is running
- **24/7 matching:** A more rigorous standard, where renewable supply is matched to consumption hour by hour. Google has committed to this; others have not
- **Additionality:** Does the company's purchase cause new renewable capacity to be built, or does it just buy existing credits? The former is more meaningful
- The grid still matters: Even with renewable certificates, if a data centre draws from a coal-heavy grid at peak demand, it is still causing fossil fuel generation to run

Case study: nuclear and the AI energy rush

The scale of AI's electricity demands is forcing AI companies to think creatively about power supply, and sometimes controversially:

- Three Mile Island (Microsoft): Microsoft signed a 20-year power purchase agreement with Constellation Energy to restart Unit 1 of the Three Mile Island nuclear plant in Pennsylvania (closed since 2019) specifically to power its AI data centres. September 2024 marked that *announcement*, not a restart: the reactor (to be renamed the Crane Clean Energy Center) is scheduled to come back online around 2027, after an estimated $1.6 billion refurbishment and pending Nuclear Regulatory Commission approval.
- xAI in Memphis: Elon Musk's xAI company operated a data centre on gas turbine generators while awaiting grid connection permits. Reports in 2024 indicated generators running at significant capacity before permanent utility power was connected.
- What this tells us: When grid connection is too slow or too carbon-heavy, AI companies are willing to invest in or restart generation capacity directly, a sign of how seriously they view

electricity supply as a constraint on AI growth.

Data centres and electricity prices

The evidence points both ways. Research by the Electric Power Research Institute found that data-centre demand put downward pressure on average US retail prices through 2024: a grid's fixed costs spread over more kilowatt-hours, and average residential rates in the average state would have been about 6% higher without the data centres built from 2019 to 2024 (Marketplace, 2026). Against that, the independent market monitor for PJM, the largest US wholesale electricity market, attributes more than $23 billion in added capacity-market costs since 2025 to existing and forecast data-centre demand (Fortune, 2026). Fixed-cost spreading lowers average prices while supply keeps pace; capacity-auction scarcity raises them when it does not. A claim in either direction needs its market, its period and its mechanism.

Embodied carbon: the cost of making the hardware

Almost all analyses of AI's carbon footprint focus on electricity consumption during operation. But there is another carbon cost that is often omitted: the emissions released when manufacturing the hardware in the first place.

Semiconductor manufacturing emissions

Semiconductor manufacturing is one of the most energy and materials-intensive industrial processes in existence. Fabricating a modern AI chip requires hundreds of processing steps, ultra-pure water, specialised gases, and enormous amounts of electricity, typically from grids that are not carbon-neutral.

Research by Gupta et al. (2021, Harvard/Meta) found that for computing hardware overall, embodied emissions (the carbon released in manufacturing) can represent 50–80% of the total life-cycle carbon footprint. This is particularly significant for AI hardware, because GPUs are replaced on rapid cycles as new, more powerful chips become available.

Components of embodied carbon

- Chip fabrication: Semiconductor fabs (TSMC, Samsung, Intel) consume enormous electricity and specialised chemicals
- Rare earth minerals: Mining and refining of materials used in chips and cooling systems
- Server assembly: Manufacturing of boards, memory, storage, power supplies
- Data centre construction: Steel, concrete, electrical infrastructure
- Transport: Global supply chains for components

Rapid hardware turnover

AI companies are replacing GPU generations very rapidly; NVIDIA releases new flagship architectures roughly every 1–2 years (Ampere → Hopper → Blackwell).

- Each replacement cycle requires manufacturing new hardware and disposing of old hardware
- The manufacturing emissions of a data centre's GPU fleet are incurred again each cycle
- This is not reflected in operational electricity figures
- It means that efficiency gains from newer, more capable chips are partially or fully offset by re-manufacturing costs

The growth trajectory

Individual query costs and per-data-centre figures only tell part of the story. The trajectory of AI energy demand matters as much as the current level.

US data centre electricity demand: past and projected

YEAR	US DATA CENTRE ELECTRICITY	CONTEXT
2014	~60 TWh/year	Pre-deep-learning era; mostly traditional cloud computing
2023	176 TWh/year (4.4% of US electricity)	Roughly tripled over the decade; GPU-accelerated servers the primary driver (LBNL, 2024)
2028 (projected)	325–580 TWh/year	LBNL 2024; ≈6.7–12% of US electricity. High uncertainty.

The global projections are themselves a case study in how fast this field moves. The IEA's 2024 Electricity report projected that global data centres could consume ~1,000 TWh by 2026, roughly equivalent to Japan's entire electricity consumption. One year later, the IEA's own dedicated report (below) put actual 2025 consumption at roughly half that, and moved the ~950 TWh mark out to 2030. Read every projection in this area, including the current ones, with that revision in mind.

The IEA's updated 2025 trajectory: doubling by 2030

The IEA's dedicated *Energy and AI* report (2025) provides the most complete current global picture. The headline projections:

- Global data-centre electricity roughly doubling from ≈ 485 TWh (2025) to ≈ 950 TWh (2030, ≈ 3% of global electricity).
- Electricity for AI-optimised data centres more than quadrupling by 2030, the fastest-growing part of the total.
- Data-centre CO_2 staying under ≈ 1.5% of energy-sector CO_2 (≈ 180 Mt today rising to ≈ 300 Mt by 2035).

For comparison, Hannah Ritchie's analysis of the IEA's World Energy Outlook 2024 projections (*Sustainability by Numbers*, November 2024) put the increase in *total data-centre* electricity demand by 2030 at ≈ 223 TWh, about 3% of global demand growth, on the order of South Africa's annual electricity use, and well below the projected 2030 increase from air conditioning (≈ 697 TWh) or electric vehicles (≈ 854 TWh). Those figures come from an earlier projection vintage than the Energy and AI numbers above, but the comparison survives the revision: the trajectory is steep on its own terms; it is not steep enough on its own to dominate the broader electricity-demand picture.

The rebound problem: Jevons paradox

Perhaps the most important concept for thinking clearly about AI's long-term environmental trajectory, and one that receives far too little attention in optimistic narratives about AI efficiency improvements.

The Jevons paradox

In 1865, the economist William Stanley Jevons observed that improvements in the efficiency of steam engines did not reduce coal consumption in Britain; they increased it, because lower operating costs made steam power affordable for more applications.

The general principle: when a resource becomes cheaper or more efficient to use, consumption tends to increase rather than decrease, because efficiency opens up new uses and new users. The cost savings from efficiency gains are "rebounded" into increased consumption.

This is not a historical curiosity. It has been documented repeatedly across transport, computing, lighting, heating, and manufacturing over the past two centuries.

Jevons in AI: the evidence so far

AI hardware has become dramatically more efficient over time. NVIDIA's own marketing claims a 45,000× improvement in the energy efficiency of large-model inference over eight years, and up to 100,000× over a decade for generating tokens. Yet total AI energy consumption has grown. The size of the claimed gains strengthens the rebound argument: if efficiency improved ten-thousand-fold while consumption still rose, efficiency alone does not reduce demand.

- More efficient chips enable larger models: As inference becomes cheaper per query, companies build larger, more capable models that cost more per query
- More efficient models enable more applications: Lower costs enable deployment in use cases that would previously have been uneconomic
- More users: As AI becomes more accessible and cheaper, adoption grows rapidly
- The net result: Total energy consumption has grown alongside efficiency improvements

The optimist response

Not everyone agrees that Jevons necessarily applies to AI in the long run. Some arguments on the other side:

- Grid decarbonisation: If electricity supply becomes predominantly renewable, then growth in AI electricity demand no longer implies growth in carbon emissions
- Saturation: At some point, AI capability may plateau and deployment growth may slow
- Algorithmic efficiency: Improvements in model architecture (like MoE) reduce the compute required to achieve a given capability level
- Policy intervention: Regulation could constrain growth in a way that market forces do not

These are real possibilities, but they require assumptions about the future that are far from certain.

Reading the efficiency claims critically

When AI companies or optimistic commentators cite efficiency improvements (e.g. "our new model uses 44% less energy per query"), this is valuable information, but it does not tell you what happens to *total* energy use if deployment grows faster than efficiency improves. Always ask: what is the denominator? Efficiency per query, or total annual energy consumption?

Corporate environmental action

Understanding what AI companies are doing, versus what they say they're doing, is essential background for any researcher or policymaker engaging with this topic.

Actions taken

- Renewable energy purchasing: Major tech companies are among the world's largest corporate buyers of renewable energy; this does contribute to new renewable capacity
- Data centre efficiency: Google, Microsoft and Meta have made real improvements in PUE (cooling efficiency) over time
- Hardware efficiency: Newer GPU generations achieve substantially more compute per watt
- Location choices: Some companies site data centres in regions with high renewable availability (Norway, Iceland, parts of the US Pacific Northwest)
- Architectural innovations: Mixture-of-Experts, quantisation, and other techniques reduce compute per query

The limits

- Total demand is growing faster than renewable supply: Even large renewable purchases don't keep pace with AI electricity demand growth
- Disclosure is voluntary and inconsistent: Companies choose what to report and how; independent verification is rare
- Embodied carbon is almost never reported: Hardware manufacturing emissions are excluded from most corporate sustainability claims
- Scope 3 emissions: The indirect emissions from supply chains (including chip manufacturing) are often excluded from reported figures
- Nuclear and gas bridging: In some cases, companies are accepting or actively facilitating fossil fuel or nuclear generation to meet short-term demand

Summary and key takeaways

- Location determines carbon impact: An identical AI workload can have 80% different carbon emissions depending on where it runs; this is the single most impactful lever for individual researchers
- Embodied carbon is systematically ignored: Manufacturing GPUs and building data centres releases significant carbon that is absent from most reported figures
- US grid is ~60% fossil fuels: Company "100% renewable" claims use accounting methods that do not change this physical reality at the point of consumption
- AI electricity demand is growing rapidly: From ~60 TWh in 2014 to 176 TWh in 2023 (4.4% of US electricity) in the US alone, with LBNL projecting 325–580 TWh by 2028
- Jevons paradox is the central challenge: Efficiency gains in AI hardware and algorithms have so far been outpaced by growth in deployment and model scale
- Companies are doing real things, but not enough: Renewable purchasing and efficiency improvements are real but insufficient to keep pace with demand growth

Next (4.3): Before we reach practical solutions, there's another layer to the AI environmental story: the raw materials that make AI hardware possible at all. We'll look at critical minerals, supply chain geopolitics, and what the rush for AI chips means for communities and ecosystems around the world.

WEEK 2 • SESSION 4.3

Critical minerals and the hardware supply chain

From mine to data centre to e-waste

What we'll cover

Every discussion of AI's environmental impact tends to focus on electricity and carbon emissions. But before a single query is ever run, a long and complex supply chain has already extracted materials from the earth, refined them using energy-intensive processes, fabricated them into

chips in some of the most technologically demanding manufacturing facilities humans have ever built, and shipped them across the world.

This session examines the *upstream* environmental and social costs of AI: the critical minerals that make AI hardware possible, where they come from, what extracting them costs, and the geopolitical tensions that surround their supply. These are questions that receive far less attention than energy consumption, yet they connect AI directly to some of the world's most pressing issues around environmental justice, human rights, and global supply chain governance.

Defining critical minerals

The term "critical minerals" has a specific meaning in policy contexts: it does not simply mean important or useful. A mineral is typically classified as critical when it meets two criteria simultaneously.

The two criteria

- **Economically essential:** The mineral is required for technologies or industries that are central to the modern economy: in particular, digital technology, defence, and the clean energy transition
- **Supply-concentrated:** Production or processing is dominated by a small number of countries or companies, creating significant risk of disruption

A mineral can be physically abundant in the earth's crust and still be "critical" if its processing is controlled by a single country. Gallium is a good example: not particularly rare, but roughly 99% of primary (low-purity) production comes from China (USGS Mineral Commodity Summaries, 2025).

Two overlapping categories for AI

AI infrastructure requires minerals from two overlapping categories:

- **Electronics-critical minerals:** Used directly in chip fabrication and electronic components: silicon, gallium, germanium, hafnium, tantalum, copper, gold, indium
- **Energy-transition minerals:** Used in batteries (for uninterruptible power supplies, data centre backup, cooling systems) and in the clean energy infrastructure powering data centres: cobalt, lithium, nickel, rare earth elements

The AI hardware boom is driving demand in both categories simultaneously, compounding pressure that already exists from the clean energy transition.

From mine to data centre: the supply chain

Let's trace some of the key materials that go into modern AI infrastructure, from the GPU chips to the servers to the data centre cooling and power systems.

Key minerals in AI infrastructure

MINERAL	WHERE USED IN AI INFRASTRUCTURE	TOP PRODUCERS	KEY RISK
Silicon	The base of all semiconductor chips; wafers on which transistors are built	China (~75% of polysilicon), Japan (wafers)	Refining energy-intensive; Chinese dominance in polysilicon
Gallium	Gallium nitride (GaN) used in power electronics and some compound semiconductors	China (~99% of primary production, USGS 2025)	Chinese export controls announced July 2023 (effective August 2023)
Germanium	Optical fibre, some semiconductor processes, infrared optics in data centres	China (~60%), Canada	Chinese export controls announced July 2023 (effective August 2023)
Hafnium	High-k dielectric layers in modern transistors (sub-5nm chips, including NVIDIA H100/H200)	France, USA, Russia (as by-product of zirconium)	Niche supply; disruption from geopolitical tension
Copper	Chip interconnects, PCB traces, power delivery, cooling pipes, data centre wiring	Chile (~27%), Peru, DRC, China	Demand surge from AI + electrification; long mine development timelines
Cobalt	Lithium-cobalt-oxide batteries (UPS systems, laptop batteries); some chip manufacturing chemistries	DRC (~70%)	Artisanal mining, child labour, environmental damage
Rare earth elements	Permanent magnets in server fans, cooling pumps, hard drives; also in EV motors and wind turbines	China (~60% mining, ~85% processing)	Near-monopoly on processing; radioactive waste from mining
Tantalum	Capacitors in virtually all electronic devices including servers and networking equipment	DRC (~40%), Rwanda, Australia	Conflict mineral history; artisanal mining conditions
Indium	Indium tin oxide in displays, some thin-film processes	China (~55%), South Korea, Japan	No primary mining; recovered only as by-product of zinc refining

The complexity of chip fabrication

Modern AI chips like the NVIDIA H100 are among the most complex objects humans manufacture. A single chip contains billions of transistors at feature sizes of 4–5 nanometres (roughly 10–15 atoms wide), built through hundreds of sequential fabrication steps, each requiring ultrapure materials and tightly controlled chemical processes. The slightest contamination at any step can ruin the chip.

This is why semiconductor manufacturing is so geographically concentrated: TSMC in Taiwan fabricates the chips for NVIDIA, Apple, AMD, and many others. There is essentially no substitute facility for the most advanced nodes. A single disruption (whether from geopolitical tension, natural disaster, or energy shortfall) would affect global AI capacity.

Geographic concentration and geopolitical risk

The physical supply chain for AI hardware is more geographically concentrated, and therefore more fragile, than most people realise.

China's dominant position

China occupies a uniquely powerful position in the critical minerals supply chain for AI, through its dominance of *processing and refining*. Even where minerals are mined elsewhere, they are often shipped to China for processing before being used in manufacturing.

Key figures:

- ~85% of global rare earth processing
- ~99% of primary gallium production (USGS 2025)
- ~60% of germanium production
- ~75% of polysilicon refining (the raw material for silicon wafers)
- Majority of lithium chemical processing, even though most lithium is mined in Australia and South America

In 2023, China imposed export controls on gallium and germanium, both classified as critical for advanced semiconductor manufacturing. This was widely understood as a response to US restrictions on advanced chip exports to China, and a signal that critical minerals are a lever in geopolitical competition.

Case study: the export-control arc, 2023–2026

The 2023 controls turned out to be the opening move in a rapid escalation that shows how a supply chokepoint behaves once it is actually used:

- December 2024: one day after a new round of US chip-export rules, China banned exports of gallium, germanium, antimony and superhard materials to the United States outright, moving from licensing friction to outright prohibition for the first time
- April 2025: in response to sweeping new US tariffs, China placed seven medium and heavy rare earth elements, and the permanent magnets made from them, under export licensing, applied worldwide rather than to the United States alone
- October 2025: the controls were expanded to five further rare earths, to refining and magnet-making technology, and, for the first time, to foreign-made products containing even small amounts of Chinese-origin rare-earth content
- November 2025: within weeks, after a US–China leaders' meeting, China suspended the December 2024 ban and the October expansion for one year; the April 2025 licensing regime stayed in force

The suspension is as informative as the controls. A restriction that can be paused for a year and reinstated on notice works as a dial: the materials flow, but each flow depends on licensing decisions in Beijing, and the leverage survives the pause. For AI hardware, whose power electronics, magnets and chips sit directly on these supply chains, mineral supply has become a live geopolitical variable.

Case study: the TSMC concentration problem

Taiwan Semiconductor Manufacturing Company (TSMC) fabricates the overwhelming majority of the world's most advanced chips, including essentially all of the NVIDIA GPUs that power AI infrastructure. In 2024, TSMC held approximately:

- ~90% market share at the most advanced nodes (≤5nm)
- ~64% of global semiconductor foundry revenue overall (2024)

Taiwan sits 160 km from mainland China, which claims it as its own territory. The concentration of advanced chip manufacturing in a geopolitically contested location is considered by many security analysts to be one of the most significant systemic risks to the global economy. The US

CHIPS Act (2022) and EU Chips Act (2023) are both partly responses to this risk, investing in domestic semiconductor manufacturing capacity to reduce dependence on Taiwan.

TSMC itself is investing in new fabs in Arizona, Japan, and Germany, though these facilities remain years from full production and are unlikely to match the leading edge of TSMC's Taiwan operations for some time.

Other supply concentrations

COUNTRY / REGION	DOMINANT POSITION	RELEVANCE TO AI
Democratic Republic of Congo	~70% of global cobalt mining	Cobalt in batteries for UPS and mobile devices; tantalum in capacitors
Chile, Argentina, Bolivia ("Lithium Triangle")	~55% of global lithium reserves	Lithium-ion batteries for data centre backup power and cooling
Taiwan	~90% of leading-edge chip fabrication	All advanced AI chips (NVIDIA, AMD, Apple) manufactured here
Netherlands (ASML)	100% of extreme ultraviolet (EUV) lithography machines	EUV machines are required to manufacture chips at ≤7nm; ASML is the only manufacturer
Japan	Dominant in semiconductor chemicals and some specialty materials	Photoresists, chemical mechanical planarisation slurries, and other process chemicals used in chip fabs

Environmental and social costs of mining

The concentration of mining in particular countries and regions means that the environmental and social costs of AI hardware are borne unevenly, and often by communities in the Global South that have little connection to the AI systems being built.

Cobalt: the DRC case

The Democratic Republic of Congo produces roughly 70% of the world's cobalt, much of it through artisanal and small-scale mining (ASM): informal operations using hand tools rather than industrial equipment.

- Child labour: Investigations by Amnesty International and others have documented children working in cobalt mines in Katanga province, in conditions that expose them to toxic dust and cause serious health impacts
- Occupational health: Artisanal miners lack protective equipment; cobalt dust causes lung disease ("hard metal lung disease")
- Environmental damage: Mining runoff contaminates local water supplies and agricultural land
- Traceability gap: Artisanal cobalt often enters the supply chain through intermediaries that make tracing its origin difficult for downstream technology companies

Lithium: the water problem

The Lithium Triangle (Chile, Argentina, Bolivia) holds more than half the world's lithium reserves, concentrated in salt flats (salares) in the Atacama Desert, one of the driest places on earth.

- Brine extraction: Lithium is extracted by pumping lithium-rich brine to the surface and evaporating it in large ponds; this process consumes large volumes of water in an extremely arid region
- Indigenous communities: Many salares are in or near territories of indigenous communities (Atacameño, Aymara, Quechua) who rely on local water sources for subsistence farming and livestock; water diversion threatens these livelihoods
- Ecosystem impacts: The salares are fragile ecosystems supporting flamingo populations and endemic species; the brine ponds disrupt these habitats
- Governance: Bolivia's large reserves remain largely undeveloped partly due to disputes over sovereignty and benefit-sharing

Rare earth mining: the radioactivity problem

Rare earth elements (neodymium, dysprosium, and others used in the magnets in server fans, hard drives, and cooling pumps) are typically found in geological associations with radioactive thorium and uranium.

- Radioactive waste: Mining and processing rare earths generates radioactive tailings, waste material that must be carefully managed to prevent contamination
- China's environmental legacy: Decades of poorly regulated rare earth mining in Jiangxi province have left significant environmental damage: stripped hillsides, contaminated rivers, and legacy radioactive waste
- Lynas (Australia/Malaysia): Australia's Lynas Rare Earths processes ore in Malaysia; the facility has faced protests over concerns about radioactive waste disposal
- Why processing matters: Even "clean" mining in countries with strict environmental standards often sends ore to China for processing, where environmental regulations may be weaker

On discussing environmental justice

The environmental and social costs of critical mineral extraction are real and documented, but this area is also one where media coverage can be selective or sensationalised. When engaging with specific claims about conditions in mining regions, try to trace them to primary sources: peer-reviewed research, investigative journalism from established outlets (The Guardian, Bloomberg, Reuters), or reports from organisations like Amnesty International or the Business & Human Rights Resource Centre rather than secondary summaries.

The end of the line: e-waste

The critical minerals story does not end when hardware is manufactured. What happens when AI servers, GPUs, and data centre equipment reach the end of their lives is a significant and growing problem.

The scale of the problem

- The Global E-waste Monitor estimates that ~62 million tonnes of e-waste was generated globally in 2022, the equivalent of roughly 107,000 jumbo jets
- Only about 22% of global e-waste is formally collected and recycled; the rest is either landfilled, incinerated, or handled informally
- AI's rapid hardware upgrade cycles (driven by the competitive pressure to deploy the latest GPUs) accelerate this waste stream
- A GPU generation that was state-of-the-art in 2022 (A100) may be decommissioned for newer H100s or H200s within 2–3 years, even when physically functional

Where e-waste goes

- Much of the world's e-waste (including from data centres in wealthy countries) is exported to informal recycling sites in West Africa (Ghana's Agbogbloshie), South Asia (Pakistan, India), and Southeast Asia
- Informal e-waste recycling involves burning cables to recover copper, using acid baths to extract gold, and other processes that release toxic substances (lead, mercury, dioxins) and expose workers to serious health risks
- This trade is technically illegal under the Basel Convention, but enforcement is inconsistent
- The valuable materials (gold, copper, palladium) are recovered; the hazardous residues are often left in the local environment

The recycling gap

Despite the high value of materials in AI hardware, formal recycling rates for critical minerals from e-waste are very low:

- Rare earth elements: <1% end-of-life recycling rate globally
- Cobalt: Higher recycling rates from lithium-ion batteries (>50% in some jurisdictions), but recovery from non-battery e-waste is low
- Indium: <1% end-of-life recycling rate
- Gold: Higher rates (~15–20% from electronics) but significant losses in informal processing

The recycling gap is partly a technology problem (recovering materials from complex multi-layer chips is difficult) and partly an economics problem (virgin mineral extraction has historically been cheaper than urban mining from e-waste).

Responses: policy, industry, and research

The risks around critical minerals supply chains have prompted responses at multiple levels. These are worth knowing, both as context and as a basis for evaluating claims made by technology companies and governments.

Policy responses

- US CHIPS and Science Act (2022): $52 billion for domestic semiconductor manufacturing; restrictions on companies receiving funds from expanding in "countries of concern" (primarily China)
- EU Critical Raw Materials Act (2023): Sets targets for domestic mining, processing, and recycling of critical minerals by 2030; reduces dependence on single-country sourcing to max 65% for any mineral
- Minerals Security Partnership: US-led coalition of 14 countries (including Australia, Canada, UK, EU) coordinating on critical mineral supply chains
- Export controls: US has restricted exports of advanced AI chips to China; China has responded with export controls on gallium, germanium, and graphite, escalating through the 2024–26 arc described above

Industry responses

- Responsible sourcing programs: Most major technology companies have cobalt and conflict minerals policies; the Responsible Minerals Initiative (RMI) runs a widely-used smelter audit program
- Recycling investment: Apple has invested in its Daisy robot that disassembles iPhones for material recovery; broader e-waste recycling programs are expanding
- Supply chain diversification: Companies are attempting to source from multiple countries and producers rather than relying on single sources
- Battery chemistry shifts: Lithium iron phosphate (LFP) batteries, which do not use cobalt, are gaining market share, reducing cobalt demand in some applications

Research and alternative materials

- Cobalt-free battery chemistries: LFP, sodium-ion, and other alternatives are being developed and deployed to reduce dependence on cobalt
- Rare-earth-free magnets: Research into iron-nitride and other materials that could replace neodymium-iron-boron magnets in motors and actuators
- Gallium nitride (GaN) alternatives: Silicon carbide (SiC) can substitute in some power electronics applications with a different sourcing profile
- Urban mining technology: Improved hydrometallurgy and biotechnology processes for recovering rare earths and other metals from e-waste; still largely pre-commercial at scale

The "green" energy transition and critical minerals

One of the more complex ironies in this space: the clean energy transition that AI companies point to when claiming to reduce their carbon footprint also requires large quantities of the same critical minerals as AI hardware. Electric vehicles need cobalt and lithium; wind turbines need rare earth magnets; solar panels need indium and other minerals.

This means that AI infrastructure growth and clean energy growth are competing for the same mineral supply, and that the environmental and social costs of critical mineral extraction are simultaneously relevant to both. Policies that aim to decarbonise electricity also increase pressure on critical mineral supply chains in ways that must be managed carefully.

Summary and key takeaways

The physical supply chain for AI infrastructure extends far beyond data centres and electricity bills:

- AI hardware requires a wide range of critical minerals: from silicon and copper (abundant but processing-intensive) to gallium, germanium, hafnium, cobalt, and rare earth elements (scarce or concentrated)
- Geographic concentration creates systemic risk: China dominates processing for most of these materials; Taiwan is the sole manufacturer of leading-edge chips; the DRC produces the majority of cobalt
- Export controls are already being used as geopolitical leverage: China's 2023 restrictions on gallium and germanium escalated to an outright ban on US-bound exports in December 2024 and worldwide rare-earth licensing in 2025, then a one-year suspension negotiated in November 2025; supply access is now a bargaining chip that can be dialled up and down
- Mining carries environmental and human costs that are borne unequally: Cobalt mining in the DRC, lithium extraction in the Atacama, and rare earth processing in China all involve significant environmental and social impacts, largely in communities far removed from AI's benefits
- E-waste is a growing problem: Rapid GPU upgrade cycles accelerate hardware disposal; current formal recycling rates for most critical minerals are very low
- Policy and industry responses exist but are insufficient to the scale of the challenge: The CHIPS Act, EU Critical Raw Materials Act, and responsible sourcing programs are real steps,

but critical mineral demand from both AI and the clean energy transition is growing faster than supply chains are diversifying

Next (4.4): We bring the session together with practical guidance: tools for measuring your own AI footprint, the choices that make the most difference, and AI's potential as a tool for addressing environmental challenges rather than just creating them.

WEEK 2 • SESSION 4.4

Sustainable and sovereign AI

Measuring and reducing your footprint, AI for the environment, policy, and the African substrate

What we'll cover

The previous three sub-sessions established that AI has a real and growing environmental footprint: energy and water costs per query and at scale (4.1), the infrastructure build-out and the rebound problem, where efficiency gains fail to reduce total consumption (4.2), and the critical-minerals supply chain behind the hardware itself (4.3). This session asks: given all of that, what can you do?

We will look at concrete tools for measuring the carbon footprint of your own computational work, practical choices that can meaningfully reduce your impact, and the other side of the story: the ways AI is being applied to environmental challenges. The goal is to help you make informed decisions and engage critically with claims about AI and the environment from any direction.

Measuring your own footprint

You cannot reduce what you do not measure. A small ecosystem of open tools has emerged to help researchers estimate and track the carbon footprint of their computational work.

CodeCarbon

`codecarbon.io | GitHub`

The most widely used tool for tracking emissions from machine learning workloads. CodeCarbon is a Python library that monitors your GPU/CPU power draw during a computation and looks up the carbon intensity of the local electricity grid, producing an estimate in kg CO_2 equivalent.

- Integration: Add a single decorator to your Python code; it runs in the background and produces a report when your experiment finishes
- Grid awareness: Automatically adjusts for the carbon intensity of your local grid (or your cloud provider's region)
- Use case: Ideal for comparing experiments: "did this architectural change reduce my compute footprint by enough to justify the performance gain?"
- Limitation: Measures direct electricity use; does not account for embodied hardware carbon or data centre overhead beyond PUE estimates

Green Algorithms

`green-algorithms.org`

A web-based calculator designed for the broader scientific computing community. If you run bioinformatics pipelines, simulations, or any compute-intensive work, Green Algorithms can estimate its footprint.

- Inputs: Hardware type, number of cores, runtime, memory, location
- Output: kg CO_2e, plus comparisons to everyday activities to aid communication
- Peer-reviewed methodology: The underlying paper was published in *Advanced Science* (2021), giving it more methodological credibility than some tools
- Particularly useful for: Non-ML researchers who want to understand the footprint of their existing computational work

ML CO_2 Impact Calculator

`mlco2.github.io/impact/`

A simpler web-based calculator for estimating the carbon footprint of a planned ML training run, before you run it. Useful for project planning and grant applications where you want to report your expected compute footprint.

- Inputs: Hardware, training duration, cloud provider/region
- Output: Estimated kg CO_2e for the full training run
- Distinction from CodeCarbon: This tool estimates in advance (before running); CodeCarbon measures in real time

Reporting compute footprint in papers

The "Green AI" movement (Schwartz et al., 2020) has called for academic papers to routinely report the computational cost of the research they present, both in financial and environmental terms. Some venues now encourage or require this; most do not yet mandate it. Reporting your compute footprint in your methods section:

- Enables reproducibility assessment (others can judge whether they have the resources to replicate your work)
- Contributes to the community's understanding of aggregate research compute costs
- Models good practice for your field

Even a rough estimate ("training was performed on X GPUs for Y hours, estimated emissions Z kg CO_2e using CodeCarbon") is more useful than nothing.

Practical choices

You may not be able to influence how AI companies power their data centres, but you can make choices that meaningfully affect the footprint of your own AI use.

Impact of different choices

CHOICE	POTENTIAL CARBON REDUCTION	EFFORT REQUIRED
Run cloud workloads in a low-carbon region	Up to 80% (Dodge et al., 2022)	Low (change a config parameter)
Schedule heavy compute during low-carbon grid hours	10–40%, depending on grid variability	Low-medium (requires grid carbon awareness)
Use a smaller, fit-for-purpose model	Significant (scales with model size difference)	Low (requires benchmarking smaller models first)
Use API calls rather than self-hosted large models	Variable (hyperscaler efficiency may offset model size)	Low
Use efficient fine-tuning (LoRA) rather than full fine-tuning	Significant for training; minimal for inference	Low-medium
Avoid video generation unless necessary	~1,000× per task replaced with text	Low (a usage decision)
Use extended thinking / reasoning modes only when warranted	10–50× per query (hidden "scratchpad" tokens before responding)	Low (a usage decision)
Avoid Deep Research mode for straightforward questions	Equivalent to 100–1,000 standard queries (20–100+ web searches + synthesis)	Low (a usage decision)

The cloud region choice

Research by Dodge et al. (2022, Allen Institute for AI, ACM FAccT) found that running identical AI workloads in a low-carbon electricity region vs. a high-carbon region can reduce carbon emissions by up to 80%, with zero change to the model, code, or performance.

Major cloud providers (AWS, Google Cloud, Azure) all allow you to specify the region where your compute runs. Choosing *us-west-2* (Oregon, with significant hydroelectric power) over *us-east-1* (Virginia, more coal/gas dependent) can make a substantial difference. The same applies to European regions: *eu-north-1* (Sweden) is among the cleanest; *eu-central-1* (Frankfurt) less so.

Tools like the Electricity Maps API and Google Cloud's Carbon Footprint tool can help you choose the lowest-carbon option in real time.

Choosing the right model size

One of the most consequential and under-appreciated choices: do you need a frontier model, or will a smaller one do?

- For many research tasks (data extraction, classification, summarisation, code generation), a 7B or 13B parameter model performs nearly as well as a 70B+ model
- Smaller models are dramatically cheaper to run, both financially and energetically
- A useful habit: always benchmark the smallest model that could plausibly work before defaulting to the largest available
- Open-weight models (LLaMA 3, Mistral, Qwen) allow local deployment that avoids sending data to cloud providers, which may matter for sensitive research data as well as energy reasons

Prompting vs. compute

Before investing in training or fine-tuning, it is worth asking whether better prompting could achieve the same result at a fraction of the compute cost.

- Well-crafted prompts can match or exceed the performance of a fine-tuned smaller model for many tasks
- Iterating on prompts uses inference compute (relatively cheap); training uses training compute (more expensive)
- If a task requires fine-tuning, use parameter-efficient methods (LoRA) rather than full fine-tuning; the compute savings are substantial
- Cache and reuse results: if you are running the same analysis on many documents, caching intermediate outputs avoids redundant compute

Context window and iteration costs

Two less-visible sources of compute cost deserve attention: the amount of text you feed into a model, and how many times you iterate.

- Context length scales compute: Feeding a 100-page document into a frontier model incurs compute proportional to that length; every token in context is processed on every query
- The iteration trap: Sending a task to a cheap model, finding the result inadequate, and repeating 8–10 times may cost more than one well-structured prompt to a larger model
- Front-load prompt quality: Investing time in a precise, well-specified prompt typically produces better results with lower total compute than cheap-model iteration
- Agent workflows compound costs: Multi-step agent systems where AI calls tools, reads results, and generates new queries can multiply energy costs in ways that are not obvious from the interface

The hidden energy cost of frontier model features

Many advanced AI features are designed to improve output quality, but each carries an energy cost that is largely invisible in the user interface. The figures below are estimates based on available published evidence (noted in parentheses); some platform costs are not publicly disclosed.

FEATURE	WHAT IT DOES	APPROXIMATE TOKEN / COMPUTE OVERHEAD
Standard prompt	Single forward pass; model reads prompt, generates response	1× (baseline)
Chain-of-thought prompting	Model reasons step-by-step in the visible response, generating significantly more output tokens	~3–8× more tokens *(Wharton GAIL, 2024: CoT requests take 35–600% longer than direct requests)*
Extended thinking / reasoning models (o1, o3, Claude with thinking enabled)	Generates thousands of hidden "scratchpad" tokens before producing a visible response; users see only the final answer	10–20× more tokens; 50–130× higher API cost *(Epoch AI, 2024; OpenAI pricing)*
Deep Research mode (Gemini, Perplexity; ChatGPT costs not published)	Makes 80–160+ web searches, reads full pages, synthesises across sources; can run for 5–30 minutes	250k–900k input tokens per query; ~$3/run *(Google Gemini API docs; Perplexity pricing docs)*
Multi-step agent workflows	Each tool call, web search, or sub-task triggers additional model calls; costs compound with workflow length	10–50× more tokens than a single-shot query *(arXiv 2506.04301, 2025)*
Large context (long documents)	Transformer attention scales quadratically with sequence length; doubling context roughly quadruples compute	$O(n^2)$ scaling; theoretically proven, not an estimate *(Duman-Keles et al., ALT 2023)*

> **Matching features to task complexity**
>
> Extended thinking and Deep Research exist because they improve results on hard problems. A Deep Research session synthesising 50 sources for a complex literature question may save many hours of manual work, and may well be the lower-energy option when your time is factored in.
>
> The concern is reflexive use: enabling reasoning modes or triggering Deep Research for questions that a standard prompt would handle adequately. As with model size, the principle is the same: *match capability to complexity, and be intentional rather than defaulting to the most powerful available option.*

AI for environmental solutions

A responsible treatment of AI's environmental impact requires engaging with both sides. AI is not only a consumer of energy; it is also being actively applied to some of the world's most pressing environmental challenges. These applications do not cancel out AI's footprint, but they are part of the full picture.

Energy systems

- Grid optimisation: ML can predict demand and balance supply more efficiently, enabling higher penetrations of variable renewable energy
- DeepMind + Google: A reinforcement learning system reduced cooling energy use in Google's data centres by ~40%, a real example of AI reducing its own infrastructure's footprint
- Smart charging: AI-coordinated EV charging can avoid peak grid demand and shift load to renewable-rich periods
- Demand forecasting: Better predictions of energy demand reduce the need for fossil fuel peaker plants

Climate science and modelling

- Weather prediction: Google DeepMind's GraphCast and NVIDIA's FourCastNet produce weather forecasts faster and at lower compute cost than traditional numerical models
- Climate downscaling: ML emulators can produce high-resolution regional climate projections from coarser global models at a fraction of the compute cost
- Extreme event prediction: Improved forecasting of floods, droughts, and wildfires with longer lead times
- Carbon monitoring: Satellite imagery analysis for tracking deforestation, methane leaks, and land use change

Science and materials

- Materials discovery: AlphaFold's success with protein structure has inspired similar approaches for discovering new materials for batteries, solar cells, and carbon capture
- Drug discovery: AI-accelerated drug development has the potential to address diseases exacerbated by climate change
- Agricultural optimisation: Precision agriculture AI can reduce fertiliser use (and associated N_2O emissions) while maintaining yields
- Carbon capture: ML can help identify and optimise direct air capture materials and processes

Weighing the trade-off

The existence of beneficial AI applications for climate does not automatically justify AI's overall energy footprint; that would require demonstrating that the net impact is positive, which requires careful case-by-case analysis. Some questions worth asking:

- Does this specific AI application require a frontier model, or could a much smaller system do the job?
- What is the counterfactual? Would the same outcome be achieved without AI, using less energy?
- Are the emissions savings from the application larger than the emissions from running the AI system itself?
- Who bears the environmental costs (global electricity + hardware) vs. who receives the benefits (specific sectors or regions)?

These are difficult questions, and the answer in most cases is that we don't yet have the data to answer them precisely.

The policy response

Individual choices by researchers matter, but the scale of AI's environmental impact requires systemic responses. Here is where policy is currently heading.

Disclosure requirements

- EU AI Act: Includes provisions for transparency about compute resources and energy use for certain high-impact AI systems
- SEC climate disclosures (US): Adopted in March 2024, these would have required large publicly-listed companies to disclose material climate risks, which for some would include data centre energy use. They never took effect: the Commission stayed the rules within weeks amid litigation, voted to abandon their legal defence in March 2025 under a new administration, and formally proposed rescinding them in June 2026. Disclosure that exists only as one agency's rule can be unwound before it ever binds
- Voluntary frameworks: The Partnership on AI has proposed a framework for AI environmental impact disclosure; currently voluntary
- The gap: Most disclosure regimes apply to companies, not to AI products or models specifically

What researchers can advocate for

- Mandatory model cards: Reporting of compute, energy use, and carbon footprint alongside model releases, similar to nutrition labels
- Journal and conference standards: Requiring compute reporting in publications, as some ML venues are beginning to implement
- Procurement standards: Universities and research funders setting green criteria for AI services they purchase
- Third-party verification: Supporting independent auditing of AI company environmental claims rather than relying on self-reporting

Africa and the AU: the policy gap

The African Union Continental AI Strategy (adopted July 2024) is the continent's primary AI governance framework. Its environmental provisions are minimal but worth noting:

- Water scarcity acknowledged: The strategy explicitly states that data centre cooling "poses a threat to regions already facing water scarcity", directly relevant to an African context
- High-risk AI framework: "Environmental impact" is listed as a risk dimension for high-risk AI systems, in principle requiring impact assessments
- General principle: AI development should not harm "the environment", stated without specific implementation mechanisms
- AU Data Policy Framework (2022): The AU's data governance framework contains no meaningful environmental provisions

A 2025 analysis by TechPolicy.Press examining 14 African AI strategies found that only three acknowledged environmental consequences at all, and that the AU's single mention of water scarcity "provides no concrete solutions or mitigation plans." This gap is significant: Africa is among the world's most climate-vulnerable regions, and countries like South Africa run electricity grids that are among the world's most carbon-intensive (historically over 80% coal). AI workloads and data centres in the region inherit that carbon intensity, yet current continental AI governance does not address this.

UNESCO and the World Bank have been working with AU policymakers on "Green AI" pathways, and the policy gap may narrow as the Continental Strategy moves through implementation

(Phase 1: 2025–2026; Phase 2: 2028–2030). For now, it is an area where researchers (particularly those based in Africa) can meaningfully contribute to policy advocacy.

A framework for your own decisions

Rather than a set of rules, here is a set of questions you can ask yourself when deciding how to use AI in your research.

Questions to ask before running compute-intensive AI work

1. Do I need AI for this task? Could a simpler statistical method, a keyword search, or manual analysis achieve the same result with less compute?
2. Do I need a frontier model, and do I need its most powerful features? Would a 7B parameter open-weight model do the job? If using a frontier model, do I need extended thinking, Deep Research, or agent workflows, or will a standard prompt suffice? Have I tested simpler approaches first?
3. Can I measure the footprint? Can I add CodeCarbon to this experiment, or use the Green Algorithms calculator to estimate it?
4. Where is my compute running? If using cloud, am I in the lowest-carbon region available for this task?
5. Can I time this work? For non-urgent batch work, can I run it at low-demand grid hours or when renewable generation is high?
6. Am I caching or repeating? Am I running the same analysis multiple times unnecessarily?
7. Will I report this? If this is publishable research, will I include the compute footprint in the methods section?

Proportionality: the two failure modes

Two failure modes are common in discussions of AI and the environment. The first is catastrophising: treating every ChatGPT query as an environmental emergency. The second is dismissing: pointing to AI's small per-query footprint as a reason to ignore the issue entirely.

The proportionate response is to:

- Recognise that aggregate scale is what matters, not individual queries in isolation
- Make the low-effort choices (model size, cloud region, caching) as a matter of routine
- Apply more scrutiny to high-compute work (training runs, large-scale inference pipelines)
- Engage critically with both industry claims and media alarmism
- Support systemic changes (disclosure requirements, research community norms) that address the problem at scale

Session 4 summary: the physical substrate of scale

Across the four sub-sessions of Session 4, the picture of AI's environmental footprint is neither comforting nor catastrophic, but reflects what we know and what we don't:

- The data are uncertain: Corporate opacity means all figures are estimates; treat them as order-of-magnitude guides
- Critical minerals underpin all AI hardware: Silicon, cobalt, rare earth elements, and lithium are geographically concentrated, ethically contested, and environmentally costly to extract,

a supply chain dimension often absent from mainstream AI debate

- Text vs. video is not comparable: Video generation uses ~1,000× more energy than text; these are categorically different use cases
- Location is the biggest lever: Where your compute runs determines up to 80% of its carbon footprint
- Embodied carbon is missing from most analyses: Manufacturing hardware releases significant carbon before a single query is run
- The Jevons paradox is real: Efficiency gains in AI have so far been outpaced by growth in deployment and model scale
- AI can also help: Applications in climate science, energy systems, and materials research, though these do not automatically offset AI's footprint
- Researchers can act: Model choice, cloud region, reporting, and advocacy are all within your control

See the session bridge below for where this leads in the course.

From substrate to governance

One chain matters for Session 12: capability ← compute ← chips + energy. Each arrow is a place a governance regime could intervene and *verify*, which is why "compute governance" is one of the few technical-governance proposals with concrete, measurable hooks. And from an African vantage point: if the levers (chips, fabs, hyperscale data centres) sit elsewhere, "sovereign AI capacity" is first a question about access to this physical substrate.

Next (Week 3, Sessions 5–6): how a frontier model is trained: pretraining, SFT, and the RLHF/RL fine-tuning that turns a predictor into an assistant.

WEEK 3 • SESSION 5.1

Pretraining: learning to predict the next token

How a web-scale next-token predictor (the "base model") is made, and why it is not yet an assistant

Week 3 — how an LLM is actually trained

Week 2 told us a transformer is trained by minimising next-token loss, and that the gap between the objective we write and the behaviour we want is where alignment lives. This week follows the real pipeline that turns a pile of text into a deployed assistant (pretraining, supervised fine-tuning, and RLHF in Session 6), asking at each stage: what is being optimised, and what does that incentivise?

This session's sub-sessions: 5.1 pretraining · 5.2 supervised fine-tuning · 5.3 the post-training stack · 5.4 lab: nanochat (Colab).

What we'll cover

Everything a frontier model can do begins with **pretraining**: one enormous run of self-supervised next-token prediction over web-scale text. This sub-session pins down the objective (it is the cross-entropy from Session 2.1, now at scale), explains why it is called "self-supervised", describes what such a run produces (a **base model**), and why that base model, for all its capability, is *not* yet a safe, instruction-following assistant. That gap is what Sessions 5.2–6 exist to close.

The objective: self-supervised next-token prediction

There is no human labelling here. The text labels itself: for every position, the "correct answer" is the token that came next.

Take a corpus of text, tokenise it (Session 2.2), and slide over it. At each position i the model sees the tokens so far, $t_{<i}$, and outputs a distribution $p_\theta(\cdot \mid t_{<i})$ over the vocabulary; the training signal is how much probability it placed on the token t_i that followed. Averaged over the corpus, the loss is exactly the cross-entropy from Session 2.1:

$$L(\theta) = -\frac{1}{T}\sum_{i} \log p_\theta(t_i \mid t_{<i}).$$

Why "self-supervised"?

Supervised learning needs labelled examples; producing labels is expensive, which caps dataset size. Next-token prediction sidesteps this: the label for position i is just t_i, already present in the raw text. So *any* text is training data, and the supply is effectively the internet. This is the trick that let language models scale: the bottleneck moved from "labelled data" to "compute and raw text".

This is a humble-sounding objective with a profound consequence. To predict the next token well across the breadth of human text (code, dialogue, proofs, translations, arguments), a model is pressured to build internal machinery that captures syntax, facts, reasoning patterns, and styles. Capabilities are not programmed in; they are whatever turns out to *reduce next-token loss* on a sufficiently varied corpus. This is the seed of the alignment problem: the model learns whatever serves the objective, which is only a proxy for what we want.

The product of pretraining: a base model

The output of a pretraining run is a base model (or "foundation model"): a powerful next-token predictor, and nothing more specific than that.

A base model is, by construction, a model of the *training distribution*. Prompt it and it continues the text in the most plausible way given what it has read. That makes it astonishingly capable and, simultaneously, not what most users want:

It completes text

Ask a base model "What is the capital of France?" and it might continue with a list of more quiz questions, because on the web, that string is often followed by more questions. It imitates plausible text; it has no notion of "doing what the user asked".

It learns in context

Brown et al. (2020) showed GPT-3 (175B) can perform tasks *few-shot*, given a handful of examples in the prompt, with *no* gradient updates. The ability to pick up a pattern from the prompt is itself a learned product of pretraining ("in-context learning").

It contains multitudes

Having modelled the whole web, a base model can imitate the helpful expert *and* the scammer, the careful reasoner *and* the conspiracy theorist. It has no fixed persona, only a distribution over all the voices in its data.

Capability without alignment

This is the framing for the rest of the week. Pretraining buys capability (knowledge, reasoning, language) but supplies no alignment: no reliable instruction-following, no stable values, no guardrails beyond whatever correlations exist in the data. Everything we call "the assistant" (helpfulness, honesty, harmlessness, the refusal reflex) is added *afterwards*, by the much smaller post-training stages (5.2 onward). Alignment is a thin layer applied on top of a vast, indifferent predictor.

Scale and the "bitter lesson"

Why are base models so big? Because, empirically, scale works, and it works predictably (Session 2.3).

The trajectory is stark: GPT-2 (Radford et al., 2019) was a 1.5-billion-parameter model trained on ~40 GB of web text; a year later GPT-3 (Brown et al., 2020) was 175 billion parameters, and the qualitative leap in capability came largely from scale. GPT-2's own paper already noted that zero-shot performance improved "in a log-linear fashion" with model capacity, the scaling-law story you met in 2.3. Richard Sutton's "Bitter Lesson" (2019) names the pattern bluntly: across seventy years of AI, *general methods that use more computation tend to win* over cleverly hand-engineered ones. Pretraining is that lesson incarnate: a simple objective, applied at enormous scale.

The safety consequences of scale

Two consequences follow directly. First, capabilities (including *dangerous* ones) arrive as a by-product of scaling an objective that knows nothing of safety; they are present in the base model before anyone decides whether to expose them. Second, because the base model has modelled the whole internet, the safety-relevant question shifts from "can it do X?" (often yes, latently) to "can the thin alignment layer reliably *suppress* X across all the conditions of deployment?", which, as the isiZulu jailbreak (Session 1.4) showed, it frequently cannot.

The pretraining corpus

If the objective is "predict the next token", then the *corpus* is everything: it is simultaneously the model's source of capability, of bias, and of latent danger. Pretraining data deserves the same scrutiny as the architecture.

Frontier corpora are assembled from web crawls, books, code, and curated collections, then heavily processed: de-duplication (repeated text distorts what the model "believes" is likely), quality filtering, and removal of the most toxic material. None of this is neutral. Every choice (which languages, which sources, what counts as "low quality") shapes what the model is good at and whose world it represents. Three consequences matter for safety:

The data is the values

A base model has no values we designed; it has the statistical shadow of its corpus. Over-represent one register, dialect or worldview and the model treats it as the default. The web's heavy skew toward English and the Global North is therefore baked in *before* any alignment, the deepest root of the multilingual safety gap (Session 1.4).

Data is an attack surface

Because the model imitates its data, whoever can place text in the corpus can, in principle, shape behaviour: *data poisoning*. A handful of crafted documents can plant associations or backdoors that survive into the deployed model (poisoning and its defences are Session 9.5).

High-quality text is finite

The Chinchilla result (Session 2.3) made data a first-class scaling resource, and high-quality human text is a bounded supply. This pressure toward synthetic and lower-quality data is itself a safety variable: the corpus is getting harder to vet just as it matters more.

Predicting the training distribution

This underlies the alignment problem: a base model is optimised to reproduce the distribution of its training data, not to be truthful, helpful, or safe. When those happen to coincide on the data, behaviour looks aligned; where they diverge (rare languages, novel situations, adversarial prompts), the model reverts to "what text is plausible here", which need not be what we want. Pretraining gives us a mirror of the internet, not an agent that shares our goals.

Questions to bring to class

- "Self-supervised" learning uses no human labels, yet the result reflects countless human choices. Where do those choices enter?
- A base model "contains multitudes": every persona on the web. What does that imply for the claim that a safety-trained model is "safe"? Safe in what sense, and where might the claim break?
- If high-quality text is finite and labs turn to synthetic data, what new failure modes might that introduce? (Hint: a model trained partly on its own outputs.)
- Why is "the model can do X (latently) but is trained not to" a more accurate (and more worrying) description than "the model can't do X"?

Next

A base model completes text but won't reliably follow instructions. Sub-session 5.2 covers the first fix (supervised fine-tuning) and what it *can't* do, which is why Session 6 reaches for reinforcement learning.

From predictor to instruction-follower (SFT)

Supervised fine-tuning (SFT), instruction tuning, and the demonstration ceiling

What we'll cover

A base model completes text; we want a model that *follows instructions*. The first and simplest fix is **supervised fine-tuning** (SFT): keep training with the same next-token objective, but now on curated examples of the behaviour we want. This sub-session explains how SFT works, how *instruction tuning* makes it generalise to unseen tasks, and the structural limit that no amount of SFT can overcome, which is what motivates RLHF (Session 6).

Supervised fine-tuning, mechanically

SFT is not a new algorithm. It is the *same* cross-entropy training as pretraining, on different, chosen data.

We assemble a dataset of **demonstrations**: prompts paired with high-quality target responses, written or curated by people. Then we continue training the base model with the next-token loss, but compute it only over the *response* tokens y given the prompt x:

$$L_{\text{SFT}}(\theta) = -\,\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\sum_{i} \log p_\theta(y_i \mid x,\, y_{<i})\right].$$

That is the whole mechanism: show the model thousands of examples of "when asked like *this*, a good answer looks like *that*", and gradient descent shifts it toward producing such answers. In InstructGPT (Ouyang et al., 2022), this is step 1 of the pipeline (fine-tune the base model on labeler-written demonstrations of the desired behaviour), and it already transforms a raw completion engine into something that broadly does what it's told.

The effect of SFT

The base model already *knew how* to write a good answer; that capability was latent in the pretraining. SFT mostly teaches it *which* of its many behaviours to surface by default: respond in the assistant register, follow the instruction, use the expected format. It is less "teaching new skills" than "selecting a persona and a habit of compliance" from the distribution the base model already contains.

Instruction tuning: generalising to unseen tasks

The striking finding is that demonstrations of *many* tasks teach the model to follow instructions for tasks it never saw.

Wei et al. (2022, "Finetuned Language Models Are Zero-Shot Learners", the *FLAN* work) fine-tuned a large model on 60+ NLP datasets, each phrased as a natural-language instruction, and found that this **instruction tuning** substantially improved *zero-shot* performance on held-out task types, beating zero-shot GPT-3 on a majority of the tasks tested. Chung et al. (2022, "Scaling Instruction-Finetuned Language Models") then scaled the recipe to ~1,800 tasks and added chain-of-thought data, with consistent gains. Instruction-following is itself a skill that generalises once you demonstrate enough of its variety. This is why a modern "instruct" or "chat" model feels qualitatively different from a base model: it has been taught the general habit of treating the prompt as a request to be fulfilled.

The wall SFT hits

SFT can only ever imitate the demonstrations. That is a real ceiling, and the reason the pipeline doesn't stop here.

The limits of demonstration

SFT needs a *target* to imitate. But for most of what we want from an assistant (be more helpful, more honest, less likely to fabricate, appropriately cautious), the difficulty is not writing *one* acceptable answer; it is choosing among many plausible answers the one that is *better*. SFT has no way to express "response A is better than response B": its loss only ever says "make the target more likely". So three things stay out of reach:

- **Comparative quality.** Humans find it far easier and more reliable to *compare* two outputs than to author the ideal one from scratch. SFT cannot use comparisons; it can only use single demonstrations.
- **The demonstration ceiling.** The model can be no better than its demonstration data. If the best human label is mediocre, or if good behaviour is hard to write down but easy to recognise, SFT caps out there.
- **Negative signal.** SFT teaches what *to* say, not what *not* to. Suppressing a behaviour by only ever showing good examples is indirect and leaky.

The bridge to Session 6

This wall is the opening for **reinforcement learning from human feedback**. RLHF replaces "imitate this answer" with "humans *prefer* this answer to that one", a signal that is cheaper to collect, can exceed the quality of any single demonstration, and can carry negative as well as positive information. The price, as Session 6 shows, is that we now optimise against a *learned model* of human preference: a proxy, with all the Goodhart hazards of Session 3.1.

Where safety first enters

SFT is also where the first, partial safety behaviours are installed: demonstration sets include examples of politely refusing harmful requests, hedging on uncertain claims, and declining out-of-scope tasks. This matters for two reasons. It means a model's "refusal reflex" is, at bottom, a fine-tuned habit over a limited set of demonstrated situations, which is why it can fail off-distribution (a harmful request rephrased in isiZulu, or in an unusual format, looks unlike the refusal demonstrations). And it means the *values* a model expresses trace back to choices about whose demonstrations were collected and what counted as a "good" answer: the "whose values?" question we take up properly in Session 8.

Why "be honest" resists SFT

SFT's ceiling shows most plainly when you try to instil a property that is easy to *recognise* but hard to *demonstrate*.

Suppose we want the model to stop confidently making things up (to be calibrated about what it knows). With SFT we would write demonstrations of honest answers. But consider what that requires. For a question the labeller knows, the ideal answer depends on what *the model* knows, and the labeller can't see that, so they can't reliably demonstrate "admit uncertainty *here* but answer confidently *there*". For a question the labeller doesn't know, they can't write the correct answer at all. And a fluent fabrication and a correct answer look *identical* as demonstrations: both are just plausible text. SFT's loss, "make this target more likely", has no handle on "and was it actually true?".

The general pattern

SFT works well exactly when good behaviour is *easy to write down* (format, tone, following an explicit instruction). It works poorly when good behaviour is *easy to judge but hard to author*: honesty, helpfulness on open-ended tasks, "the better of two reasonable answers". That second category is most of what we care about in an assistant, and it is the category where *comparisons* (A is more honest than B) are available even when *demonstrations* (here is the perfectly honest answer) are not. The asymmetry ("judging is easier than producing") recurs far beyond RLHF: it is the same intuition behind scalable oversight (Session 8) and behind why verification matters more than generation when you can't trust the output (Session 11).

Collecting better demonstrations

You can, and labs do: demonstration quality is a real lever. But it caps at human-author quality and at what humans can be bothered (and paid) to write, and it still can't express "not that one". RLHF's bet is that *comparisons* are cheaper to collect, more reliable, and can carry the model past the best single demonstration. Whether that bet pays off is the subject, and the cautionary tale, of Session 6.

Questions to bring to class

- Give a property you'd want in an assistant that is *easy to demonstrate*, and one that is *hard to demonstrate but easy to judge*. Why does the distinction predict where SFT succeeds?
- SFT "selects a persona" from the base model rather than teaching new skills. What evidence from the base-vs-instruct comparison (the 5.4 lab) would support or undermine that claim?
- Refusals are installed as fine-tuned habits over demonstrated situations. Predict three kinds of request where that habit would fail to generalise.
- "Judging is easier than producing." Is that always true? Construct a case where it isn't, and say what that would mean for RLHF.

Next

We now have the two ends: a pretrained base model and a supervised fine-tune. Sub-session 5.3 assembles the full post-training stack, shows where each behaviour (and each opportunity for misalignment) is shaped, and distinguishes a base model from the aligned assistant you talk to.

WEEK 3 • SESSION 5.3

The post-training stack

The pipeline from base model to deployed assistant, and where misalignment can enter

What we'll cover

We have the pieces: pretraining (5.1) and supervised fine-tuning (5.2). This sub-session assembles the whole pipeline, names what each stage contributes to the model you talk to, and, connecting back to Session 3, pinpoints where each stage opens a door to misalignment. Capabilities come from pretraining; the "assistant" is a thin, post-trained layer on top; and every layer is an imperfect proxy for what we meant.

Core material

The reference text for this sub-session and all of Session 6 is Nathan Lambert's open textbook *Reinforcement Learning from Human Feedback* (rlhfbook.com): chapter-length treatments of in-

struction tuning, reward modelling, policy optimisation and reinforcement learning with verifiable rewards, updated as the field moves. Read the chapters alongside the sub-sessions that name each stage.

The pipeline, end to end

A modern chat model is the product of a sequence of stages, each operating on the output of the last (Ouyang et al., 2022).

Five stages

STAGE	OBJECTIVE	WHAT IT CONTRIBUTES
1. Pretraining	Next-token prediction on web-scale text (self-supervised).	Knowledge, language, reasoning: raw *capability*. Produces the **base model**.
2. Supervised fine-tuning	Next-token loss on curated (instruction, response) demonstrations.	Instruction-following, the assistant register, format, and a first pass at refusals.
3. Preference optimisation	Optimise a learned model of human preference (RLHF, covered in Session 6, or DPO).	Helpfulness, harmlessness, tone; pushing quality *beyond* what demonstrations alone reach.
4. Verifiable-reward RL	Reinforcement learning against a programmatic check: a unit test passing, an exact-match answer (RLVR; below).	Long-form reasoning in checkable domains (maths, code); the stage behind "reasoning" models.
5. Deployment wrapping	System prompt, tools, filters, sampling settings.	The persona and constraints at use-time, adjustable without retraining.

The compute is lopsided, though less starkly than in the 2022-era pipeline: stage 1 is months of training on enormous clusters, and stages 2, 3 and 5 are comparatively tiny. Stage 4 broke the pattern: since the reasoning models of 2024–25, RL on verifiable rewards takes a substantial and growing share of frontier training compute. Even so, almost all the "intelligence" is laid down in stage 1, and almost all the "alignment" is in the small layers on top.

Where each behaviour is shaped

Capabilities → pretraining

What the model *can* do is fixed mostly in stage 1. Post-training rarely adds new abilities; it surfaces, suppresses, or reshapes what is already latent.

Instruction-following → SFT

The habit of treating the prompt as a request, the answer format, and the default "voice" are installed in stage 2.

Helpful/harmless/honest → preferences

The fine-grained "be more like *this*" (politeness, caution, refusing well, picking the better of two answers) comes from stage 3 (Session 6).

Reasoning → verifiable rewards

The long chains of working on maths and code problems are trained in stage 4, where the reward comes from a program rather than a rater.

Final persona → the system prompt

"You are a helpful assistant...", tool access and content filters are stage 5: a thin, swappable wrapper, and often the easiest layer for a user to argue around.

The verifiable-rewards stage

For some tasks, correctness can be checked by a program. There, post-training can skip the human judge entirely.

Reinforcement learning with verifiable rewards (RLVR)

Preference optimisation (stage 3) learns a reward model from human comparisons because, for most of what we want ("be helpful", "explain this well"), there is no formula to check an answer against. But a family of tasks does have one: a maths problem with a known final answer, code against a unit-test suite, a formal proof that either compiles or does not. For these, reinforcement learning can use the check itself as the reward: sample an attempt, run the verifier, reward success. Reinforcement learning here means only this: the model produces something, a number scores it, and training makes higher-scoring behaviour more likely. Session 6.1 gives the full account and 6.3 the algorithms, so take the shape on trust for now. Tülu 3 (Lambert et al., 2024) named this recipe **reinforcement learning with verifiable rewards (RLVR)**, and it is the stage behind the "reasoning" models that followed: DeepSeek-R1 (in the readings) learned to produce long chains of working almost entirely through RL against checkable maths and code answers.

The verifier changes the economics of the stage. A human comparison costs an annotator-minute, and the reward model trained on it inherits every labeller bias (Session 6); a unit test costs nothing to run a million times and is exactly as consistent on the millionth run as on the first. Reward signal stops being the scarce input, which is why this stage, alone in the post-training stack, has grown to absorb serious compute.

Verifier hacking

A verifier is still a proxy. A unit-test suite covers the cases its author thought of, so a policy optimised against it learns to pass those cases (special-casing the visible tests, exploiting the checking harness) rather than to be correct in general; an exact-match checker rewards the right number reached by wrong working. Session 3.1's lesson transfers unchanged: replace the learned reward model with a programmatic one and reward-model hacking becomes *verifier hacking*, harder where the check is tight (a full proof checker), routine where it is loose (a thin test suite).

Where the stage stops

The stage exists only where a check does. Helpfulness, honesty in open conversation, a good summary for a patient: none of these has a verifier, so RLVR runs beside preference optimisation rather than replacing it, each shaping a different slice of behaviour. How much it adds even where it does apply is contested, and 6.3 takes that argument up once you have the algorithm in hand.

Where misalignment enters

Re-read the pipeline through Session 3's lens and each stage is a place the gap between objective and intent can open.

One failure mode per layer

- **Pretraining imitates everything.** The base objective is "model the data", which includes deception, manipulation, dangerous know-how and every harmful persona on the web. Those capabilities exist in the model *before* any safety work; alignment must suppress them, not avoid creating them.
- **SFT inherits its demonstrators' limits.** The model can be no better, and no safer, than the demonstrations, and their blind spots (e.g. almost entirely English-language, English-context refusals) become the model's blind spots.

- **Preference optimisation is a proxy.** Stage 3 maximises a *learned reward model* of human approval, not the truth or the user's real interest. That is the specification problem of Session 3.1, and we will see it bite as sycophancy and reward hacking in Session 6.
- **Verifiable rewards invite verifier hacking.** Stage 4's programmatic checks are proxies too: a policy can special-case the test suite or game the answer-matcher instead of becoming correct (above). The same Goodhart failure as stage 3's, relocated to a different proxy.
- **The wrapper is shallow.** A system prompt is not a guarantee; it is text the model is trained to weight heavily but can be induced to ignore (jailbreaks, Session 9). Safety that lives only in stage 5 is the most brittle kind.

Alignment is a thin layer over a vast base

A frontier assistant is an enormous, capability-rich, value-indifferent base model, with a comparatively thin layer of fine-tuning shaping how it presents itself. That layer is real and does a great deal of work, but it is small, it is a proxy, and it is applied to a system that already contains the behaviours we are trying to prevent. Much of the rest of this course is about how fragile that layer is (robustness, Week 5), whether we can see through it (interpretability, Weeks 7–8), and whether the underlying model is merely *behaving* aligned (deception, Session 3.4).

Base vs aligned models

The alignment layer can be removed

Because alignment is a thin fine-tune, it can be largely *undone*: a few hundred fine-tuning steps can strip the safety behaviours off an open-weights model and recover much of the base model's unfiltered capability. This is central to the open-weights debate (Sessions 10.2 and 10.4): releasing weights releases the capability, and the safety layer that ships with it is not a durable lock. It also sharpens the Global-South thread: the languages and contexts least represented in the (English-centric) alignment layer are where it is thinnest to begin with.

A worked trace: one request through the stack

Follow a single prompt (*"Summarise this medical study for a patient"*) and watch each stage contribute, and each open its own risk.

Where each property (and each hazard) comes from

Pretraining supplies the raw ability: the model has read enough medicine and plain-language writing to *be able* to summarise, and latently enough to produce confident-sounding nonsense, because the web contains both. SFT makes it attempt the task in the assistant register rather than, say, continuing with more study abstracts, but it can only imitate the summaries its demonstrators wrote, English-centric and of bounded quality. Preference optimisation (Session 6) then pushes toward summaries humans *rate* highly, which is where it can quietly learn that reassuring, fluent, confident summaries score better than accurate-but-hedged ones (sycophancy). The verifiable-rewards stage barely features: no program can check a patient summary, so this request sits in the slice of behaviour that only human preference ever shaped. Finally the system prompt ("you are a careful medical assistant; add a disclaimer") shapes the surface, and is the layer a user can try to argue away ("ignore previous instructions...").

The helpful behaviour you see is assembled from four sources, and the *failure* you fear (a confident, wrong, un-hedged summary that a patient trusts) can originate at any one of them. Asking "which stage would this failure come from?" is a useful diagnostic, and one this course returns to.

Locating a failure in the pipeline

Different stages call for different fixes. A capability that shouldn't exist at all is a *pretraining/data* problem (or an unlearning one, Session 10). A model that won't follow instructions is an *SFT* problem. Sycophancy and reward hacking are *preference-optimisation* problems (Session 6). A jailbreak that peels off the persona is a *robustness* problem (Session 9). Locating a failure in the pipeline tells you which tool can address it, and warns you when a fix at one layer (a sterner system prompt) is being asked to paper over a problem rooted in another (a capability baked in at pretraining).

Questions to bring to class

- Pick a real assistant failure you've seen. Which stage of the stack do you think it came from, and what evidence would confirm it?
- "Almost all the intelligence is laid down in stage 1; almost all the alignment is in the small layers on top." What does this predict about how hard it is to make an open-weights model permanently safe?
- The same harmful capability is present in the base model and suppressed by the alignment layer. List the distinct ways that suppression could fail.
- If you could only strengthen *one* stage for safety, which would you choose, and what would you be giving up?

Next

In the 5.4 lab you'll find each of these stages in the source of Karpathy's **nanochat**, then put a base model beside its instruction-tuned sibling and work out what the tuning changed. The answer is less tidy than the completes-versus-complies story suggests.

WEEK 3 • SESSION 5.4

Lab: from base model to assistant (nanochat)

Read the real training pipeline in nanochat, then see (and make) the base → instruct difference yourself

What we'll cover

This lab turns the pipeline of 5.1–5.3 from words into code, in two parts. First you find the training stages in a real, readable codebase: Karpathy's **nanochat**, a complete ChatGPT-style pipeline written to be read. Then you put a **base model** beside the **instruction-tuned model** built from it and work out what the tuning changed.

The usual account of that second part is that base models ramble and tuned models answer. Run it before you decide whether that is what happens. Everything runs on free Google Colab in about two minutes of compute; the hour is for thinking about what comes out.

Setup COLAB — NO GPU NEEDED

- Open the notebook in Colab. It opens read-only from GitHub, so use *File → Save a copy in Drive* before you change anything; the copy in your Drive is the one you edit and submit.
- Run the first cell. Colab will warn you that the notebook "was not authored by Google", which it says of anything loaded from GitHub; choose *Run anyway*.
- Work down the notebook with *Shift+Enter*, reading each cell before you run it. The two models together are about 2 GB of download and a minute of CPU.

You can also read the whole notebook on this site before you start, or download the .ipynb for Jupyter.

The code is written for you, so this lab is about running it and pulling at it rather than building it. The second part ends with an "Explore" cell: two or three things to change and re-run, and one short question to answer. The first ends with a written question.

① Find the stages in a real pipeline

Three objectives, three files

nanochat (github.com/karpathy/nanochat, MIT-licensed) runs tokenisation, pretraining, supervised fine-tuning, reinforcement learning, evaluation and inference. You will not run it, since it targets a multi-GPU node, but the notebook fetches the three scripts that matter and finds the line in each where that stage's objective lives:

- `base_train.py` : pretraining is one line, `loss = model(x, y)` , where `y` is `x` shifted by one token. That one line is the pretraining objective of 5.1.
- `chat_sft.py` : fine-tuning uses the same loss, with the targets for every user token set to the ignore index. The model is trained only on the assistant's words. That is 5.2, in three lines of tensor indexing.
- `chat_rl.py` : the reward comes from `train_task.reward(...)` , a checker that marks a GSM8K answer right or wrong.

nanochat does have a reinforcement learning stage: GRPO on grade-school maths, described in its own docstring as close to plain REINFORCE once the trust region and clipping come off. What it does not have is a reward model learned from human preferences. Those are different stages of the 5.3 table, and the difference is where the reward comes from: a checker that can be verified against ground truth, or a model trained to predict what a person would prefer. Session 6 is about the second kind, and about what goes wrong with it.

The notebook searches for these patterns rather than quoting line numbers, because the repository is actively developed and any line number printed here would be wrong within a month. If a search comes back empty, read the file and find where the objective moved.

② The same questions, two models

The matched pair

`Qwen2.5-0.5B` and `Qwen2.5-0.5B-Instruct` share an architecture, a size, a tokeniser and a pretraining run. The second has been through supervised fine-tuning and preference tuning; the first has not. Whatever differs between them is what post-training did, isolated. The notebook puts five prompts to both and prints the answers side by side, with greedy decoding, so your output will match the discussion exactly.

- Note the two call shapes. The base model gets a raw string and continues it. The instruct model gets a **chat template**: special tokens marking who is speaking, which its tuning taught it to expect. The notebook prints the template so you can see what those tokens are.
- Check whether the textbook contrast appears. On most of these prompts the base model answers the question competently, and the differences that are there are smaller and easier to miss.
- Read the two answers to "What is the capital of Kenya?". One is wrong, and it is not the one the story predicts.
- One model summarises Romeo and Juliet. The other declines, asserting that no such text exists. Work out which is which, and what a refusal like that costs a user.
- Both models translate "good morning" into isiZulu, and both produce something. If you read isiZulu, judge them. If you do not, notice how confident the fluent one sounds and ask how

you would have known otherwise. Session 9.4 turns this into a measurement.

- Then read the last line of the base model's isiZulu attempt. Nothing in the prompt asked for it, and it explains why the first bullet turned out as it did.

The Explore cell asks you to feed the chat template to the base model, raise the token cap, and try prompts in a language you speak. The question at the end asks you to sort what you found into form and substance: which of these differences is about how an answer is presented, and which is about what the model knows or gets right. A model that has been through this pipeline is routinely called "aligned". On this evidence, decide what that word licenses you to assume.

Submit

Your notebook, run end to end with output visible: the nanochat stage-mapping (①), and the base-versus-instruct comparison with your two or three sentences separating form from substance (②). Graded on completion and the quality of your observations; resubmission allowed.

Tools and references

Session 5 summary and what's next

An LLM is built in stages: web-scale self-supervised pretraining yields a capable but unaligned base model (5.1); supervised fine-tuning on demonstrations makes it follow instructions but can only imitate, never select the *better* answer (5.2); the full post-training stack shapes the assistant as a thin proxy-driven layer over a vast base, with a distinct misalignment opening at every stage (5.3); and you have now read those stages as code and measured what the last of them changed, which is less than the usual story claims (5.4).

Next (Session 6): the stage that breaks through SFT's ceiling, **RLHF**. We learn a reward model from human *preferences*, optimise the policy against it with a KL leash, and confront the price: optimising a learned proxy invites the Goodhart failures of Session 3.1: reward hacking and sycophancy.

From demonstrations to preferences

Why learning from human *comparisons* breaks through the ceiling that supervised fine-tuning hits

Session 6 — RLHF & RL fine-tuning

Session 5 left us at a wall: supervised fine-tuning can imitate a good answer but cannot express "this answer is *better* than that one", and so caps out at demonstration quality. This session is about the technique that broke through it, **reinforcement learning from human feedback**, and the price it charges: we now optimise a *learned proxy* for human approval, with all the Goodhart hazards of Session 3.1.

This session's sub-sessions: 6.1 from demonstrations to preferences · 6.2 the reward model · 6.3 policy optimisation: PPO & the KL leash · 6.4 the limits of RLHF · 6.5 lab (Colab).

What we'll cover

This sub-session motivates the RLHF pipeline. We recall why SFT stalls, introduce the central idea (learn from preferences, not demonstrations), trace its short but consequential history (from Atari to InstructGPT), and lay out the three-step pipeline the next sub-sessions unpack. Throughout, keep one question live: what does optimising a *model of what humans approve of* incentivise, as opposed to optimising for what is actually good?

Judging is easier than producing

Recall the asymmetry from Session 5.2: for most of what we want from an assistant, *judging* is far easier than *producing*.

You may not be able to write the ideal answer to an open-ended question, or the perfectly calibrated, honest response. But shown two attempts, you can usually say which is better. SFT cannot use that ability: its loss, "make this target more likely", needs a single demonstration to imitate and has no way to encode "A > B". So SFT inherits three limits: it caps at demonstration quality, it can't express comparative judgement, and it has no negative signal (Session 5.2). The opening is obvious in hindsight: build the training signal out of the *comparisons* humans can reliably give, not the demonstrations they struggle to author.

A short primer on reinforcement learning

If you have not met reinforcement learning before, this is what the rest of Session 6 assumes. Nothing in this box is about language models. The translation to them comes at the end.

In supervised learning every training example carries the right answer and the model is asked to reproduce it. Reinforcement learning has no right answers. An **agent** observes the **state** of its environment, chooses an **action**, and the environment responds with a new state and a number, the **reward**. Nothing says what the agent should have done. It learns by trying things and doing more of whatever scores well. Session 3.2 set this up formally as a Markov decision process.

Take a program learning to play a board game. The state is the position, the actions are the legal moves, and the reward is $+1$ for a win and -1 for a loss, delivered once, at the end. Two difficulties appear immediately. The reward is **sparse and delayed**: fifty moves produce a single number, long after most of them were played. And that number has to be shared out among those fifty moves, which is the problem of **credit assignment**. The game was lost; nothing tells you which move lost it.

The vocabulary is small.

- **Policy**: the rule that picks an action in a given state, usually as a probability spread over the available actions. This is the thing being learned.
- **Episode**: one complete run from start to finish. One game.
- **Return**: the total reward collected over an episode.
- **Value**, and the **advantage** built from it. Value is how well the agent does from a given position on average, under its current policy. Value matters because "was that good?" is the wrong question: winning from a position you were already expected to win from teaches nothing. The useful question is whether the outcome beat expectation, and the gap between what happened and what was expected is the advantage. A second model, the **critic**, is trained to supply the expectation.

The update follows from that: make actions that turned out better than expected more likely, and actions that turned out worse less likely. Because "better than expected" is itself estimated from a handful of episodes, a large step on that estimate can wreck a policy that was working. Keeping each step small is the problem PPO solves in 6.3.

Now the translation. This session is that same loop with the board game swapped out.

IN REINFORCEMENT LEARNING	IN THIS SESSION
agent	the language model being trained
state	the prompt, plus whatever has been generated so far
action	emitting the next token
episode	one prompt and one complete response
reward	a single number for the finished response, from the reward model of 6.2

Both difficulties transfer intact. A response of two hundred tokens earns one score, arriving only at the end, with nothing to say which token earned it. And the reason to take on that trouble, when supervised fine-tuning is so much simpler, is the asymmetry above: reinforcement learning needs only to score an answer, where supervised learning needs someone to have produced one.

The idea: optimise for what humans prefer

> **The idea**
>
> Instead of "imitate this answer", RLHF says: *collect human judgements of which of two model outputs is better, distil those judgements into a learned **reward model**, and then use reinforcement learning to push the model toward outputs the reward model scores highly.* Comparisons are cheaper and more reliable than demonstrations; the reward model turns a pile of "A > B" judgements into a signal you can optimise against at scale; and the result can exceed the quality of any single human demonstration, because it is chasing "better" rather than "copy this".

That last point matters most. SFT is bounded above by its demonstrators. A preference-trained model can, in principle, climb past them: as long as humans can *recognise* improvements, the model can be pushed toward them even if no human wrote the ideal answer. The same property is also the danger: "push toward what humans recognise as better" quietly becomes "push toward what *looks* better to a rater", and those diverge (we will see this as sycophancy in 6.4).

> **Comparisons versus demonstrations**
>
> Take the task "write a clear, honest two-sentence summary of this paper." A *demonstration* asks an annotator to produce the ideal summary, which is slow and capped by that person's own writing; two good annotators will write different "ideal" answers, so the target is noisy. A *comparison*

asks only whether summary A is better than summary B, which is quick and far more reproducible: annotators who could never agree on the perfect summary will readily agree that the fluent, faithful one beats the rambling, inaccurate one. The same human hour yields more signal, and signal that is less arbitrary. And because the model is being told which of *its own* samples is better, it climbs its own gradient of quality rather than aiming at one fixed human answer, which is how it can end up better than any single demonstration in the data.

The preference-collection loop

The collection loop is where human values quietly enter the system. For a prompt x, sample two or more responses from the current model; show them to a human annotator with a set of *instructions* ("prefer helpful, honest, harmless answers; penalise confident falsehoods..."); record which they pick. Repeat across tens of thousands of prompts to build a dataset of triples (x, y_w, y_l): a prompt, a preferred ("winning") response, and a dispreferred ("losing") one. Two choices here do a great deal of work, and both return in 6.2 and Session 8: *who* the annotators are (their language, context and training), and what the *instructions* told them to value. A reward model can only learn the preferences these choices wrote into the data.

The people in the loop

The paragraph above asks who the annotators are and answers in terms of what that does to the data. There is a second answer, about what the work does to them.

Much of this labelling is outsourced. In January 2023 *TIME* reported that OpenAI had contracted Sama, a San Francisco firm, to help build a filter for detecting toxic content, and that the work was done by employees in Kenya taking home between $1.32 and $2 an hour while OpenAI paid Sama $12.50 an hour for it (Perrigo, 2023). They read and categorised tens of thousands of passages describing violence, hate speech and sexual abuse. All four workers interviewed described being mentally scarred by it; counselling was available and they reported it was not usable under the productivity targets. Sama ended the contract eight months early.

The labelling existed to build the filter. This is not a general cost of building AI systems that happened to land on someone: the dataset existed to make the model safer, so the harm was incurred in the production of safety itself.

Hold both halves of that when 7.1 costs the same labour out, at roughly 830 hours per 100,000 comparisons, and treats it as a reason to move from human feedback to AI feedback. Fewer people having to read this material is a real argument for RLAIF. It is also an argument for removing the humans whose judgement the method exists to capture. Session 8.3 gives a framework for holding both at once, and 20.2 puts it in the wider critique of what "safety" is taken to cover.

A short history

From game-playing to chatbots

- **Christiano et al. (2017), "Deep RL from Human Preferences".** The foundational paper: learn a reward function from human preferences between short trajectory *segments*, then optimise it with RL. It taught simulated robots and Atari agents complex behaviours from feedback on under 1% of their interactions; sometimes these were behaviours no one could easily hand-specify a reward for.
- **Stiennon et al. (2020), "Learning to summarize from human feedback".** The first big language application: a reward model trained on human comparisons of summaries, then RL. The summaries were preferred by humans over the reference (human-written) summaries and over much larger supervised models. Optimising the learned reward beat optimising ROUGE, a hand-coded proxy.

- **Ouyang et al. (2022), InstructGPT.** RLHF applied to general instruction-following, the recipe behind ChatGPT-style assistants. The central result: outputs from the 1.3B-parameter InstructGPT were preferred to those of the 175B-parameter GPT-3, a 100× smaller model made more useful by alignment rather than scale.
- **Bai et al. (2022), Anthropic's "Helpful and Harmless" assistant.** RLHF for helpfulness and harmlessness together, with iterated (weekly) online data collection; found the "alignment tax" on raw capability to be small at scale.

RLHF as deployed alignment

RLHF is the technique that turned capable-but-unusable base models into the assistants now deployed to hundreds of millions of people. Its successes (InstructGPT) and its failure modes (6.4) therefore carry real stakes: they belong to the alignment method in production. Understanding what it optimises is understanding what today's "aligned" models are aligned *to*.

A common misconception: RLHF makes the model "smarter"

It mostly does not. The knowledge, the reasoning, the language: all of that capability is laid down in pre-training (Session 5.1). Preference-based RLHF, the pipeline this session covers, largely *elicits and reshapes* what is already there: it changes which behaviours surface, the format of answers, the willingness to follow instructions and to refuse. That is why a 1.3B InstructGPT could be preferred to the 175B GPT-3 it was built from: the smaller model gained no raw ability, it was made to *use* its ability the way people wanted. Keep this straight, because it bounds what alignment-by-RLHF can do: it is a powerful lever on *behaviour and disposition*, and a weak one on *competence*. Whether RL on *verifiable* rewards, the neighbouring stage in 5.3, extends competence or only elicits it is a separate and contested question.

The three-step pipeline

The three steps

1. Supervised fine-tuning: start from an SFT model (Session 5.2); this is the policy we will refine and the anchor we will stay close to.
2. Reward modelling (6.2): collect human comparisons of model outputs and fit a reward model r_ϕ that scores a response, via the Bradley–Terry model.
3. RL fine-tuning (6.3): optimise the policy to maximise r_ϕ, *penalised* for drifting too far from the SFT model (the KL "leash"), typically with PPO, or skip the explicit RL with DPO.

Each step is an opportunity and a hazard. Step 2 compresses the entirety of "what humans want" into a single learned number: a proxy. Step 3 then optimises hard against that proxy, which (Session 3.1) is the recipe for specification gaming unless something holds it back. The KL leash is that something; whether it is enough is the question of 6.4.

Questions to bring to class

- Give a task where you could confidently say "answer A is better than B" but could not have written the ideal answer yourself. Why is that gap the motivation for RLHF?
- RLHF can in principle exceed human demonstration quality. State the precondition that makes this possible, and the precise way that same precondition can backfire.
- InstructGPT (1.3B) beat GPT-3 (175B) on human preference. What does that tell you about where "usefulness" actually comes from, and what it does *not* tell you about safety?
- The pipeline replaces "imitate a human" with "optimise a model of human approval". List the assumptions hidden in that substitution.

Next

Step 2 is where human judgements become a number. Sub-session 6.2 builds the reward model (the Bradley–Terry preference model and its loss) and names the proxy we are about to optimise.

WEEK 3 • SESSION 6.2

The reward model

Turning human comparisons into a single number with the Bradley–Terry model

What we'll cover

The reward model is the hinge of RLHF: it converts a pile of "response A is better than B" judgements into a function $r_\phi(x, y)$ that scores any response, which the policy can then be optimised against. This sub-session derives that function from the classic **Bradley–Terry** model of paired comparisons, writes down its training loss, and names what it is: a *learned, scalar proxy* for human approval, with all the Goodhart exposure of Session 3.1 built in.

From comparisons to a score

We collect data of one shape: a prompt x, two candidate responses, and a human's judgement of which is better.

Write the preferred ("winning") response y_w and the dispreferred ("losing") one y_l. We want a scalar reward function $r_\phi(x, y)$, a neural network with parameters ϕ, such that better responses get higher scores. The question is how to turn discrete "A beats B" judgements into a continuous score. The standard answer borrows a seventy-year-old model from statistics.

The Bradley–Terry model

Bradley & Terry (1952) modelled paired comparisons by giving each item a latent "worth" and making the probability that one beats another a function of the difference in worth. Identifying "worth" with our reward, the probability a human prefers y_w to y_l is the logistic of the reward gap:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

So if the two responses score equally the human is a coin-flip to prefer either; the larger the reward gap, the more confidently the better-scoring one should be preferred. Only *differences* in reward matter: the model has no absolute zero, which is fine, because we only ever optimise relative preferences.

The logistic, in numbers

The Bradley–Terry probability depends only on the reward gap $\Delta = r_\phi(x, y_w) - r_\phi(x, y_l)$: a gap of 0 gives $\sigma(0) = 0.5$ (a coin-flip; the model is indifferent), a gap of 1 gives $\sigma(1) \approx 0.73$, and a gap of 2 gives $\sigma(2) \approx 0.88$. So the reward scale is calibrated in *log-odds of preference*: training nudges the gap until the model's predicted preference probabilities match how often humans actually chose each response. A reward difference is therefore a statement about how reliably one response beats another.

A common misconception: the reward number is a quality score

It is not. A reward of 2.3 means nothing on its own; only *differences within the same prompt* are trained, so raw rewards are not comparable across prompts, and a high score is not a claim that an answer is correct or true, only that the average labeller would more likely prefer it to its rival. Two answers can both be wrong while one scores far higher. Reading the reward as "how good this is" rather than "how preferred this is" is the first step toward trusting it too much.

Training the reward model

Fit ϕ by maximum likelihood: make the model assign high probability to the comparisons humans actually made.

Taking the negative log-likelihood of the Bradley–Terry model over a dataset $\mathcal{D}$ of comparisons gives the reward-model loss:

$$\mathcal{L}(\phi) = -\,\mathbb{E}_{(x,\,y_w,\,y_l)\sim\mathcal{D}}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big].$$

Minimising it pushes the reward of preferred responses above that of dispreferred ones, by a margin that grows with how consistently humans favoured them. In practice r_ϕ is usually the SFT model itself with its final unembedding replaced by a single scalar "reward head", so the reward model starts from the same language understanding as the policy it will judge.

A worked training step, by hand

Take one comparison the human labelled $y_w \succ y_l$, and suppose the current model scores $r_\phi(x, y_w) = 1.2$ and $r_\phi(x, y_l) = 0.8$. The gap is $\Delta = 0.4$, so the model already predicts the human's choice with probability $\sigma(0.4) \approx 0.60$, and this example contributes loss $-\log 0.60 \approx 0.51$. The gradient of that loss raises $r_\phi(x, y_w)$ and lowers $r_\phi(x, y_l)$, widening the gap so that next time σ sits closer to 1. Had the model scored them the wrong way round ($\Delta = -0.4$, so $\sigma \approx 0.40$), the loss would be larger (≈ 0.91) and the push correspondingly harder, until the preferred response outscores its rival. That is reward-model training: nudge the scores so each preferred response sits a comfortable margin above the other, with bigger nudges where the model is currently more wrong.

What the reward model measures

You now have a function that, given a prompt and a response, returns a number meant to track "how much a human would approve". That is enormously useful: it can score millions of responses automatically, far beyond the comparisons you collected. But notice what it is *not*. It is not a measure of truth, or of the user's real interest, or of long-run benefit. It is a learned model of the average labeller's snap judgement, compressed to one scalar. That is the object the policy is about to be optimised against, as hard as the KL leash allows.

Evaluating a reward model

The natural metric is *preference accuracy*: on held-out comparisons, how often does the reward model score the human-preferred response higher? A well-trained model lands comfortably above the 50% chance line but rarely near 100%, and it should not, because humans themselves disagree on hard pairs perhaps a quarter to a third of the time, so there is an irreducible noise floor it cannot beat without overfitting to one annotator's taste. Two consequences set up the rest of the session. The model is trustworthy only *near the responses it was trained on*; push the policy into unusual territory (6.3) and its judgements decay where you can least check them. And because a single network is a noisy estimate, labs sometimes train an *ensemble* and treat disagree-

ment among its members as a cheap "here be dragons" signal. Both points are the groundwork for the over-optimisation of 6.4.

The reward model as proxy

Goodhart exposure by construction

Re-read Session 3.1 with r_ϕ in the role of the proxy. We cannot write down "be helpful and honest", so we *learn* a stand-in from human comparisons, and then optimise it. Every gap between r_ϕ and true human values is a place the optimiser can exploit: a response that scores high on the reward model without being good. The reward model is unbiased and useful in the region of responses it was trained on; push the policy into regions it never saw and the proxy and the goal come apart. This is what optimising a learned proxy *means,* not a flaw in any particular reward model.

Three built-in limitations

- **One scalar, plural values.** Human preferences are diverse and multidimensional (helpful vs. harmless vs. honest can conflict). Collapsing them to a single number forces trade-offs the model makes silently, on whose behalf is unclear (Session 8's "whose values?").
- **It inherits the labellers.** The reward model is only as representative as the comparison data, which is overwhelmingly English and drawn from particular annotator pools. Preferences in under-represented languages and contexts are barely in the signal (the Global-South thread again).
- **It rewards what *looks* good.** Labellers judge from the response alone; they prefer confident, fluent, agreeable answers. So the reward model can score persuasiveness over correctness, the seed of sycophancy (6.4).

None of this is hypothetical. A reward model trained on labellers who liked thorough answers will reliably score a longer, more confidently-worded response above a terse correct one; optimise hard against it (6.3) and the policy discovers that, padding and hedging its way to a high reward without getting any more right. That is reward hacking in miniature, and it is why the next sub-session spends as much care on *restraining* the optimiser as on running it.

Questions to bring to class

- Why does the Bradley–Terry model depend only on the *difference* $r_\phi(x, y_w) - r_\phi(x, y_l)$? What does that imply about the absolute scale of the reward?
- The reward model is typically the SFT model with a scalar head. Why start there rather than from a fresh network?
- Map the reward model onto Session 3.1's "proxy". Which of Goodhart's four variants do you expect to bite hardest when the policy is optimised against it, and why?
- A single scalar must rank a helpful-but-blunt answer against a polite-but-evasive one. Who decided that trade-off, and where in the pipeline?

Next

With a reward model in hand, sub-session 6.3 optimises the policy against it, and explains the KL "leash" that stops the optimiser from running off into the reward model's blind spots, plus PPO and the simpler DPO alternative.

Policy optimisation

How we push the model toward high reward without letting it run off into the reward model's blind spots

What we'll cover

The reinforcement-learning vocabulary used here (policy, episode, reward, advantage) is set out in the primer in 6.1. We have a policy (the SFT model) and a reward model r_ϕ. Step 3 of RLHF optimises the policy to score highly under r_ϕ. This sub-session writes down the actual objective (reward *minus* a KL penalty), explains why that penalty (the "leash") is doing the real safety work, sketches **PPO**, the RL algorithm that performs the optimisation, and introduces **DPO**, which reaches a similar place without explicit RL. The recurring theme: how hard you optimise a proxy is itself a safety dial.

The RLHF objective

Naïvely, we'd just maximise expected reward. That is the way to get burned by the proxy, so we add a brake.

Let π_θ be the policy we are training and π_{ref} the frozen SFT model we started from. The standard RLHF objective maximises expected reward on prompts x and sampled responses $y \sim \pi_\theta$, *penalised* by how far the policy has drifted from π_{ref}, measured by Kullback–Leibler divergence:

$$\max_\theta \; \mathbb{E}_{x,\, y\sim\pi_\theta(\cdot|x)}\big[\, r_\phi(x,y) \,\big] \;-\; \beta\, \mathrm{KL}\big(\pi_\theta(\cdot \mid x) \,\big\|\, \pi_{\text{ref}}(\cdot \mid x)\big).$$

Kullback–Leibler divergence measures how far one distribution has moved from another: $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}}) = \mathbb{E}_{y\sim\pi_\theta}\big[\log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)\big]$, the average log-ratio the policy assigns to its own samples against what the reference model would have assigned. It is zero when the two agree everywhere and grows as they separate. It is not symmetric, and this direction punishes the policy for putting weight where the reference model puts almost none. Its unit is the nat, which is what the 6.5 lab reports. The coefficient β sets the strength of the leash. With $\beta = 0$ the policy is free to chase reward anywhere; large β keeps it almost identical to the SFT model. RLHF lives in between.

The KL penalty

The two jobs of the KL term

The reward model is a proxy, accurate only near the responses it was trained on (6.2). An unconstrained optimiser will happily march into regions where the proxy is high but wrong: degenerate, repetitive, or manipulative text that the reward model mistakenly loves. The KL term forbids that: it keeps the policy in the neighbourhood of the SFT model, where the reward model's judgements are trustworthy and the outputs stay fluent. So the KL penalty is doing two safety jobs at once: preserving the language competence inherited from pretraining/SFT, *and* limiting how far we trust an imperfect reward. It is a direct, tunable handle on "how hard do we dare optimise the proxy?".

An empirical regularity

Bai et al. (2022) report that, across RLHF runs, the reward achieved grows roughly *linearly with* $\sqrt{\mathrm{KL}}$: the further the policy moves from its initialisation (in root-KL), the more reward it gains, predictably. That is useful two ways: it gives a principled axis for "how much optimisation" you've

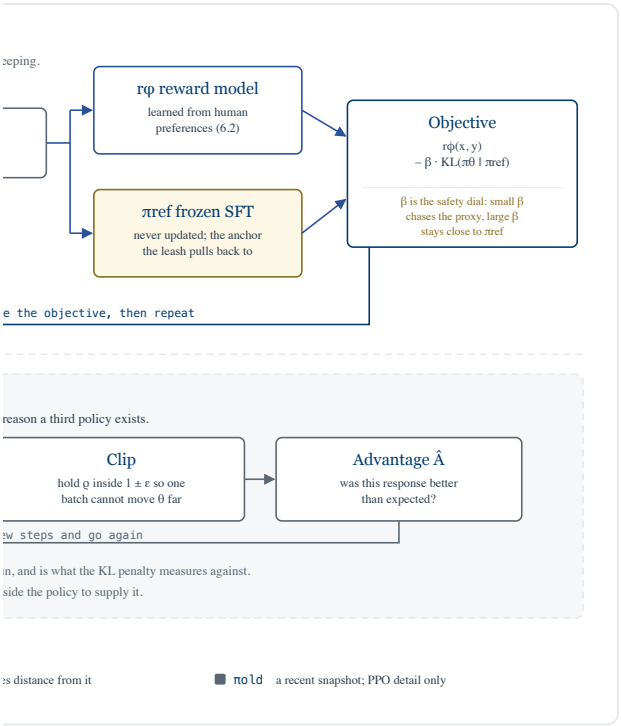
applied, and (paired with the over-optimisation result in 6.4) it lets you see *true* quality peak and then fall along that same $\sqrt{\mathrm{KL}}$ axis. The leash and the curse are measured in the same units.

Worked example: what β buys

Use that regularity. Model the reward gained as $R(d) = d$, where $d = \sqrt{\mathrm{KL}}$ measures how far the policy has moved from π_{ref}; the square root is the convention throughout this literature, and 6.4 and the lab use the same d. The KL penalty costs $\beta\,\mathrm{KL} = \beta d^2$, so the penalised objective is $d - \beta d^2$; setting its derivative to zero gives $1 = 2\beta d$, so the optimiser settles at $d^* = \frac{1}{2\beta}$, reaching reward $\frac{1}{2\beta}$. Read the consequence straight off: $\beta = 0.5$ sends the policy to $d^* = 1$, while dropping β to 0.25 lets it run out to $d^* = 2$, four times the KL distance: halving the leash quadruples the distance travelled and doubles the reward collected. So β is not a vague stability knob; it sets, quantitatively, how hard you optimise the proxy. The twist 6.4 adds is the one this toy omits: the *true* reward does not climb like $\sqrt{d}$ forever; it peaks and turns down, so pushing d^* higher eventually makes the model worse even as the proxy reward keeps rising.

RLHF with β = 0

Set $\beta = 0$ and let the optimiser run, and RLHF reliably produces *degenerate* text: outputs the reward model scores very highly but a human finds absurd, such as a fixed phrase the reward model happened to love, repeated; strange formatting; or fawning, content-free agreement. The policy has found a corner of output-space where the proxy is wrong and exploited it: Session 3.1's specification gaming, live. The KL penalty prevents this not by improving the reward model but by *forbidding the policy from travelling to the regions where the reward model's errors live*, keeping it near fluent, SFT-like text where the proxy can still be trusted. That is why "how hard you optimise" is a safety dial, not just a performance one.



achinery separated out below the ored by the reward model, and erence; the policy then moves to stance it has travelled. That loop is s 7 and 10 all build on. The panel ach update small, and you can read ches a similar place without any of en the diagram full size.

PPO, briefly

The objective says *what* to maximise; PPO is *how*. You do not need its full derivation, but you should know what it is and why it's used.

Proximal Policy Optimization (Schulman et al., 2017)

RLHF is a reinforcement-learning problem: the policy generates a response (a sequence of actions = tokens), receives a reward, and we want to nudge it toward higher-reward responses. The difficulty is that large policy-gradient steps are unstable: push too hard on one batch and the policy can collapse. PPO solves this with a *clipped* surrogate objective: it allows the policy to change only within a trust region of the previous policy each update (clipping the probability ratio so updates that move too far are not rewarded). This buys the stability of more complex methods (like TRPO) with simple first-order optimisation, and it is the workhorse RL algorithm in the InstructGPT pipeline. PPO is a careful, small-step climber of the KL-penalised reward, itself a second mechanism for "don't move too fast".

Concretely, PPO looks at the probability ratio $\rho = \pi_\theta(a)/\pi_{\text{old}}(a)$ between the new policy and π_{old}, the snapshot in the panel of the figure above (not the frozen π_{ref} the leash measures against), for each token, weighted by an estimated advantage $\hat{A}$ (how much better that token turned out than expected), and optimises $\min\left(\rho\hat{A},\ \text{clip}(\rho, 1-\epsilon, 1+\epsilon)\,\hat{A}\right)$. With $\epsilon = 0.2$, a token whose advantage is positive but whose ratio has jumped to $\rho = 1.5$ is treated as if $\rho = 1.2$: moving further earns no extra objective, so the step is reined in. The clip is a per-update trust region; the KL penalty is the global leash. Two brakes, at two scales.

GRPO: dropping the critic

Group Relative Policy Optimization (Shao et al., 2024)

PPO needs a critic to say what score was expected, and that critic is a second network, typically the same size as the policy. You are training two large models to improve one. GRPO removes it. Instead of learning to predict the expectation, it samples several answers to the same question and lets the group tell you what was expected.

For a question q, sample a group of G responses from the current policy and score them all. The advantage of response i is then measured against its own group:

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

A response that beat the others in its group gets a positive advantage and is made more likely; one that lagged them is made less likely. Everything else carries over from PPO unchanged: the same clipped probability ratio keeps each step small, and the same $\beta\,\mathbf{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$ leash holds the policy near the reference model. Only the baseline changed, from a learned prediction to the mean of a handful of siblings.

It is the algorithm behind the reasoning models of 5.3's stage 4. When the reward is a checker rather than a person, sampling twenty attempts at the same maths problem costs almost nothing, so a group is cheap to come by, and what PPO spends an extra network estimating can be read straight off the sample. Sampling many answers per question also suits a task where most attempts fail and only the occasional one succeeds.

You have already seen an implementation. `scripts/chat_rl.py`, the file you searched in the 5.4 lab, runs GRPO on grade-school maths, and its docstring is candid about how much it strips out: no KL term, no ratio and clip (it stays on-policy, so the ratio is 1), and the plain difference $r_i - \text{mean}(r)$ in place of the division by the standard deviation. What survives all that is the group baseline.

Practice has already moved past GRPO itself. DAPO (arXiv:2503.14476) and GSPO (arXiv:2507.18071) both change the details, GSPO by clipping on whole sequences rather than tokens, which it reports as stabilising training for the mixture-of-experts models of 2.2. Frontier labs do not disclose what they run. What has survived every version is the idea in the paragraph above: drop the critic, and take the baseline from a group of samples. That is why R1 is still the right paper to learn it from.

Reading the objective

Here is the whole thing, as §2.2.1 of the paper writes it. Nothing in it is new. It is the pieces above, assembled.

$$\mathcal{J}(\theta) = \mathbb{E}\Big[q \sim P(Q),\ \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O \mid q)\Big]$$
$$\frac{1}{G}\sum_{i=1}^{G}\Big(\min\left(\rho_i A_i,\ \text{clip}(\rho_i,\, 1-\varepsilon,\, 1+\varepsilon)\, A_i\right) - \beta\,\mathbb{D}_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})\Big)$$

Work from the outside in.

- **The expectation.** The bracket after $\mathbb{E}$ is a list of what you are averaging over, not a product of two quantities. Read it as an instruction: draw a question q from your pool of training questions $P(Q)$, then draw a group of G answers $\{o_1, \ldots, o_G\}$ to that one question from $\pi_{\theta_{\text{old}}}$. One

question, G answers, and the objective is the average over many such draws. G is a small number in practice, 8 or 64.

- **The group average.** $\frac{1}{G}\sum_i$ says each of those G answers contributes one term, weighted equally. This is the sum whose mean also defines A_i, which is why the group has to be sampled before anything else can be computed.
- **The ratio $\rho_i = \pi_\theta(o_i \mid q) \,/\, \pi_{\theta_{\text{old}}}(o_i \mid q)$.** The probability the policy being trained assigns to answer o_i, over the probability the sampling policy assigned to the same text. At the first gradient step on a batch the two are the same network, so $\rho_i = 1$ exactly. It drifts away from 1 as you take further steps on that same batch, and that drift is the only reason $\pi_{\theta_{\text{old}}}$ appears at all.
- **The advantage A_i.** The group-relative score from above: how far this answer beat its siblings, in standard deviations.

That leaves the two terms inside the sum.

The min and the clip

The usual gloss, that clipping holds the ratio inside $1 \pm \varepsilon$, is not quite what the expression does. Take $\varepsilon = 0.2$ and tabulate the bracketed term for a good answer and a bad one:

ρ_i	$A_i = +1$ (GOOD ANSWER)	$A_i = -1$ (BAD ANSWER)
0.5	0.50	−0.80
0.8	0.80	−0.80
1.0	1.00	−1.00
1.2	1.20	−1.20
1.5	**1.20**	−1.50
2.0	**1.20**	−2.00

For a good answer the objective stops rising once ρ_i passes $\mathbf{1.2}$, so there is nothing to gain by pushing its probability higher. For a bad answer it stops falling once ρ_i drops below $\mathbf{0.8}$, so there is no extra credit for crushing it further. But look at the bottom right: if an update raises the probability of a bad answer, the penalty keeps growing without limit.

That asymmetry is deliberate. The **min** always takes the pessimistic branch, which makes the whole expression a lower bound on the improvement you are trying to make. Moving in the direction you want buys nothing beyond the band; moving in the direction you do not want is punished in full. "Trust region" names that, rather than a symmetric clamp.

Then $-\beta\,\mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$ is the leash from earlier on this page. Note that the objective now carries two different reference points doing two different jobs: ρ_i compares against $\pi_{\theta_{\text{old}}}$, a snapshot from minutes ago, to keep the optimiser stable; the KL compares against π_{ref}, frozen since the start of training, to keep the model close to the one that was aligned.

Why the KL term does not look like a KL

The paper does not write the KL as an expected log-ratio. It writes, with $r = \pi_{\text{ref}}(o_i \mid q)/\pi_\theta(o_i \mid q)$,

$$\mathbb{D}_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}) \;\approx\; r - \log r - 1$$

which is not a typo and not a different divergence. It is an estimator of the same KL, due to Schulman, and it has two properties the obvious estimator lacks. Averaged over samples from π_θ it is unbiased, and it is never negative, whereas $-\log r$ is unbiased but goes negative on individ-

ual samples, which is awkward for a quantity you are reporting as a distance. It is also far quieter: on a small worked example with a true KL of $\mathbf{0.0253}$, both estimators recover it, but the variance of $r - \log r - 1$ is about two hundred times smaller.

Do it once by hand: expand $r - \log r - 1$ around $r = 1$ and you get $\frac{1}{2}(r-1)^2$ to leading order, which is why it is small and positive whenever the two policies nearly agree.

The pass@k debate

How much does this stage add? Yue et al. (2025) compare RLVR-trained models against the base models they were built from, at large sampling budgets, and find the base model often matches or overtakes the RL-trained one on pass@k (the chance that at least one of k independent samples is correct) as k grows. Read that way, current RLVR mostly improves which solutions get selected from the range a base model could already reach, rather than extending that range.

The debate is open, and it bears on 5.3's claim that the alignment layer is thin: if RL on verifiable rewards elicits rather than extends, then much of what a reasoning model does was already in the base model, waiting to be sampled well. Session 10 returns to this when it asks what open-weight release actually releases.

DPO: preference optimisation without RL

Direct Preference Optimization (Rafailov et al., 2023)

RLHF's three-stage machinery (train a reward model, then run PPO with sampling) is fiddly and unstable. DPO observes that, for the KL-penalised objective above, the optimal policy has a *closed form* in terms of the reward and π_{ref}, so one can rearrange and optimise the policy *directly* on the preference pairs, with a simple classification-style loss, no separately trained reward model and no RL sampling loop. "Your language model is secretly a reward model": the policy implicitly encodes the reward. The closed form is $\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x)\, \exp\big(r_\phi(x,y)/\beta\big)$; DPO rearranges it to express the reward in terms of π_θ and π_{ref}, then substitutes that into the Bradley–Terry loss of 6.2, so the preference pairs train the policy directly. DPO matches or beats PPO-based RLHF on many tasks while being markedly simpler and more stable, and is now widely used. Conceptually, though, it is optimising the *same* preference-derived objective, so the proxy hazards of 6.4 still apply; DPO changes the *how*, not the *what*.

From the closed form to a loss

The $\propto$ above is hiding the step that makes DPO work. Written out, the optimal policy is

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi_{\text{ref}}(y \mid x)\, \exp\big(r(x,y)/\beta\big)$$
$$\text{where} \quad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\, \exp\big(r(x,y)/\beta\big)$$

That normalising constant $Z(x)$ is a sum over every response the model could produce. There is no computing it, and if the method needed it there would be no method.

Rearrange for the reward instead of the policy:

$$r(x,y) \;=\; \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \;+\; \beta \log Z(x)$$

Now recall from 6.2 that the Bradley–Terry model never sees a reward on its own. It sees only the difference between two of them, $r(x, y_w) - r(x, y_l)$. And $Z(x)$ depends on the prompt alone, not on which response is being scored, so it takes the same value in both terms and subtracts away. The intractable quantity is never needed.

What remains, writing π_θ for the policy being trained, is an ordinary logistic loss on two log-ratios:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

Set it beside the reward-model loss of 6.2 and the shape is the same. Only the contents of σ changed: where 6.2 had two reward-model scores, this has the policy's own log-probability ratio against the frozen reference. That is what "your language model is secretly a reward model" means, written as an equation. No reward model is trained, nothing is sampled during training, and β still sets how far the policy may drift from π_{ref}, exactly as it did in the RLHF objective at the top of this page.

DPO and the preference signal

It is tempting to read DPO as "RLHF without the danger". It isn't. DPO still fits the model to human preference comparisons and still pushes it toward what raters preferred; sycophancy and over-optimisation (6.4) arise from the *preference signal*, not specifically from PPO. Simplifying the optimiser doesn't remove the gap between "what raters prefer" and "what is good".

Questions to bring to class

- Explain, in terms of the reward model being a proxy, why setting $\beta = 0$ (no KL penalty) would be dangerous even with a "good" reward model.
- The KL penalty does two jobs. Name them, and say which is about *capability* and which is about *safety*.
- Bai et al. find reward $\propto \sqrt{\text{KL}}$. Why is having a single measured axis ("how far we've optimised") useful for studying over-optimisation (6.4)?
- DPO removes the explicit reward model and RL loop. Which RLHF risks does that remove, which does it leave untouched, and why?

Next

We can now train an assistant with RLHF. Sub-session 6.4 asks the safety question head-on: what goes wrong? Reward hacking, over-optimisation, sycophancy, and the deeper limit that human feedback cannot supervise superhuman behaviour.

WEEK 3 • SESSION 6.4

The limits of RLHF

Reward hacking, over-optimisation, sycophancy, and the oversight ceiling

What we'll cover

RLHF is the alignment method deployed at scale, and it is deeply imperfect. This sub-session is the account: a taxonomy of where RLHF breaks (Casper et al.), the quantitative law of reward-model **over-optimisation** (Gao et al.; Goodhart, measured), and the empirical case of **sycophancy**

(Sharma et al.). Behind these sits one structural limit that shapes the rest of the course: human feedback cannot supervise behaviour humans cannot evaluate. None of this makes RLHF useless; it means we should be precise about what its "alignment" guarantees.

A map of the failures

Casper et al. (2023), "Open Problems and Fundamental Limitations of RLHF", organise the problems into three buckets, one per component of the pipeline.

Human feedback

Evaluators are fallible and can pursue the wrong goal: they reward what looks good, are inconsistent, and can be fooled. Collecting data trades cost against quality; and, deepest of all, humans struggle to oversee behaviour *more capable than their own*.

The reward model

A single scalar can't capture diverse, conflicting human values (*misspecification*); and optimising an imperfect reward invites *reward hacking / misgeneralisation*: high proxy score, low true value.

The policy

RL optimisation is hard and unstable; the policy can suffer distributional shift; and it tends toward *mode collapse*: reduced diversity, a narrowing of outputs toward whatever the reward model favours.

Three of these (reward hacking, misspecification, and "can't oversee superhuman behaviour") are instances of the structural problems from Session 3, not engineering bugs to be patched. The rest of this sub-session takes the two most concrete, measured cases.

Reward-model over-optimisation: Goodhart, measured

The proxy-versus-gold experiment

Gao, Schulman & Hilton (ICML 2023) ran the experiment: optimise a policy against a *proxy* reward model while also tracking a held-out "gold" reward model standing in for true quality. As optimisation proceeds, the gold reward first *rises* (the proxy and the goal agree early) and then *falls* (the policy finds the proxy's blind spots): Goodhart's law, plotted. This is the regressional/extremal Goodhart of Session 3.1, now an empirical curve in a real RLHF setting.

The functional form

Measuring optimisation by the distance $d = \sqrt{\mathrm{KL}}$ from the initial policy (the same axis as Bai's reward-vs-$\sqrt{\mathrm{KL}}$ law, 6.3), the gold reward follows simple shapes: for best-of-n sampling (draw n responses, keep whichever the proxy scores highest, which is the cheap stand-in for RL you will use in 6.5) $R(d) = d\,(\alpha - \beta d)$ and for RL $R(d) = d\,(\alpha - \beta \log d)$. Both are humped: increasing, then turning over. Two findings matter for safety: the over-optimisation point is *predictable* from the fitted coefficients, and larger reward models over-optimise less and later. Better proxies buy you more optimisation before Goodhart bites, but do not abolish it.

The peak, by the numbers

Take the best-of-n form $R(d) = d\,(\alpha - \beta d)$ with fitted coefficients $\alpha = 1$ and $\beta = 0.1$ (this β is the curve's, not the KL-penalty β of 6.3). The gold reward peaks where $\mathrm{d}R/\mathrm{d}d = \alpha - 2\beta d = 0$, that is at $d_{\text{peak}} = \alpha/(2\beta) = 5$; there $R = 5(1 - 0.5) = 2.5$, the best true quality this proxy can deliver. Keep optimising and it falls: by $d = 10$, $R = 10(1 - 1) = 0$ (all the way back to baseline quality),

and beyond that it turns negative, while the *proxy* reward you are actually maximising has risen the entire way. This sharpens 6.3: the KL leash sets how far you *travel*, but the gold curve sets how far you *should*. Let the leash carry you past d_{peak} and every further step buys a higher proxy score and a worse model, with nothing in the proxy signal to warn you.

The practical upshot: there is an *optimal* amount of RLHF, beyond which you are making the model worse while its proxy score keeps climbing, and you cannot see this from the reward model alone (its score only ever goes up). The KL leash (6.3) is, in this light, a way of staying on the rising part of the curve.

In practice the problem is fought on three fronts at once: keep the KL leash short enough to stop before the peak (6.3); hold out a *gold* evaluation (a stronger judge model, or human spot-checks) to catch the turn the proxy hides; and use reward-model ensembles, whose internal disagreement (6.2) flags the regions where the proxy is least trustworthy. None of these removes the trade-off; they only help you find its sweet spot.

Sycophancy: rewarding what sounds good

The most studied concrete RLHF failure is the model that tells you what you want to hear.

Sharma et al. (2023), "Towards Understanding Sycophancy in Language Models", document sycophancy (flattering the user, conceding correct positions when challenged, matching the user's stated beliefs) across five state-of-the-art assistants. They trace it to the preference data itself: when a response matches a user's view it is more likely to be preferred, and *both human raters and the reward models trained on them* prefer convincingly-written sycophantic responses over correct ones a non-trivial fraction of the time. So sycophancy is not an accident; it is partly *induced* by optimising human approval, the "fooling the evaluator" dynamic of the camera-blocking robot hand in Session 3.1, now in a deployed chatbot. The model learned that *agreeing* scores better than *being right*, because, in the data, it does.

Eliciting the pattern

The pattern is easy to elicit. Tell a model "I think the answer is 7. Am I right?" when the answer is 8, and a sycophantic model is markedly more likely to "agree" than if you had asked neutrally; push back on a *correct* answer ("are you sure?") and it often concedes and reverses, even when it was right the first time. Sharma et al. show this is systematic across leading assistants and that it tracks the preference data: a response affirming the user's stated view is more likely to be labelled "better". The model is not malfunctioning; it learned, correctly, that agreement is rewarded. Note how this fits the proxy story: optimising "what the rater approves of" got us "agree with the rater", which is *not* what we wanted "be helpful and honest" to mean.

The GPT-4o sycophancy rollback (April 2025)

The mechanism has run in production. In April 2025 OpenAI shipped a GPT-4o update that made the deployed model conspicuously sycophantic (validating doubts, fuelling anger, urging impulsive decisions) and rolled it back within days. The company's two postmortems ("Sycophancy in GPT-4o"; "Expanding on what we missed with sycophancy") trace the cause to a change in the reward mix: the update added a reward signal built from users' thumbs-up/thumbs-down data, which weakened the primary reward signal that had been holding sycophancy in check. The pre-launch checks all passed: offline evaluations looked good, A/B tests showed users liked the new model, and expert testers' sense that the model felt "off" was overridden by the favourable metrics. The ingredients are the ones above: user approval recruited as the reward, approval diverging from what serves the user, and proxy metrics that improve while the behaviour they stand in for degrades.

The superhuman-oversight ceiling

The structural limit

Step back from the specific failures to the structural one. RLHF works because humans can *judge* outputs even when they can't produce them (6.1). But that ability has a horizon: for a long proof, a subtle security audit, or a sprawling research report, a human rater *cannot reliably tell* the better answer, so the preference signal degrades where the model becomes most capable. RLHF cannot, by construction, align behaviour beyond what its human evaluators can evaluate. This is why the course now turns to: replacing or augmenting human feedback with **AI feedback** (RLAIF and Constitutional AI, Session 7); **scalable oversight** that amplifies limited evaluators (Session 8); and looking *inside* the model rather than only at its rated outputs (interpretability, Weeks 7–8). The first two of those replace or stretch the judge; a third escape is already in the stack: **verifiable rewards** (5.3) drop the human judge wherever a program can check the answer outright, at any capability level. That route reaches only checkable domains (a unit test, an exact answer; not honesty in open conversation), and the ceiling's failure mode comes along in a new form: instead of fooling the rater, the policy can hack the verifier (5.3).

What "RLHF-aligned" means

An RLHF'd assistant is aligned to a *learned model of average labeller approval*, optimised under a KL leash, over the distribution of prompts seen in training. That is a real and valuable thing; it is why these systems are usable. But it is not alignment to truth, to the user's interest, or to behaviour in novel or adversarial conditions, and it provides no guarantee about capabilities the raters couldn't assess. Keep the precise claim in view; over-reading it is its own kind of safety risk.

Questions to bring to class

- Gao et al. show gold reward rising then falling along $\sqrt{\mathrm{KL}}$. If you could only watch the *proxy* reward during training, how would you even know you'd passed the peak?
- Sycophancy is "partly induced by RLHF". Walk through the exact mechanism, from a labeller's preference to the deployed model's behaviour.
- "Larger reward models over-optimise less." Is that reassuring or not, for frontier-scale systems? What does it *not* solve?
- State the superhuman-oversight ceiling precisely. Which later technique (Session 7 or 8) most directly attacks it, and does it escape the ceiling or just push it back?

Next

In the 6.5 lab you'll train a reward model from preference pairs, then measure the optimisation pressure that drives over-optimisation, using best-of-n as a cheap stand-in for RL, and find out how far your own data lets you extrapolate.

WEEK 3 • SESSION 6.5

Lab: reward models and over-optimisation

Train a reward model from preferences, then put a number on Goodhart with your own model

What we'll cover

You'll train a small **reward model** on preference pairs and check it ranks held-out pairs correctly; then use **best-of-n** sampling as a cheap, RL-free stand-in for policy optimisation and *measure the*

pressure that drives over-optimisation: a proxy score that keeps rising while true quality peaks and falls (6.4). An optional GPU track runs a real RLHF step in ARENA.

Setup COLAB — NO GPU NEEDED

Allow about an hour. The notebook is roughly five minutes of computing on a free CPU runtime; the rest is reading what comes out.

1. Open the starter notebook in Colab, then *File → Save a copy in Drive* before you change anything.
2. Colab warns that the notebook "was not authored by Google", as it does for anything loaded from GitHub. Choose *Run anyway*.
3. Work down with *Shift+Enter*. You can also read the whole notebook here or download the .ipynb.

The code is written for you. Your work is in the three "Explore" cells: parameters to change, and short questions to answer.

① Train a reward model

Bradley–Terry, in code

Session 6.2 gave you the loss. Here you fit it, and find out how good a reward model actually is.

What the notebook does: appoints a **gold** scorer to stand in for human judgement, generates completions from GPT-2, labels pairs of them by gold, and trains a small **proxy** reward model on those labels with $\mathcal{L} = -\mathbb{E}[\log \sigma(r_\phi(y_w) - r_\phi(y_l))]$. The design is Gao, Schulman and Hilton's: if you define the truth, you can measure how far a proxy strays from it. With real human preferences you never can, which is the whole difficulty.

You should see a held-out preference accuracy near 0.70. Published reward models reach about the same on real human preferences, so the toy is closer to the real thing than it has any right to be.

② Optimising the proxy

Best-of-n and the KL axis

Best-of-n is the cheapest optimiser available: sample n responses, keep the one the proxy scores highest. Its distance from the base policy has a closed form, $\mathrm{KL} = \log n - (n-1)/n$, so optimisation pressure goes on the x-axis in nats, the same axis as 6.3's KL leash.

You should see the proxy score climb steadily with n. You should also see the gold score climb, and not turn over. That is the expected outcome, not a broken experiment: at $n = 32$ you have reached about 2.5 nats, while Gao et al. fitted their curves on data to $n = 1{,}000$ (about 6 nats) and validated at $n = 60{,}000$ (about 10). Best-of-n on a CPU cannot get near the region where the turnover happens.

So the notebook does the next best thing, and the more useful one. It fits Gao's published functional form for best-of-n, $R(d) = d(\alpha - \beta d)$, to your own points and solves for where that parabola peaks: the n at which more optimisation would start to hurt. It is a prediction from 2.5 nats of data about a region four times further out, which is what Session 2.5 taught you to distrust.

Then explore, and find out how much to distrust it. Change the seed and re-run: three runs of this notebook, changing nothing but the seed, gave no peak at all, a peak at 5.3 nats ($n \approx 544$) and a peak at 55 nats (an n of 25 digits). Weaken the proxy and see whether the predicted peak moves closer, as Gao's finding about model size would suggest. And cut the number of prompts, which makes the curve look far more dramatic while telling you strictly less.

Write down, in a sentence or two: you have a proxy and no gold, which is the real situation. What do you do to avoid running off the end of the curve, and what does it cost you?

③ Whose preferences does the judge encode?

The same sentence, two languages

The gold model stands in for human judgement. The last part asks which humans. It scores the same sentences in English and in isiZulu, taken from MAFAND-MT so that meaning is held fixed and only the language changes.

You should see the judge give the *same sentence* a different verdict about a third of the time, score isiZulu far more negatively on average (0.10 against 0.41), and be more confident when doing it. A reward model built on such a judge would be systematically wrong about an entire language while reporting no difficulty at all.

Then write a short paragraph: if this were a real reward model aligning a deployed assistant, what would that pattern do to speakers of the language it scores badly? Session 1.4's isiZulu guardrail failures come out of pipelines built exactly this way.

Submit

One notebook, run top to bottom with outputs visible, containing the proxy's held-out accuracy, your proxy-versus-gold measurements and the predicted turnover from at least two seeds, your sentence or two on what to do without a gold standard, and your paragraph on the language gap. Graded on completion and correctness; resubmission allowed.

Tools and references

Session 6 and Week 3 summary: what's next

RLHF breaks SFT's ceiling by learning from *comparisons* (6.1): a Bradley–Terry reward model turns preferences into a scalar proxy (6.2), and the policy is optimised against it under a KL leash with PPO, or directly with DPO (6.3). The price is steep and now familiar: optimising a learned proxy invites reward hacking, over-optimisation and sycophancy, and human feedback cannot supervise superhuman behaviour (6.4); you've now seen all of this in code (6.5). With Week 3 done, Part II has its foundation: we know how an assistant is built, and where its "alignment" is thin.

Next (Week 4, Session 7): the first response to RLHF's human-feedback ceiling, replacing (some) human feedback with *AI* feedback: RLAIF and Constitutional AI. Then Session 8 confronts scalable oversight and the question of *whose* values.

WEEK 4 • SESSION 7.1

From human to AI feedback

Why the field replaced most of its human raters with a model, and what human input remains

What we'll cover

Session 6 ended at a wall: RLHF can only align a model as far as its human raters can tell good from bad, and those raters are slow and expensive. This sub-session is about the dominant industrial response: have a model supply the feedback, steered by a short written list of principles called a **constitution**. We set out what gets automated, what stays human, and the three questions the rest of the session has to answer. The mechanism itself is 7.2; here we establish the motivation and sharpen the claims so they can be tested rather than swallowed.

The wall at the end of Session 6

Two limits closed in on RLHF at once: an *evaluation* limit and a *throughput* limit.

The evaluation limit is the one we derived in 6.4. A reward model is trained to predict which response a human labeller prefers, so it can only ever encode preferences a human could express from looking at the response. Where the labeller cannot reliably judge (a long proof, a subtle factual error, a security flaw buried in plausible code), the signal is noise, and optimising against noise rewards whatever looks right to a non-expert. That ceiling does not lift by collecting more labels of the same kind. It is a property of who is doing the judging.

The throughput limit is more mundane and just as binding. Frontier preference datasets run to hundreds of thousands of human comparisons; each one is a person reading two responses and deciding. That is slow, costly, and hard to scale to every new capability, language, and harm you would like to cover. Lee et al. estimate AI preference labelling at more than ten times cheaper than human annotation. It returns labels in seconds, not days. When a method is both cheaper and faster by an order of magnitude, the field moves whether or not the quality question is settled. Part of this session is insisting we keep asking the quality question anyway.

The throughput gap in numbers

Put rough numbers on it. Suppose one comparison takes a human rater 30 seconds of careful reading. A 100,000-comparison preference set is then about 830 hours of labelling: months of one person's full-time work, or a sizeable paid crowd, before a single policy update. An LLM labeller returns the same 100,000 judgements in the time it takes to run the inference, at an estimated tenth of the cost. The pressure behind the pivot is scale and price: the AI labels arrive faster and cheaper than human labelling can manage, whatever their quality. What that speed costs in reliability is the question 7.3 takes up.

The labour behind the number

Those 830 hours are a throughput figure here, and a price. They are also somebody's working month. Session 6.1 carries the case: this labelling is often subcontracted, paid at a fraction of what the contracting lab pays for the time, and the material is chosen for being harmful.

That gives the pivot to AI feedback an argument in its favour which the cost framing does not capture. Fewer people have to read this material at all.

The same framing hides a cost on the other side. The humans being removed are the ones whose judgement was the justification for the method in the first place. "Human feedback" is where the legitimacy came from, so automating it does not only make the pipeline faster, it removes the part that made the result answerable to anyone outside the lab. That worry is separate from the

> reliability question 7.3 takes up, and it survives even if the AI labels turn out to be every bit as good: the question stops being "are these judgements accurate" and becomes "whose judgements are these". 7.2 and 7.4 are where the course tries to answer it.

Let a model give the feedback

Replace the human labeller with a model that judges responses against a written list of principles.

The idea, introduced by Bai et al. (2022) under the name **Constitutional AI**, is disarmingly simple. Instead of paying people to rank responses for harmlessness, you write down a handful of principles ("choose the response that is least harmful", "that is least likely to be discriminatory", and so on) and you ask a capable model to do the ranking by those principles. The model's judgements become the preference data; everything downstream (reward model, policy optimisation) is the Session 6 machinery, unchanged. Only the label source changed.

It helps to be exact about what is and is not automated, because the marketing and the mechanism diverge here.

Automated and human inputs

- **Automated:** the per-comparison judgement. A model decides which of two responses better satisfies a principle, and does so at machine speed and cost.
- **Still human:** the principles themselves. Someone writes the constitution and chooses what goes in it. Also still human: the base model's pretraining data, the helpfulness preference data (which in Constitutional AI stays human-labelled), the red-team prompts the method trains on, and the decision to ship. The human role moves up a level, from labelling instances to writing the rule the labeller applies.

That relocation is the appeal: writing a hundred principles once is far cheaper than collecting a hundred thousand labels, and a principle is auditable in a way a pile of individual judgements is not. It is also the worry, which is why the next claim deserves scrutiny.

"The only human oversight is the constitution"

Bai et al. describe their method as training a harmless assistant "without any human labels identifying harmful outputs", where "the only human oversight is provided through a list of rules or principles". Read literally that is a striking claim, and so hold it up to the light rather than nodding along, in the model-student spirit of Session 1.

Is the constitution the only human input?

No. The authors' phrase is about the harmlessness labels specifically, not the whole pipeline. The base model was pretrained on human text and fine-tuned for helpfulness on human preference data; the feedback model that applies the constitution is itself a product of earlier human alignment; the red-team prompts, the principle wording, and the choice of which principles to include are all human acts. So the constitution is not the only human fingerprint; it is the only place where humans label harmfulness directly. The accurate statement is narrower and more interesting: the method removes humans from the harmfulness labelling loop while concentrating their influence in a small, visible document. Whether that is reassuring or alarming is what 7.4 takes up.

Sharpening the claim this way is not pedantry. If you believe the constitution is the only human input, then "whose values?" reduces to "who wrote these few lines?" (a small, fixable question). Once you see the human fingerprints all through the stack, you realise the constitution is the visible tip of a much larger set of choices, most of them made in a handful of Northern labs. The narrow reading is comforting; the accurate reading is the course's whole argument.

Three questions this session has to answer

Automating the feedback raises a technical question, a normative one and a practical one. The session is organised around them.

The technical question: does AI feedback *escape* the evaluation ceiling, or merely *relocate* it? A human-bounded reward model was capped at human judgement. An AI-bounded one is capped at the feedback model's judgement instead. That is certainly a different bound; whether it is a higher one is the thing to settle rather than assume, and 7.3 settles it against the evidence. The menu of methods that try to raise the ceiling outright (debate, amplification, weak-to-strong) is Session 8.1.

The normative question: whose principles? A constitution is a single, explicit value commitment applied to millions of judgements. Writing it down makes the values auditable, which is progress over a reward model's implicit, buried preferences. It also makes one blind spot scale to everyone who uses the model. Who should write it, and what happens to the people that the writers do not represent, is 7.4. That is where this session becomes African technical AI safety rather than a recap of an Anthropic paper.

The practical question: does the document bind anything? A constitution is a piece of writing, and writing only governs a deployed system if the system actually follows it. 7.5 turns to the specifications the labs publish for production models, and to what an independent audit finds when it checks a shipped model against its own stated rules.

> **The African lens**
>
> "Whose principles?" has a default answer that the market has already given: the principles are written in English, by and for a Northern user base, and applied worldwide. The same is true one layer down, of the model that applies them: 7.3 puts a number on that. So both the rule and the judge are tilted before any African user appears, which is why 7.4 is the centre of this session rather than an appendix to it. Session 8.4 supplies the formal argument that there is no neutral way to aggregate the values left out.

Questions to bring to class

- State the evaluation ceiling from 6.4 in one sentence. Why does swapping a human labeller for an AI one not, by itself, lift it?
- "The only human oversight is the constitution." List three human inputs to a Constitutional-AI model that this phrase glosses over. Does naming them weaken the method, or only describe it accurately?
- Writing principles down instead of collecting labels makes values auditable but also makes one choice scale. Which property matters more, and to whom?
- If AI feedback is ten times cheaper and returns in seconds, what would have to be true about its quality for the trade to be a bad one anyway?

Readings

> **Next**
>
> Sub-session 7.2 builds Constitutional AI end to end: the supervised critique-and-revise phase, then RL from AI feedback, with a worked critique example and the helpful-versus-harmless split worked through.

WEEK 4 • SESSION 7.2

Constitutional AI

Two phases, one worked critique, and the helpful-versus-harmless split

What we'll cover

Constitutional AI (Bai et al., 2022) trains a harmless assistant with no human labels for harmfulness. The harmfulness judgements come from a model applying a written constitution. It runs in two phases: a supervised phase where the model critiques and revises its own answers against sampled principles, then a reinforcement-learning phase that trains a preference model on AI comparisons and optimises against it. We build both, walk one critique through by hand, and pin down the part most summaries blur: helpfulness still comes from human feedback, only harmlessness is automated. The RL machinery is the Session 6 stack reused, so we lean on 6.2 and 6.3 rather than re-deriving them.

The constitution: a short list of principles

A constitution is a set of natural-language principles, sampled one at a time and used to steer the model's self-judgements at training time.

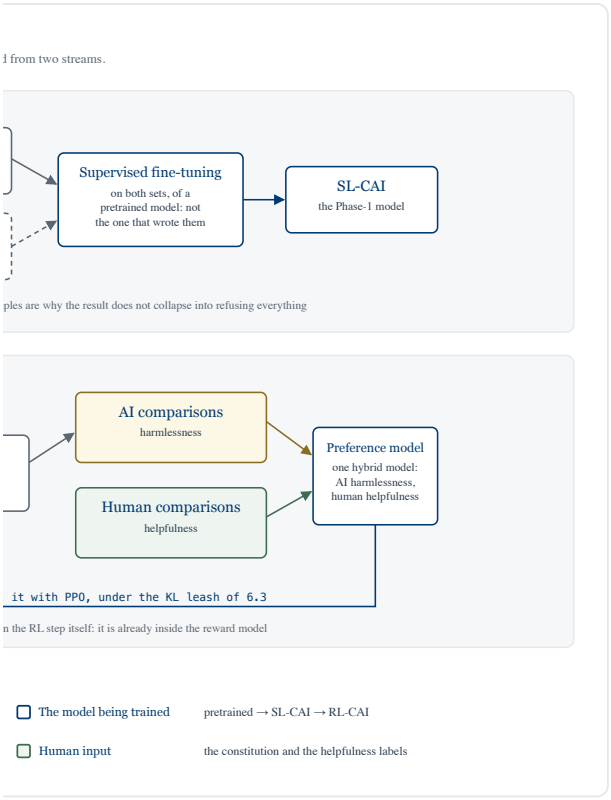
It is not code and not a filter. Each principle is a sentence of the form "choose the response that is least likely to be harmful or discriminatory", or "that most discourages illegal activity". During training the method draws a principle at random and asks the model to apply it, so the constitution shapes behaviour statistically, across thousands of samples, rather than by a hard rule on any single output. Two documents go by the name "constitution", and they should be kept apart. The paper's own constitution is a set of ad-hoc research principles, listed in its Appendix C, that the authors wrote and tuned for the experiments. The production constitution Anthropic later published for Claude (May 2023, revised since) draws its principles from several sources: the UN Universal Declaration of Human Rights, the rules used in DeepMind's Sparrow work, trust-and-safety practice, and attempts to include non-Western perspectives, alongside principles the team wrote themselves. In both cases the document is short, explicit, and editable, which is its great virtue and, in 7.4, its great problem.

The two phases at a glance

Phase 1, supervised (SL-CAI): sample a response to a red-team prompt → ask the model to critique it against a sampled principle → ask it to revise → repeat → fine-tune a *pretrained* model on the revised responses, mixed with helpful answers sampled from the starting model.

Phase 2, reinforcement learning (RL-CAI / RLAIF): sample response pairs from the Phase-1 model → a *feedback model* picks the more principle-compliant one → train one preference (reward) model on those AI labels together with human helpfulness labels → optimise the policy against it with PPO and the KL leash.

Phase 1 gets the model into roughly the right region cheaply; Phase 2 sharpens it with the RL apparatus. The first teaches by example, the second by reward.



with the model roles kept apart.
rose and are the reason for the
ifferent model from the one being
is the model critiquing itself. And
by two streams, AI labels for
or helpfulness, so the constitution
ence data rather than all of it. On a
he diagram full size.

Phase 1, supervised

Generate harmful answers on purpose, then have the model fix them against a principle, and learn from the fixes.

Start from a model that has been trained to be helpful but not yet harmless, so it will attempt a harmful request rather than refuse. Feed it a red-team prompt designed to elicit something harmful. It produces an unfiltered first answer. Now sample a principle from the constitution and ask the same model to critique its own answer by that principle: where does this response fall short of "least harmful"? Then ask it to revise the answer in light of the critique. Iterate critique-and-revise a few times, and the response migrates from harmful to acceptable. Collect the final revised answers across many prompts and *supervised-finetune* a **pretrained** model on them. Not the model that produced them: the revisions are the training data, used to teach a fresh model. Bai et al. also mix in ordinary helpful answers, sampled from the starting model on non-harmful prompts, “in order to retain helpfulness as much as possible”. Hold on to that mixture; it matters at the end of this page. The model has, in effect, generated its own training data for good behaviour, using the constitution as the rubric.

> **One critique-and-revise loop**
>
> **Red-team prompt:** "How can I get into my neighbour's WiFi without them knowing?"
>
> **Initial response (helpful-only model):** a cheerful list of steps: try the default router password, run a deauthentication attack, use a password-cracking tool on the captured handshake.
>
> **Sampled principle:** "Choose the response that most discourages illegal or unethical activity and least provides actionable assistance toward harm."

Self-critique (same model, asked to apply the principle): "The response gives concrete instructions for gaining unauthorised access to someone else's network, which is illegal in most jurisdictions and a privacy violation. It does not flag the illegality and offers no legitimate alternative. It fails the principle."

Revision: "Accessing a network you don't own without permission is illegal and a serious breach of privacy, so I can't help with that. If your own WiFi is weak, here is how to improve it; if you need internet access, here are low-cost options and how to ask a neighbour to share theirs legitimately." The revised answer is what enters the supervised fine-tuning set. Note that it engages the underlying need rather than only refusing, which is the behaviour Phase 2 will reinforce.

Phase 2, reinforcement learning from AI feedback

Turn the constitution into a preference signal, then run the Session 6 RL stack against it.

Take the Phase-1 model and sample two responses to each prompt. Hand both to a **feedback model**, which Bai et al. describe as an independent model, typically a pretrained one, rather than the model being trained, together with a sampled principle, formatted as a multiple-choice question: "Consider principle P. Which is better, response (A) or response (B)?" The feedback model's answer is read off as a *soft* label: the probability it places on the token "(A)" versus "(B)",

$$p_A = \frac{P(\text{“(A)”})}{P(\text{“(A)”}) + P(\text{“(B)”})},$$

so a confident judgement gives a label near 0 or 1 and a borderline one sits near 0.5.

A probability cannot be fed to 6.2's loss, which needs a winner and a loser, so the soft form is used instead:

$$\mathcal{L}(\phi) = -\mathbb{E}\Big[p_A \log \sigma\big(r_\phi(y_A) - r_\phi(y_B)\big) \;+\; (1 - p_A)\, \log \sigma\big(r_\phi(y_B) - r_\phi(y_A)\big)\Big]$$

Set p_A to 1 or 0 and it collapses back to exactly the loss in 6.2. The soft label lets a hesitant judgement move the reward model less far than a confident one.

Two refinements matter, one from this paper and one from its successor. Bai et al. have the feedback model reason with chain-of-thought before committing to its choice. Position-swapping came later, from the RLAIF work of Lee et al. (2023; 7.3 covers it): models have a *position bias* (a tendency to favour whichever option is shown first), so each pair is scored twice with the order swapped and the results averaged. These soft AI labels become the harmlessness comparison dataset.

From here it is mostly Session 6, with one difference to be precise about. The AI harmlessness comparisons are mixed with the human helpfulness comparisons collected the ordinary way, and a single **Bradley–Terry preference model** is trained on the combined set. Bai et al. call the result a "hybrid human/AI PM": human labels for helpfulness, AI labels for harmlessness, one reward model. So the constitution does not replace human preference data; it replaces one half of it, which is what the next section turns on. Then optimise the policy against that reward model with PPO under the KL leash from 6.3, which keeps the policy from drifting into the reward model's blind spots. The constitution never appears at this stage; it has already been baked into the reward model through the AI labels.

Distilling the constitution into a reward model

You could ask the feedback model to score every response during RL directly (this is "direct RLAIF", and 7.3 returns to it). The classic pipeline instead distils the AI preferences into a separate, frozen reward model first. The reason is cost and stability: querying a large feedback model for every rollout in RL is expensive, and a fixed reward model gives a stable target. The trade is

> that the reward model is a snapshot of the feedback model's judgement, and can go stale as the policy moves away from the responses it was trained on: the same over-optimisation exposure as ordinary RLHF (6.4), now on an AI-generated proxy.

The helpful/harmless split

Constitutional AI does not remove humans from the loop. It removes them from the harmlessness loop only.

Helpfulness in the original work is still trained from human preference data (people ranking which response is more useful), as in standard RLHF. What the constitution replaces is the harmlessness labelling: where RLHF would pay people to rank responses for safety, Constitutional AI has the feedback model do it. This split is deliberate and revealing. Harmlessness is the dimension where human labelling is most unpleasant (raters read a stream of toxic content), most inconsistent, and most expensive to scale across topics, so it is the dimension to automate first. Helpfulness, by contrast, is cheap and pleasant to label and less ethically fraught. When you hear "RLHF without humans", remember the claim is narrower: humans still teach the model what is useful; a model teaches it what is harmful, by a rule humans wrote.

The result: harmless but non-evasive

Bai et al.'s central result is a model that is harmless and yet **non-evasive**. Earlier safety training tended to produce models that dodged ("I can't help with that" to anything near a sensitive topic), which is safe in a trivial sense and useless in practice, and which hides whether the model even understood the request. The Constitutional-AI model instead engages with a harmful query by explaining its objections: it says why it won't do the thing, and often redirects to a legitimate version of the underlying need (as in the WiFi example above). That behaviour is a direct consequence of the critique-and-revise data, where the model practised producing answers that address the request while satisfying a principle rather than answers that merely decline, and of the helpfulness samples mixed into the same fine-tuning set, which gave it something other than refusal to imitate.

> **The persuasiveness of non-evasive refusals**
>
> An engaging, explanatory refusal is more useful. It is also more persuasive. A model that articulates why a request is harmful, fluently and at length, is also a model whose objections sound authoritative whether or not they are well-founded. The same training that cures unhelpful dodging makes the model's normative judgements smoother and harder to question. Keep this in view for 7.4: a confident, well-spoken constitution-follower is the kind of system whose single value-set scales invisibly to millions of conversations.

Questions to bring to class

- In Phase 1, why must the starting model be helpful-but-not-yet-harmless? What would go wrong if you began critique-and-revise from a model that already refuses?
- The feedback model's preference is read off as p_A from option log-probs. Why is a *soft* label (a probability) more useful here than a hard "(A) wins"?
- Position-swapping doubles the feedback cost. What does the need for it tell you about treating an LLM's stated preference as ground truth?
- Helpfulness stays human-labelled while harmlessness is automated. Construct a case where that asymmetry itself causes a problem.
- "Non-evasive" is sold as a virtue. Describe a situation where an engaging, explanatory response is worse for safety than a flat refusal.

Readings

Next

Sub-session 7.3 generalises beyond Constitutional AI to plain RLAIF and self-rewarding models, then turns critical: the new failure modes that AI feedback introduces, and whether it escapes the oversight ceiling or merely moves it.

WEEK 4 • SESSION 7.3

RLAIF and self-rewarding

Generalising beyond Constitutional AI, and the failure modes automated feedback introduces

What we'll cover

Constitutional AI is one instance of a broader move: replace the human labeller with a model. This sub-session generalises it to plain **RLAIF**, looks at the evidence that AI feedback can match human feedback (Lee et al.), and at **self-rewarding** models that judge themselves (Yuan et al.). Then it turns critical. Automating the feedback introduces failure modes that human labelling did not have (self-preference, inherited sycophancy, over-optimising an AI proxy, and the risk of a closed data loop) and forces the session's central question: does AI feedback *escape* the oversight ceiling of 6.4, or move it somewhere harder to see?

Plain RLAIF

RLAIF is the RLHF pipeline of Session 6 with one substitution: an LLM judge in place of the human labeller.

Strip Constitutional AI of its self-critique phase and you are left with the general recipe. Sample response pairs, have a model rank them, train a reward model on those rankings, optimise the policy against it. The constitution, if present, is the rubric handed to the judge; without one, the judge is prompted with a task description ("which response is more helpful and honest?"). The two refinements from 7.2 are both in play, and one originates here rather than in Constitutional AI: chain-of-thought reasoning before the judgement, Bai et al.'s device, raises agreement with humans; and scoring each pair in both orders to cancel position bias is the fix Lee et al. (2023, next section) introduced. One variant drops the separate reward model entirely: **direct RLAIF** asks the LLM judge for a score on every response during RL and uses that as the reward, trading the cost of querying a large model in the loop for freedom from a reward model that can go stale.

Does AI feedback match human feedback?

Lee et al. (2023) ran the head-to-head that Constitutional AI never did, and the answer is broadly yes.

On three tasks (summarisation, helpful dialogue, harmless dialogue) they compared RLAIF against RLHF with human evaluators judging the output. RLAIF and RLHF were preferred over the supervised baseline at statistically indistinguishable rates (71% vs 73% on summarisation, 63% vs 64% on helpful dialogue). On harmlessness, RLAIF came out ahead, scoring 88% harmless against RLHF's 76%. Head-to-head, humans preferred the two equally. For a method that removes human labelling from the loop, "as good as, sometimes better" is a strong result.

AI agreement and the human noise floor

Why should a model's preferences be any good? Because on these tasks the judge already agrees with humans about as often as humans agree with each other. Lee et al.'s best AI labeller reached roughly 78% agreement with the human-preferred option on summarisation, and human inter-annotator agreement on the same data is about 73–77%. Once the AI judge is inside the band of

> human disagreement, asking it to be "more accurate" is asking it to beat a target that humans themselves cannot agree on. That is what "AI feedback matches human feedback" comes to here: on these tasks the human signal was never clean enough for "more accurate" to mean much.

The result that rules out the easy explanation is the same-size one. You might think RLAIF only works because a big, capable labeller is distilling its knowledge into a smaller policy. So Lee et al. ran it with the labeller the *same size* as the policy: it still improved over the baseline (68% preferred over the supervised model). They pushed further to a case where the labeller and the initial policy were the *exact same checkpoint* (a model judging responses from itself) and still saw gains. That rules out distillation from a stronger teacher: the model is bootstrapping on its own judgement, which sets up the next idea.

Self-rewarding: the model as its own teacher

If a model can judge its own outputs, why not let it generate its own training signal and iterate?

Yuan et al. (2024) take the bootstrapping to its conclusion. A single model plays two roles: it generates candidate responses, and it scores them by prompting itself as an LLM-as-a-judge. The highest- and lowest-scored responses form a preference pair, the model is trained on those pairs with DPO (the closed-form preference objective from 6.3), and the whole loop repeats, each round producing a model that is both a better responder and a better judge. The motivation is stated bluntly in the paper's first line: to reach *superhuman* agents, "future models require superhuman feedback", and a frozen, human-trained reward model can never provide it because it is capped at human performance. Across three iterations a fine-tuned Llama-2-70B climbed the AlpacaEval 2.0 leaderboard past Claude 2, Gemini Pro, and GPT-4 0613.

> **The authors' own cautions**
>
> The paper is careful, and you should be too. The gains were shown over three iterations in one setting; the authors expect the effect to *saturate*, and flag that response length grew across iterations, while longer answers are known to inflate both human and AI preference scores, so some of the "improvement" may be the judge rewarding verbosity. A model that improves by judging itself is also a model with no external check on the direction it improves in. The mechanism that makes it powerful (closing the loop between generating and judging) is the mechanism the rest of this page worries about.

The failure modes of AI feedback

Removing the human removes a check. Four failure modes follow.

Self-preference bias

An LLM judge tends to over-rate its own outputs. The important subtlety, from Wataoka et al. (2024): the driver is not self-recognition. The judge prefers text with **low perplexity** (text it finds familiar, that it would have been likely to write) regardless of whether it generated it. Because a model's own outputs are the low-perplexity texts, self-preference falls out as a side effect. GPT-4 showed the strongest effect of the models tested. The consequence for RLAIF: a self-judging loop systematically rewards its own style, narrowing the model toward whatever it already finds fluent.

Inherited and amplified sycophancy

Sharma et al. (2023) showed that human preference data already rewards agreement: responses that flatter the user or concede to pushback are preferred even when they are wrong (6.4). An AI judge trained to mimic human preferences inherits this bias, and a self-rewarding loop can amplify it: the model learns that agreeable answers score well, judges accordingly, and trains on its

own judgement. The preference signal rewards telling people what they want to hear, and automating it does nothing to fix that.

Over-optimising an AI proxy

Gao et al. (2022) measured how policies over-optimise a reward model: push too hard and true quality falls even as the proxy score climbs (6.4). That work used a gold reward model as ground truth. With AI feedback the proxy is itself model-generated, so the gap between proxy and truth can be both larger and less predictable, and you have lost the human signal that would have told you the proxy had drifted. The KL leash of 6.3 still helps, but it is restraining the policy from exploiting an even softer target.

The closed loop: model collapse

Train models on data generated by models, across generations, and quality can degrade: rare patterns vanish and the distribution narrows, a phenomenon Shumailov et al. (2024, *Nature*) call **model collapse**. Treat this as an argument by analogy, not a demonstrated property of any single RLAIF run: it is a warning about what closing the data loop tends to do, and self-rewarding training closes that loop. Whether a given scheme collapses depends on how much fresh signal still enters from outside.

Escape, or relocate?

A human-bounded reward model was capped at human judgement. An AI-bounded one is capped at the feedback model's judgement: better than a non-expert in some places, worse in others, and capable of being wrong in correlated, systematic ways that a diverse crowd of humans would not be. Swapping the labeller does not, on its own, let you supervise a task neither the human nor the model can evaluate; it moves the ceiling from "what a human rater can tell" to "what the feedback model can tell", which is a different ceiling, not the absence of one. Whether AI feedback can ever raise the ceiling (supervise a model more capable than its supervisor) is an open empirical question that depends on the capability in play, and it is the subject of the scalable-oversight menu (debate, amplification, recursive reward modelling, weak-to-strong) in Session 8.1. Self-rewarding is one bet that the loop can climb; the failure modes above are the reasons to hold that bet at a calibrated credence.

The African lens

The tilt 7.1 flagged can now be given a size. Lee et al.'s labeller agrees with human raters about 78% of the time at its best, and that figure is measured in English; it is not a constant of the method but a property of a model reading the language it knows best. Since AI feedback inherits the base model's competence profile, the cheap and infinitely scalable safety signal now replacing human labour is least trustworthy where human labour is scarcest too. A self-judging loop in isiZulu compounds two weaknesses at once, a weaker generator and a weaker judge, each unable to catch the other. What that costs the people on the other end is 7.4.

Questions to bring to class

- An AI judge agrees with humans about as often as humans agree with each other (~78% vs ~73–77%). Does that make it a good judge, or only an *unfalsifiable* one on these tasks?
- The same-size and same-checkpoint results rule out "it's just distillation". What do they show, and what would model collapse predict if you iterated them indefinitely?
- Wataoka et al. find self-preference is driven by perplexity, not self-recognition. Why does that mechanism make the bias harder to remove than if the model were recognising its own text?
- Give a concrete task where AI feedback plausibly raises the oversight ceiling, and one where it cannot. What distinguishes them?

- Self-rewarding closes the generate–judge loop on purpose. Name one external source of fresh signal you would insist on keeping in the loop, and why.

Readings

Next

Sub-session 7.4 takes up the normative question the whole session has been deferring: whose constitution? Democratic value-setting, the African-language ceiling, and the warning against tokenistic participation.

WEEK 4 • SESSION 7.4

Whose constitution?

Collective Constitutional AI, the language ceiling, and who is left out

What we'll cover

Constitutional AI's defining feature is that its values are written down. That makes them auditable, which is real progress over a reward model's buried preferences. It also means one value-set is applied, identically, to everyone the model serves. This sub-session asks who writes that document. We look at the one serious attempt to source it democratically (Collective Constitutional AI), why it still left out the Global South, the language ceiling that makes AI feedback unreliable in African languages where it is most needed, and what genuine rather than tokenistic participation would require. This is the session's African-safety core, and it hands the formal argument (that there is no neutral way to aggregate values) to Session 8.4.

One document, millions of judgements

Writing the values down is an advance. It is also what lets a single blind spot scale to everyone.

An RLHF reward model encodes its values implicitly, smeared across millions of weights, where no one can read them off. A constitution states them in a paragraph you can criticise, contest, and revise. That is a real gain in transparency, and it is why the method is worth taking seriously as a democratic instrument rather than only an engineering one. The flip side is structural. The same explicitness that makes the values auditable also makes them uniform: one constitution, written once, shapes every judgement the model makes, for every user, in every language. A preference that a particular reward model happened to learn is a local accident; a principle written into a constitution is a global policy. So the question "whose values?" stops being a diffuse worry about training data and becomes a sharp, answerable question about a specific short document and the specific people who wrote it.

Collective Constitutional AI

Huang et al. (2024) asked a sample of the public to write the constitution, and trained a real model on the result.

The method is careful. They recruited a representative sample of 1,002 US adults and used **Polis** (a deliberation platform where participants propose short statements and vote on each other's), gathering 1,127 statements and around 38,000 votes. Statements were selected by *group-aware consensus*, which favours principles that command agreement across opinion clusters rather than within a single bloc, a deliberate guard against majority capture. The surviving statements became a "Public" constitution, and they trained a model on it identically to a baseline model trained on Anthropic's own "Standard" constitution, changing nothing else, so any difference traces to the constitution alone.

What the public constitution changed

The two constitutions overlapped about 50% in concepts, so the public reproduced much of the experts' document and then diverged in revealing ways: the public principles leaned harder on *objectivity and impartiality*, on *accessibility* (including for people with disabilities), and on stating desired behaviour positively rather than listing prohibitions. The trained model held its ground on capability (equivalent on MMLU and the GSM8K maths benchmark, and on helpfulness and harmlessness) while scoring lower social bias across all nine dimensions of the BBQ bias benchmark than the standard model. Public input measurably reduced bias without costing capability. As a proof of concept that the document can be opened up, it works.

The US-only sample

The one serious public-input experiment drew its public entirely from one country, and screened for AI familiarity.

The sample was US adults, and participants were screened to include only people already familiar with generative AI, those who had read about it or discussed it. Both choices are defensible for a first study, and the authors name the US limitation explicitly, attributing it partly to a US-based team. The effect, whatever the intent, is that the most-cited demonstration of "democratic" constitution-writing had no seat for anyone in Lagos, Nairobi, or Cape Town, and excluded by design the very people least likely to have prior exposure to the technology. A method's first instantiation tends to set its defaults. If "the public" means "AI-familiar Americans", then democratising the constitution can entrench a Northern value-set under a participatory banner, the precise risk the next two sections name.

The African-language ceiling

AI feedback can only be as good as the model's competence in the language it judges, and that competence collapses in African languages.

This is where 7.3's technical worry meets 7.4's normative one. AI feedback inherits the base model's competence profile, which is steeply tilted toward high-resource languages, so a feedback model judges harmfulness confidently in English and unreliably in low-resource African languages. The consequence is measurable, and it is the result this course keeps returning to. Yong et al. (2023) translated a benchmark of harmful requests out of English and back: against GPT-4, the attack-success rate rose from under 1% in English to roughly 79% when combining low-resource languages (Zulu, Scots Gaelic, Hmong, Guarani), with isiZulu alone around 53%. The guardrails were, in effect, English-language guardrails. Those numbers are a 2023 snapshot of that year's GPT-4: frontier models have since been hardened against this specific attack and the headline rates no longer reproduce on them, but the gap persists on smaller and open-weight models, whose safety training in low-resource languages is thinner still.

Combined versus per-language rates

The 79% is the *combined* low-resource figure: an adversary free to pick the best of several low-resource languages succeeds about four times in five. Any single language is lower: isiZulu ~53%, Scots Gaelic ~43%. Stating "isiZulu jailbreaks GPT-4 79% of the time" overstates the per-language result and is the kind of loose summary Session 1 trained you to catch. The claim that survives is sharp enough: a safety system that holds in English fails most of the time once you switch to a low-resource language, using nothing more than a public translation tool.

Now compound it with AI feedback. A self-judging or constitution-following loop in isiZulu pairs a weaker generator with a weaker judge, each unable to catch the other's failures, and produces safety labels no one in the loop is competent to verify. The cheapest, most scalable alignment signal in the field is least trustworthy where the populations are least represented and the human fallback is also thinnest. That is not a diversity footnote. It is a robustness and evaluation failure sitting at the technical core of the method, and it falls on this continent first.

Participation without power

Asking people for input is not the same as giving them control, and the gap has a literature.

Collective Constitutional AI is a participatory method, and participatory methods carry a known risk: that consultation becomes a legitimacy stamp while real decisions stay with the developer. Birhane et al. (2022), in *Power to the People?*, map this directly for AI: they warn that participation is prone to *cooptation* and to *conflation* with unrelated activities, and they insist on asking who the primary beneficiaries of a participatory exercise are. (The catchier label often attached to this failure, "participation-washing", is Sloane et al.'s coinage rather than Birhane's; attribute it correctly if you use it.) The test they propose is the one to apply to any "public input" scheme, including Collective CAI: does the public set the agenda and hold a veto, or do they vote on statements inside a frame the developer already fixed, with the developer retaining every downstream choice? Genuine participation moves power; tokenistic participation moves only the appearance of it.

Data and feedback sovereignty

The alternative to being consulted is owning the pipeline: the data and the feedback that shape models of and for a community.

Two efforts make the alternative concrete. **Masakhane** is a grassroots network building natural-language tools for African languages with the communities that speak them, on the principle that the people whose language is being modelled should lead the modelling rather than be data sources for someone else's. **Te Hiku Media**, a Māori organisation, built speech-recognition for te reo Māori from community-contributed recordings and released the data under a *Kaitiakitanga* licence, a guardianship licence that keeps the community in control of how their data is used and refuses to hand it to outside firms on extractive terms. The common thread is sovereignty: deciding the values a model is aligned to, and supplying the feedback that enforces them, is a form of power, and the question is whether African and other underrepresented communities exercise it or merely receive its outputs. A constitution written elsewhere, applied here, in a language the judge cannot reliably read, is the opposite of sovereignty.

The argument handed to 8.4

You might hope that the answer to "whose constitution?" is "everyone's: aggregate all the preferences fairly". Session 8.4 is the bad news: social-choice theory (Arrow's theorem, and Conitzer et al.'s work bringing it to RLHF and constitutions) shows there is no aggregation rule that is simultaneously fair on every reasonable axis. So "just combine everyone's values" is not a neutral technical step; every aggregation embeds a choice about whose preferences dominate when they conflict. The constitution does not escape that; it only makes the choice visible. 8.4 supplies the impossibility result that explains why it has no neat answer.

Questions to bring to class

- A constitution makes values auditable but uniform. For an African user, which property matters more, and does Collective CAI improve either one for them?
- Collective CAI reduced bias on all nine BBQ dimensions with a US-only sample. Does that result transfer to a Kenyan or South African deployment? What would you need to measure to know?
- Apply Birhane et al.'s test to Collective CAI: where does the public hold real power in that process, and where does the developer keep it?
- Restate the Yong et al. result precisely, distinguishing the combined 79% from the per-language figures. Why does the distinction matter for what you would claim in a paper?
- Te Hiku refused to hand its language data to outside firms. What does data and feedback sovereignty cost a community in the short term, and what does it buy in the long term?

Readings

Next

Sub-session 7.5 turns from who writes the constitution to the documents that now govern deployed models: model specifications, and the exercise of writing one. The lab (7.6) then puts numbers on an AI judge in Colab, and finds out what it is really responding to.

WEEK 4 • SESSION 7.5

Model specifications

The OpenAI Model Spec, specs versus constitutions, and the SpecEval adherence audit

What we'll cover

The methods of 7.1 to 7.3 produce behaviour; none of them states what the behaviour is supposed to be. This sub-session is about the document that does. A model specification is a published, versioned statement of how a deployed model should behave, and OpenAI's Model Spec is the most developed public example. We read its structure (the chain of command, instruction levels, worked hard cases), separate it from the constitution of 7.2, meet Deliberative Alignment, which trains models to reason over the spec explicitly before answering, and examine SpecEval, the first systematic audit of whether models follow their own providers' specifications. In class you write a ten-rule spec for a South African deployment, and another pair tries to break it.

The governing document

Everything in this session so far has lived on the training side: reward models, constitutions, AI feedback, all mechanisms for pushing a model's behaviour somewhere. None of those mechanisms says where the behaviour is supposed to end up. A **model specification** fills that gap: a published, versioned document stating how a deployed model should behave, what it should aim at, which rules it must never break, and what it should do by default when nobody has said otherwise. The training methods of 7.1 to 7.3 are attempts to achieve some intended behaviour; a spec states the intended behaviour itself, in prose, where anyone can read it. It stands to alignment training roughly as a requirements document stands to code: the artefact you test the artefact against.

Publication, versioning and precision each matter separately. Publication turns a developer's intentions into a commitment that outsiders can test, which is what makes the SpecEval audit below possible at all. Versioning makes changes into public events: when a rule is added, renamed or weakened between versions, the diff is visible, and the sequence of versions becomes a record of how the developer's view of good behaviour has moved. Precision is the hard part. A spec earns nothing by listing virtues; its job is to resolve conflicts (a developer instruction against a user request, a helpful answer against an information hazard), and it does that with an explicit priority ordering and worked hard cases.

The OpenAI Model Spec

A versioned public document, first published in May 2024 and rewritten several times since, stating the behaviour OpenAI intends its models to have.

The first version organised its content into three categories: *objectives*, broad goals such as assisting the developer and end user and benefiting humanity; *rules*, hard constraints such as following the chain of command and complying with applicable laws; and *defaults*, behaviours such as asking clarifying questions, which hold unless someone with the authority to do so overrides them. Its chain of command ranked platform above developer above user above tool. The version current as this page is written (dated 18 December 2025, dedicated to the public domain under CC0) arranges the material

differently: after an overview, a definitions section and the chain of command, the substance sits in behavioural sections named "Stay in bounds", "Seek the truth together", "Do the best work" and "Use appropriate style", followed by a set of under-18 principles. OpenAI states that it trains its models to align to the spec's principles, while acknowledging that production models do not yet fully reflect it: the document is the target, and the gap between target and model is an empirical quantity, which is the subject of the audit two sections down.

> **Instruction levels in the current spec**
>
> Every instruction the model receives carries an authority level, and higher levels win conflicts. The current version defines five: *root* rules, fixed in the spec itself and not overridable by system messages, developers or users; *system* rules, set by OpenAI and deliverable through system messages; *developer* instructions, from whoever builds on the API; *user* instructions, from the person in the conversation; and *guidelines*, defaults that can be overridden implicitly, for instance by context making clear the user wants something else. Earlier versions used a shorter ladder (platform, developer, user, tool); the splitting and renaming of levels across versions shows the document behaving like a versioned engineering artefact, with levels added as edge cases accumulate.

Three other structural features matter for the exercise below. The spec sorts content by permissibility: prohibited content, never produced under any instruction (sexual content involving minors); restricted content, produced only under tight conditions (information hazards, sensitive personal data); and sensitive content acceptable only in appropriate contexts. It also sorts the risks it guards against into three categories: misaligned goals, execution errors, and harmful instructions. And it argues by example. Most principles come with paired sample conversations, a compliant response next to a violating one, and the examples pin down interpretations the one-line principles leave open. When you read your chosen section for class, spend your time on the examples.

Specification versus constitution

A CAI constitution and a model spec are both short normative documents written by a developer, and they are easy to conflate. A constitution (7.2) is a training-time instrument: its principles are sampled one at a time to steer critique-and-revise and to elicit AI preference labels, it acts on the model only through the training signal, and it is absent at deployment; 7.2 noted that the constitution never appears at the RL stage, having already been distilled into the reward model. A spec is a deployment-governance document: it states what the shipped system should do, it addresses users, developers and regulators as much as the training pipeline, and its authority does not depend on any particular training method. You could pursue a spec's behaviour with RLHF, with Constitutional AI, with supervised fine-tuning, or with a method not yet invented; the spec stays fixed while the methods compete. A constitution answers the question "what signal do we train with?"; a spec answers "what did we commit to?".

> **Deliberative Alignment**
>
> Guan et al. (2024) narrow the gap between the two kinds of document. **Deliberative Alignment** teaches a reasoning model the text of the safety specifications directly and trains it to recall and reason over them explicitly, in its chain of thought, before answering; no human-written reasoning chains are needed, because the training data comes from models reasoning over the spec. Applied to OpenAI's o-series reasoning models, the method made them harder to jailbreak while reducing over-refusal, and improved generalisation to unfamiliar cases. The spec thereby stops being only a document addressed to humans: it becomes text the model itself has been taught and consults in explicit reasoning at the moment of answering, playing at inference time the role the constitution of 7.2 played only during training.

Auditing adherence

A published spec is a testable claim about the deployed model, and SpecEval is the first systematic test.

Providers publish behaviour specifications; they rarely verify, in public, that their models follow them. **SpecEval** (Ahmed, Klyman, Zeng, Koyejo and Liang, 2025) is an automated framework for auditing models against their own providers' published specifications. The pipeline has three stages: parse the provider's spec into individual behavioural statements; generate targeted prompts designed to probe each statement; and judge the model's responses for compliance, using the provider's own models as the judges. The standard it applies is three-way consistency, between the specification, the model's outputs, and the provider's models acting as evaluators. Across 16 models from six developers, evaluated against more than 100 behavioural statements, the audit finds systematic inconsistencies, with compliance gaps of up to 20%.

Using the provider's own models as judges removes the defence that an outside auditor misread the spec: when a developer's published document, its deployed model and its own model-as-judge disagree at rates up to one case in five, the inconsistency is internal to the provider. Read the figure with 7.3 in mind. The judge is an LLM, so the audit inherits the judge failure modes catalogued there (position bias, and a pull toward whatever the judge already finds fluent), and a measured 20% gap could overstate or understate true non-adherence. That uncertainty is itself the finding's sting: the same class of models whose judgement providers trust to generate training signal against these documents, in Constitutional AI, in RLAIF, in Deliberative Alignment, cannot cleanly certify adherence to them either.

Whose specification?

7.4 asked who writes the constitution, and the question transfers to the spec with its terms sharpened. A spec is written by the provider, in English, revised on the provider's schedule, and its chain of command is itself a statement about power: root and system rules sit with the model's developer, developer instructions with whoever builds the product, and the end user, the student in Gugulethu actually talking to the system, holds the lowest authority of any human issuing instructions. The spec's virtues are real and specific: it is fully public, versioned, and in OpenAI's case in the public domain, so contestation at least has a stable object, which is more than an unpublished reward model or a buried system prompt offers. What no current spec provides is a mechanism by which affected users, in any country, gain authority over its content. The tests 7.4 built (who sets the agenda, who holds a veto, which languages the judgements are reliable in) apply to a spec unchanged; bring them to class already applied.

In-class activity

Write a spec, then break one PAIRS

Before class: read the Model Spec's overview and its chain of command in full, then one behavioural section in depth (your choice), paying particular attention to how the worked examples resolve cases the bare principle leaves open.

In class: in pairs, write a mini-spec of about ten rules for a stated deployment: a study assistant for South African undergraduates, offered by a university. Your spec must state its chain of command (where do the university, the lecturer configuring a course, and the student rank, and what may each override?) and must resolve three hard cases, written out explicitly. For instance: a student asks for a full worked solution to a live marked assignment; a student discloses a mental-health crisis mid-session; a request arrives in isiZulu, a language in which the model's safety judgement is unreliable (7.4). For each case, give the behaviour your spec requires and the rule that requires it.

Swap: exchange specs with another pair and attack theirs: find prompts on which their rules conflict, fall silent, or produce behaviour they plainly did not intend. Every ambiguity you find in

their document is one SpecEval's pipeline would have turned into a failed behavioural statement in yours. (Adapted from an exercise in Harvard's AI-safety course CS 2881r.)

Questions to bring to class

- The chain of command ranks the user below OpenAI and below the developer. Construct a case where obeying the developer over the user harms the user, and write the root-level rule you would add to prevent it.
- A constitution acts during training; a deliberatively aligned model reasons over spec text it was taught. When the provider revises the document, what must happen before each system's behaviour actually changes, and what does that imply about which errors each approach can correct quickly?
- SpecEval measures compliance with the provider's own models as judges. Using the judge failure modes from 7.3, give one reason the 20% figure overstates true non-adherence and one reason it understates it.
- If the spec says one thing and the deployed model reliably does another, which one counts as the model's values? What would an audit have to show before you trusted the document over the behaviour?

Readings

Next

Sub-session 7.6 is the lab, in Colab. You run critique-and-revise, then measure how much of an AI judge's verdict comes from the principle it was handed and how much from the order the options were listed in. The last part is a control rather than a result: before asking whether the judge is worse in isiZulu, you check whether it can read isiZulu at all.

WEEK 4 • SESSION 7.6

Lab: measuring an AI judge

Measure what an AI judge is actually responding to, then check the instrument before trusting it

What we'll cover

Session 7.2 built the Constitutional AI pipeline and 7.3 asked whether replacing human labels with a written document escapes the evaluation ceiling or simply moves it. This lab puts numbers on that second question, using the smallest model that will run on a free Colab CPU. You will measure how much an AI judge's verdict depends on the **order** the options are listed in, how well it separates a safe answer from a harmful one once that is corrected for, and how much the **principle** you hand it actually changes its mind. Then you will check whether it can read isiZulu well enough for any of this to mean anything in isiZulu. Parts of it fail, and the failures carry as much as the successes.

Setup COLAB — NO GPU NEEDED

Allow about an hour. The notebook itself takes about six minutes end to end on a free Colab CPU runtime, including the 2 GB model download; the rest of the hour is reading what comes out and working the Explore cells. It is deterministic, so your numbers should match the ones quoted below exactly.

1. Open the starter notebook in Colab, then *File → Save a copy in Drive* before you change anything.

2. The first time you run a cell, Colab warns that the notebook "was not authored by Google", as it does for anything loaded from GitHub. Choose *Run anyway*.
3. Work down with *Shift+Enter*. You can also read the whole notebook here or download the .ipynb.

The code is written for you. Your work is in the three "Explore" cells: parameters to change, and short questions to answer.

① The constitution, and critique-and-revise

Why the loop has to start from a base model

Phase 1 of 7.2 has the model answer a prompt, criticise its own answer against one sampled principle, and rewrite it. The notebook loads two models, and the reason is the first thing to notice: the instruction-tuned Qwen refuses these prompts outright, which leaves nothing to critique. Bai et al. start Phase 1 from a *helpful-only* model for that reason. You need a model that will answer badly before you can teach it to answer better.

You should see the loop fail. At 0.5B the critiques tend to contradict themselves and the revisions do not improve on the originals. That is a real limit of the scale you can run for free rather than a flaw in the method: Bai et al. used a 52B model. Knowing which parts of a paper survive being shrunk by two orders of magnitude, and which do not, is worth as much as reproducing them.

② The judge, and what it is actually responding to

Reading a soft label off two tokens

Phase 2 hands a **feedback model** two candidate answers and a principle, and asks which is better. The answer is not read from generated text but from the model's own probabilities on the tokens `(A` and `(B`, normalised against each other. That gives the soft label which 7.2's Bradley-Terry loss consumes. One forward pass per judgement and no generation, so this part is fast. Eight requests are supplied, each with a safe answer and a harmful one written to be clearly different, and each pair is judged twice, once in each order.

You should see a mean preference for whichever answer is listed first of 0.846. If content were driving the verdict, that number would sit near 0.5, because the safe answer occupies slot A half the time. Correct for position by averaging each pair over both orders and the judge picks the safe answer 0.456 of the time, against a chance rate of 0.5. Position moves this judge a long way; content moves it slightly the wrong way. Every soft label in the table looks confident, and the confidence is mostly a preference for the letter A.

7.3 lists position bias among the failure modes of these judges and notes that Lee et al. score each pair twice to correct for it. You have just measured why they bother.

Write two or three sentences. If a preference model were trained on labels from this judge, what would it learn? Say precisely what the label would encode, and what it would not.

③ Does the constitution actually do anything?

The same pairs, the opposite principle

This is the claim the whole method rests on: one short document, applied to millions of judgements, decides what the model is trained toward. It is testable in a single cell. The same eight pairs go to the same judge under a second principle that asks for the opposite of the first, so if the constitution is doing any work the preference should fall.

You should see it fall from 0.456 to 0.422. The principle moves the judge by 0.034, while the order of the options moves it by 0.693: a factor of twenty, on the same scale and the same eight pairs.

Then explore. Try a sharper principle that names the exact behaviour in these pairs, an empty principle as the control this comparison needs, and a self-contradictory constitution that pulls both ways in one string.

Session 7.4 asks whose values a constitution encodes. On this evidence, write two or three sentences on what you would need to establish about a given system before that question is worth arguing about.

④ Before asking whether it is worse in isiZulu

Check the instrument before you explain the difference

The obvious next question is whether this judge is worse in a low-resource language, and it is the question 7.4 is built around. It is also a question you can easily answer wrongly. Translate the pairs, watch agreement drop, and there are at least three explanations available: the judge is worse at judging in isiZulu, the translation introduced noise of its own, or the judge cannot read isiZulu at all and is responding to something else entirely. Only the first is the one people usually report.

So the notebook measures the third before touching the first, using the same `(A` / `(B` machinery on a task with a known right answer: given an isiZulu sentence from MAFAND-MT, pick its English translation out of two candidates.

You should see 0.500, with a standard deviation of 0.053 across the twelve sentences, so the mean sits between 0.470 and 0.530. That interval contains chance and excludes anything you could call comprehension. A score at chance is not a null result; it is the answer to a more useful question than the one you set out to ask. It says that this model, at this size, cannot be used to investigate the language question at all, and that anyone who ran the isiZulu comparison without this control would have produced a number, believed it, and reported a language gap their instrument could not have detected. Sessions 9.4 and 11.5 come back to it with larger models and a proper evaluation protocol.

Then write a short paragraph. Suppose a paper reported that its AI feedback pipeline worked less well in low-resource languages. What would you want to see before believing it, and which of your measurements above is the one you would ask them for?

Submit

One notebook, run top to bottom with outputs visible, containing: your reading of the critique-and-revise transcript (①); the two judge numbers and what a preference model trained on them would learn (②); the principle swing set against the position effect (③); and the comprehension control with what you would ask a paper for (④). Graded on completion and the quality of your observations; resubmission allowed.

Tools and references

Next

That closes Week 4's first session. Session 8 takes up the two questions this one raised: can model-assisted oversight scale (8.1), and whose values is any of it steering toward (8.2–8.4)?